МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Донецкий национальный технический университет»

Факультет интеллектуальных систем и программирования
Кафедра «Программная инженерия» им. Л.П. Фельдмана

# ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту
по дисциплине «Программирование систем с серверами баз данных»

Тема:
«Создание клиент-серверной информационной системы средствами СУБД»
Вариант Б-16: учёт услуг связи интернет-провайдеров

Руководители:
ст. преп. каф. ПИ Щедрин С.В.
асс. каф. ПИ Филипишин Д. А.

Разработал:
ст. гр. ПИ-24б
Ольховский Д. Д.

Донецк – 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНЕЦКИЙ НАЦИОНАЛЬНЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Факультет Интеллектуальных систем и программирования

Кафедра «Программная инженерия» им. Л.П. Фельдмана

Утверждаю
зав. кафедрой ПИ им. Л.П. Фельдмана
проф. Зори С.А.

_____
__. __.2026 г.

**ТЕХНИЧЕСКОЕ ЗАДАНИЕ**
**на курсовой проект по дисциплине**
**«Программирование систем с серверами баз данных»**

**Тема: «Создание клиент-серверной информационной системы средствами СУБД»**

выдано студенту группы ПИ-24б Ольховскому Дмитрию Дмитриевичу

Руководители курсового проекта: ст. пр. Щедрин С.В., асс. Филипишин Д. А.

**Описание предметной области:**

16. Для автоматизации учета предоставления услуг связи фирмами-провайдерами Internet необходима информация о фирмах-провайдерах (название, тип собственности (государственный, частный, ЗАО, ОАО,…), адрес, телефон, год начала работы), о заключенных с ними договорах (абонент (ФИО или название предприятия, тип (частное лицо, школа, ВУЗ, предприятие, …), физический адрес, IP-адрес, телефон), дата подключения, стоимость подключения, стоимость 1 Мб информации) и о сеансах модемной связи (дата и время сеанса, количество Мб, тариф (бизнес-время — 100% оплата, вечерний тариф — скидка 20%, ночной тариф — скидка 50%,…)).

Примечание: один абонент может заключать несколько договоров.

Донецк – 2026

## Задание на курсовой проект

1. Спроектировать концептуальную модель базы данных (БД) для заданной предметной области и представить ее в виде взаимосвязанных таблиц, находящихся в третьей нормальной форме (в случае денормализации БД – обосновать необходимость). Выделить базовые таблицы и таблицы-справочники, указать для них первичные и внешние ключи.
2. Создать базу данных в среде СУБД средствами языка SQL. Добавить таблицы, домены, индексы.
3. Разработать не менее шести триггеров (по одному для каждого типа события), как минимум для двух различных таблиц БД.
4. Заполнить таблицы БД с использованием соответствующих запросов на языке SQL (не менее десяти записей в каждом справочнике, не менее 10 000 - 50 000 псевдослучайных записей в таблицах).
5. Сформулировать следующие виды запросов:
   - симметричное внутреннее соединение с условием (два запроса с условием отбора по внешнему ключу, два – по датам);
   - симметричное внутреннее соединение без условия (три запроса);
   - левое внешнее соединение;
   - правое внешнее соединение;
   - запрос на запросе по принципу левого соединения;
   - итоговый запрос без условия;
   - итоговый запрос без условия с итоговыми данными вида: «всего», «в том числе»;
   - итоговые запросы с условием на данные (по значению, по маске, с использованием индекса, без использования индекса);
   - итоговый запрос с условием на группы;
   - итоговый запрос с условием на данные и на группы;
   - запрос на запросе по принципу итогового запроса;
   - запрос с использованием объединения

- запросы с подзапросами (с использованием in, not in, case, операциями над итоговыми данными).

6. Запросы без параметров реализовать в виде представлений, остальные запросы – в виде хранимых процедур и/или функций. Создать, по меньшей мере, одно модифицируемое представление, используя механизм триггеров. ВСЯ логика проектируемого ПО – на сервере.

7. Разработать клиентское приложение, которое предоставляет следующие возможности для работы с созданной базой данных: многопользовательский режим работы (одна программа для всех ролей – ситуативный доступ к интерфейсу)

- наличие нескольких ролей пользователя (администратор – добавление/удаление/редактирование пользователей, их прав/ролей; пользователи_1 – …, пользователи _2 – ...)
- просмотр содержимого таблиц и представлений (здесь и далее – с учетом прав пользователей);
- добавление, редактирование и удаление записей таблиц и модифицируемых представлений;
- работа с наборами данных, находящимися в отношении «один-ко-многим» (создать составную форму для просмотра и редактирования данных родительской и дочерней таблиц);
- поиск и фильтрация данных отображаемых таблиц;
- просмотр результатов выполнения запросов;
- визуализация результатов одного из итоговых запросов (диаграммы, экспорт в Excel).

8. Обеспечить защиту данных, информации от несанкционированного доступа, сделать защиту на уровне строк, выполнить партицирование одной из основных таблиц

**Рекомендуемое содержание пояснительной записки:**

Титульный лист

Техническое задание

Реферат

Содержание

Введение

1. Описание предметной области, постановка задачи

2. Обоснование выбора СУБД, описание возможностей СУБД

3. Обоснование выбора инструментальные средств для написания клиентской части, проектирование структуры ПО

3.1 Невизуальные компоненты для работы с данными

3.2 Визуальные компоненты отображения данных

3.3 Разработка шаблонов приложений для работы с таблицами базы данных

4. Проектирование базы данных в выбранной СУБД

4.1 Проектирование концептуальной модели БД

4.2 Создание таблиц, доменов, индексов, сиквенсов

4.3 Разработка триггеров

4.4 Организация многоролевого доступа к данным

4.5 Разграничение доступа к данным на уровне строк (в зависимости от роли и логина)

4.6 Партицирование одной из основных таблиц БД

4.7 Проектирование запросов к базе данных

4.8 Создание представлений и хранимых процедур, функций

5. Разработка клиентского приложения

5.1 Формы и компоненты для работы в «роли 1»

5.2 Формы и компоненты для работы в «роли 2»

5.3 …

5.4 Генерация результатов не менее трех итоговых запросов (диаграммы, экспорт в Excel)

6 Тестирование разработанной информационной системы (в т.ч. включая защиту от несанкционированного доступа, одновременную работы с данными, каскадное удаление)

Заключение/выводы и предложения

Список литературы

Приложение А. Листинг шаблонов

Приложение Б. Листинг серверного приложения

Приложение В. Листинг клиентского приложения

Приложение Г. Руководство пользователя

Приложение Д. Руководство суперпользователя

Приложение Е. Руководство администратора

## График выполнения курсового проекта

| Неделя | Работа |
|---|---|
| 1-2 | Выдача и изучение задания |
| 3 | Анализ требований к системе и способов их реализации |
| 4-5 | Проектирование и реализация БД (таблицы, домены, индексы, роли, RLS, партицирование) |
| 6-7 | Создание триггеров и заполнение таблиц БД |
| 8-9 | Создание представлений и хранимых процедур, запросов |
| 10-13 | Разработка клиентского приложения |
| 14 | Тестирование и отладка системы |
| 15 | Оформление пояснительной записки |
| 16-17 | Защита курсового проекта |

Дата выдачи задания: 02.09.2026

Дата защиты курсового проекта: 23.12.2026

**Задание принял:** Ольховский Д.Д. _____

(подпись студента)

**Согласовано:**

Руководители проекта Щедрин Сергей Валерьевич

Филипишин Дмитрий Александрович

# РЕФЕРАТ

Пояснительная записка содержит 128 с., 79 рис., 24 табл., 8 источников, 6 прил.

Объект разработки — учёт услуг связи, оказываемых фирмами-провайдерами Internet: провайдеры, общий реестр абонентов, договоры, сеансы связи и тарифы.

Цель работы — создание клиент-серверной информационной системы, в которой хранение данных, проверка ограничений, расчёты, отчёты и разграничение доступа выполняются средствами СУБД PostgreSQL, а настольное приложение служит тонким клиентом.

В процессе работы построена концептуальная модель предметной области в третьей нормальной форме, создана база данных с доменами, индексами и последовательностями, заполненная псевдослучайными данными, разработаны представления и функции схемы api для запросов всех видов, требуемых заданием, реализованы пять групп пользователей средствами PostgreSQL, ограничение видимости строк по провайдеру пользователя и сброс пароля с временным паролем. Разработано клиентское приложение на Qt 6, которое строит интерфейс по группе пользователя, выводит списки постранично, показывает отчёты с диаграммами и сохраняет их в Excel. Корректность серверной части подтверждена набором из 117 автоматических тестов.

Использованные средства: PostgreSQL 18, PL/pgSQL, pgAdmin 4, Qt 6, C++17, CMake, Qt Charts, QXlsx.

СУБД POSTGRESQL, КЛИЕНТ-СЕРВЕР, РЕЛЯЦИОННАЯ МОДЕЛЬ, НОРМАЛИЗАЦИЯ, РОЛИ, РАЗГРАНИЧЕНИЕ ДОСТУПА, ХРАНИМЫЕ ФУНКЦИИ, ПРЕДСТАВЛЕНИЯ, QT, ИНТЕРНЕТ-ПРОВАЙДЕР

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....11
1 ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ, ПОСТАНОВКА ЗАДАЧИ.....13
1.1 Описание предметной области.....13
1.2 Постановка задачи.....15
2 ОБОСНОВАНИЕ ВЫБОРА СУБД, ОПИСАНИЕ ВОЗМОЖНОСТЕЙ СУБД 17
3 ОБОСНОВАНИЕ ВЫБОРА ИНСТРУМЕНТАЛЬНЫХ СРЕДСТВ ДЛЯ НАПИСАНИЯ КЛИЕНТСКОЙ ЧАСТИ, ПРОЕКТИРОВАНИЕ СТРУКТУРЫ ПО.....20
3.1 Невизуальные компоненты для работы с данными.....21
3.2 Визуальные компоненты отображения данных.....23
3.3 Разработка шаблонов приложений для работы с таблицами базы данных .....25
4 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ В ВЫБРАННОЙ СУБД.....28
4.1 Проектирование концептуальной модели БД.....28
4.2 Создание таблиц, доменов, индексов, сиквенсов.....31
4.3 Разработка триггеров.....40
4.4 Организация многоролевого доступа к данным.....41
4.5 Разграничение доступа к данным на уровне строк (в зависимости от роли и логина).....44
4.6 Партицирование одной из основных таблиц БД.....45
4.7 Проектирование запросов к базе данных.....45
4.7.1 Симметричное внутреннее соединение с условием.....47
4.7.2 Симметричное внутреннее соединение без условия.....50
4.7.3 Внешние соединения и запрос на запросе.....54
4.7.4 Итоговые запросы.....57
4.7.5 Объединение, запрос на запросе и запросы с подзапросами.....62
4.8 Создание представлений и хранимых процедур, функций.....67

5 РАЗРАБОТКА КЛИЕНТСКОГО ПРИЛОЖЕНИЯ.....71
5.1 Формы и компоненты для работы в роли «Оператор».....72
5.2 Формы и компоненты для работы в роли «Руководитель».....73
5.3 Формы и компоненты для работы в роли «Главный аналитик».....75
5.4 Формы и компоненты для работы в роли «Администратор приложения»75
5.5 Формы и компоненты для работы в роли «Администратор БД».....77
5.6 Генерация результатов итоговых запросов (диаграммы, экспорт в Excel)80
6 ТЕСТИРОВАНИЕ РАЗРАБОТАННОЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ 82
6.1 Защита от несанкционированного доступа.....83
6.2 Одновременная работа с данными.....84
6.3 Каскадное удаление.....84
6.4 Корректность запросов и отчётов.....85
ЗАКЛЮЧЕНИЕ.....86
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....88
ПРИЛОЖЕНИЕ А. Листинг шаблонов.....89
ПРИЛОЖЕНИЕ Б. Листинг серверного приложения.....94
ПРИЛОЖЕНИЕ В. Листинг клиентского приложения.....104
ПРИЛОЖЕНИЕ Г. Руководство пользователя.....114
ПРИЛОЖЕНИЕ Д. Руководство суперпользователя.....121
ПРИЛОЖЕНИЕ Е. Руководство администратора.....125

# ВВЕДЕНИЕ

Интернет-провайдер ведёт учёт абонентов, заключённых с ними договоров и сеансов связи, по которым начисляется плата. Когда провайдеров несколько и они объединены общей информационной системой, к учёту добавляются требования разграничения доступа: сотрудник провайдера должен видеть только договоры и сеансы своей фирмы, руководитель — показатели своей фирмы, а аналитический центр объединения — сводные данные по всем провайдерам. Такие требования естественно выполняются на стороне сервера баз данных: если проверка прав, ограничения целостности и расчёты находятся в базе данных, их нельзя обойти, подключившись к ней другой программой.

Целью курсового проекта является создание клиент-серверной информационной системы учёта услуг связи интернет-провайдеров (вариант Б-16) средствами СУБД PostgreSQL, в которой вся логика программного обеспечения размещена на сервере, а клиентское приложение предоставляет пользователю интерфейс в соответствии с его ролью.

Для достижения цели решаются задачи:

1) Проанализировать предметную область, построить концептуальную модель базы данных в третьей нормальной форме и обосновать допущенную денормализацию.

2) Создать базу данных средствами SQL: схемы, домены, таблицы, индексы, последовательности; заполнить справочники и основные таблицы псевдослучайными данными.

3) Реализовать группы пользователей средствами PostgreSQL и ограничить видимость данных ролью и провайдером пользователя.

4) Спроектировать запросы всех видов, требуемых заданием, оформив запросы без параметров представлениями, а запросы с параметрами — функциями.

5) Разработать клиентское приложение на Qt 6 с постраничным выводом, поиском, составной формой «один-ко-многим», отчётами с диаграммами и выгрузкой в Excel.

6) Проверить систему автоматическими тестами, в том числе на защиту от несанкционированного доступа, одновременное изменение данных и каскадное удаление.

Пояснительная записка содержит описание предметной области, обоснование выбора СУБД и инструментальных средств, проектирование базы данных и клиентского приложения, результаты тестирования; в приложениях приведены листинги шаблонов, серверного и клиентского приложений и руководства пользователя, суперпользователя и администратора.

# 1 ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ, ПОСТАНОВКА ЗАДАЧИ

## 1.1 Описание предметной области

Предметная область — учёт услуг доступа в сеть Интернет, оказываемых несколькими фирмами-провайдерами. Информационная система предназначена для регионального объединения провайдеров: сотрудники каждого провайдера ведут договоры и сеансы связи своей фирмы и общий для объединения реестр абонентов, руководители провайдеров анализируют показатели своей фирмы, аналитический центр объединения получает сводные данные по всем провайдерам, администраторы ведут учётные записи пользователей, справочники и обслуживают базу данных.

В системе хранятся сведения:

– о фирмах-провайдерах: название, тип собственности (государственный, частный, ЗАО, ОАО и т. п.), город и адрес, телефон, год начала работы;

– об абонентах: ФИО или название предприятия, тип абонента (частное лицо, школа, вуз, предприятие и т. п.), город и физический адрес, телефон;

– о договорах абонентов с провайдерами: номер договора, дата подключения, стоимость подключения, стоимость 1 Мб информации, выделенный IP-адрес, адрес подключения, контактный телефон и контактное лицо, дата расторжения;

– о сеансах связи: дата и время сеанса, объём переданной информации в мегабайтах, тариф.

Один абонент может заключить несколько договоров, в том числе с разными провайдерами. Абонент хранится в реестре объединения одной записью, а сведения, которые у разных провайдеров различаются (адрес подключения, контактный телефон и лицо), — в договоре. Тариф определяет скидку к стоимости трафика в зависимости от времени сеанса: в бизнес-время оплата составляет 100 %, по вечернему тарифу предоставляется скидка 20 %, по ночному — 50 %; предусмотрены также тарифы выходного дня и тарифы,

назначаемые вручную (праздничный, образовательный, акционный, служебный). Стоимость сеанса вычисляется как произведение объёма, стоимости 1 Мб по договору и доли оплаты по тарифу.

Основные процессы предметной области:

1) Регистрация провайдера и ведение справочников — администратор приложения.

2) Регистрация абонента и заключение договора с провайдером — оператор провайдера.

3) Регистрация сеансов связи с подбором тарифа и расчётом стоимости — оператор (в реальной эксплуатации — загрузка из биллинговой системы провайдера).

4) Расторжение договора — оператор.

5) Анализ выручки, трафика, распределения сеансов по тарифам и типам абонентов по временным отрезкам — руководитель провайдера и главный аналитик объединения.

Задание не определяет ряд деталей, необходимых для работы системы; приняты следующие решения:

– IP-адрес хранится в договоре, а не в сведениях об абоненте: адрес назначается провайдером по конкретному договору, и абонент, заключивший несколько договоров (например, предприятие с несколькими точками подключения), имеет несколько IP-адресов;

– абонент один для всех провайдеров объединения (без дублей по наименованию и телефону), связь абонента с провайдером — договор;

– введены номер договора (уникален в пределах провайдера) и дата расторжения договора;

– для тарифа задаются временное окно и тип дня, по которым тариф назначается сеансу;

– стоимость сеанса и скидка фиксируются в момент регистрации сеанса и не меняются при последующем изменении тарифов и условий договора.

## 1.2 Постановка задачи

Требуется разработать клиент-серверную информационную систему на основе СУБД PostgreSQL, в которой бизнес-логика, проверка ограничений целостности, расчёты, отчёты и разграничение доступа реализованы средствами сервера базы данных, а настольное приложение на Qt выполняет функции тонкого клиента: подключается к серверу под учётной записью пользователя, вызывает функции и представления схемы api и отображает результат. Одна программа обслуживает все роли; набор разделов и инструментов определяется группой пользователя, которую сообщает сервер после входа.

Пользователи системы и их задачи приведены в таблице 1.1.

Таблица 1.1 – Группы пользователей и их задачи

| **Группа** | **Роль PostgreSQL** | **Задачи и видимость данных** |
|---|---|---|
| Оператор | grp_operator | Абоненты, договоры и сеансы связи своего провайдера; ведение договоров и абонентов, которых не обслуживают другие провайдеры |
| Руководитель провайдера | grp_manager | Данные и статистика своего провайдера за выбранный период; только просмотр |
| Главный аналитик | grp_chief | Сводная статистика по всем провайдерам с отбором по провайдеру; только просмотр |
| Администратор приложения | grp_admin | Учётные записи операторов, руководителей и главных аналитиков; справочники, провайдеры, сведения общих абонентов; оперативных данных не видит |
| Администратор БД | grp_dba | Учётные записи администраторов приложения, журнал изменений, состояние таблиц и подключений, резервное копирование; данных приложения через программу не видит |

Требования к системе определены техническим заданием и методическими рекомендациями [5]: модель данных в третьей нормальной форме; домены, индексы, не менее шести триггеров; не менее 10 записей в каждом справочнике и 10–50 тыс. записей в основных таблицах; запросы перечисленных в задании видов, оформленные представлениями и функциями; модифицируемое представление; роли пользователей, защита на уровне строк и секционирование одной из основных таблиц; клиентское приложение с постраничным

просмотром, поиском, составной формой «один-ко-многим», отчётами с диаграммами и выгрузкой в Excel.

# 2 ОБОСНОВАНИЕ ВЫБОРА СУБД, ОПИСАНИЕ ВОЗМОЖНОСТЕЙ СУБД

В качестве сервера баз данных выбрана СУБД PostgreSQL 18 — свободно распространяемая объектно-реляционная СУБД с открытым исходным кодом [3, 4, 6]. Выбор определён требованиями задания (вся логика программного обеспечения — на сервере, роли пользователей — средствами СУБД, защита на уровне строк, секционирование) и возможностями, которые PostgreSQL предоставляет для их выполнения:

– роли и группы ролей: пользователь системы — роль с правом входа, член группы; права на схемы, таблицы и функции выдаются группам командами GRANT и REVOKE; пароли хранятся в системном каталоге в виде хеша SCRAM-SHA-256;

– функции на SQL и PL/pgSQL, выполняемые с правами владельца (SECURITY DEFINER) и с заданными параметрами сеанса (search_path, часовой пояс), — через них пользователи получают доступ к данным, не имея прав на таблицы;

– политики защиты на уровне строк (ROW LEVEL SECURITY), действующие и на владельца таблицы при включённом режиме FORCE;

– декларативное секционирование таблиц по диапазону, списку и хешу, в том числе многоуровневое;

– домены, проверочные ограничения, внешние ключи с выбором правила удаления, столбцы идентификаторов (GENERATED AS IDENTITY) и последовательности;

– типы данных предметной области: numeric для денежных сумм без ошибок двоичного округления, inet для IP-адресов и подсетей, timestamptz для моментов времени, jsonb для слабоструктурированных документов;

– оконные функции, группировка с ROLLUP, общие табличные выражения, частичные индексы и индексы по выражениям, пропуск по первой колонке составного индекса (Skip Scan, появился в версии 18);

– инструменты анализа плана выполнения запросов (EXPLAIN ANALYZE, модуль auto_explain) и сбора статистики.

Сервер установлен на компьютере разработчика под управлением Windows 11 (служба postgresql-x64-18, порт 5432). Параметры, влияющие на использование памяти и ресурсов, изменены командой ALTER SYSTEM (запись в файл postgresql.auto.conf) с последующим перезапуском службы; значения приведены в таблице 2.1.

Таблица 2.1 – Параметры сервера PostgreSQL

| **Параметр** | **Значение** | **Обоснование** |
|---|---|---|
| shared_buffers | 512MB (было 128MB) | Для Windows рекомендуется 64–512 МБ с опорой на кэш операционной системы; компьютер не является выделенным сервером |
| work_mem | 64MB (было 4MB) | Свободная память около 10 ГБ, до 35 одновременных запросов по две операции сортировки или хеширования: (10 ГБ / 2 − 0,5 ГБ) / 70 ≈ 66 МБ |
| temp_buffers | 32MB (было 8MB) | Верхняя граница рекомендуемого диапазона 16–32 МБ для временных таблиц |
| max_prepared_transactions | 5 (было 0) | Возможность двухфазной фиксации транзакций; требует перезапуска сервера |
| max_files_per_process | 1000 | Значение по умолчанию задано явно; ошибок нехватки дескрипторов файлов нет |
| track_counts | on | Сбор статистики обращений к таблицам, необходимый для автоочистки и представлений pg_stat_* |

Так как функции смены и сброса пароля передают новый пароль серверу параметром запроса, для журнала сервера установлено log_parameter_max_length = 0: значения параметров не записываются в журнал ни для медленных, ни для ошибочных запросов. Пароли администраторов базы данных меняются командой \password программы psql, которая отправляет на сервер уже вычисленный хеш.

Кроме реляционных таблиц PostgreSQL хранит слабоструктурированные данные в типе jsonb — двоичном представлении документа JSON с индексируемым доступом к ключам. В предметной области такие данные — учётные записи сеансов, которые биллинг провайдера выгружает по каждому сеансу: набор атрибутов зависит от производителя оборудования доступа и меняется со сменой оборудования, поэтому запись хранится целиком, а нужные поля извлекаются запросами. Для работы с документами СУБД предоставляет операторы извлечения значения по ключу и пути, проверки вхождения образца и наличия ключей, язык путей SQL/JSON (jsonpath), функции разбора документа в строки таблицы и сборки документа из строк, изменения части документа. Поиск ускоряют обратные индексы GIN по документу целиком и индексы B-дерева по выражению над отдельным ключом. На 100 025 учётных записях, построенных из данных системы в лабораторной работе № 7, индексы сократили время типовых запросов с десятков миллисекунд до долей миллисекунды. В самой системе тип jsonb предназначен для журнала изменений audit.change_log: строка до и после изменения хранится документом, поэтому журнал не зависит от структуры таблиц.

Секционирование позволяет хранить большую таблицу частями (секциями) по значению ключа: запросы за период читают только нужные секции, а устаревшие данные отсоединяются целиком. Для системы кандидатом является таблица сеансов связи — единственная неограниченно растущая таблица; её первичный ключ уже включает дату сеанса, как того требует секционирование по дате. Применение секционирования описывается в подразделе 4.6.

Для администрирования используется pgAdmin 4 (просмотр объектов, построение схемы данных, выполнение запросов) и программа psql (сценарии создания и пересборки базы, автоматические тесты).

# 3 ОБОСНОВАНИЕ ВЫБОРА ИНСТРУМЕНТАЛЬНЫХ СРЕДСТВ ДЛЯ НАПИСАНИЯ КЛИЕНТСКОЙ ЧАСТИ, ПРОЕКТИРОВАНИЕ СТРУКТУРЫ ПО

Клиентское приложение разработано на языке C++17 с библиотекой Qt 6 (модули Widgets, Sql, Charts, PrintSupport, Svg) и системой сборки CMake [7]. Выбор обусловлен следующим:

– модуль Qt Sql содержит драйвер QPSQL, работающий через клиентскую библиотеку PostgreSQL libpq, и позволяет выполнять подготовленные запросы с привязкой параметров (prepare и bindValue), что исключает внедрение SQL-кода через пользовательский ввод;

– архитектура «модель — представление» Qt (QAbstractTableModel, QTableView) отделяет загрузку данных от их отображения, что позволяет вынести постраничную загрузку из функций сервера в один общий класс модели;

– Qt Charts строит диаграммы отчётов, модуль PrintSupport печатает их, а библиотека QXlsx [8] сохраняет отчёты в формате Excel без установленного Microsoft Office;

– формы проектируются в Qt Designer, а оформление задаётся таблицей стилей Qt (QSS) — единый стиль всех экранов по согласованному макету интерфейса;

– приложение компилируется в исполняемый файл Windows без среды выполнения, для запуска достаточно библиотек Qt и libpq рядом с программой.

Приложение — тонкий клиент: оно подключается к PostgreSQL под учётной записью пользователя, вызывает функции и представления схемы api и отображает результат. SQL-текст в клиенте формируется только вида «SELECT * FROM api.<функция>(<параметр> => :<параметр>, …)» с проверкой имени объекта; значения передаются параметрами. Пароль не сохраняется: в файле config.ini рядом с программой хранятся только адрес сервера, порт, имя базы и последний логин. Структура исходного кода приведена в таблице 3.1.

Таблица 3.1 – Структура исходного кода клиента

| Каталог | Содержимое |
|---|---|
| src/app | Параметры подключения (AppConfig), состав разделов и навигации по группе пользователя (RoleConfig), ключи отладочной сборки (DevOptions) |
| src/db | Подключение к серверу (Db) и вызов функций и представлений api (ApiCall) |
| src/models | Модель постраничного списка PagedTableModel |
| src/ui/widgets | Общие визуальные компоненты: таблица со страницами DataGrid, панель страниц PagerBar, элементы форм FormKit |
| src/ui/pages | Страницы разделов: начальная, абоненты, договоры, сеансы связи, провайдеры, статистика, пользователи, страницы администратора БД |
| src/ui/dialogs | Карточки договора и абонента, формы выбора, создание пользователя, смена и сброс пароля |
| src/charts, src/export | Диаграммы отчётов (ReportChart) и выгрузка в Excel (ExcelExport) |

## 3.1 Невизуальные компоненты для работы с данными

Класс Db создаёт единственное подключение QPSQL под логином и паролем пользователя, после подключения вызывает функцию api.whoami и заполняет структуру Whoami (логин, ФИО, группа, название роли, провайдер, версия сервера); по ней строится весь интерфейс. Метод Db::userMessage извлекает из ошибки сервера текст сообщения RAISE EXCEPTION без служебных префиксов драйвера — пользователь видит, например, «Недостаточно прав для этого действия».

Класс ApiCall — единственный путь клиента к данным. Метод select формирует вызов функции api с именованными параметрами, проверяя, что имена функции и параметров состоят только из строчных латинских букв, цифр и знака подчёркивания, и передаёт значения через bindValue (рисунок 3.1). Методы selectView и selectViewRow читают страницу представления и одну строку представления по коду.

```
bool ApiCall::select(const QString &function, const QList<ApiParam> &params, QSqlQuery *query, QString
*error)
{
    QStringList names{function};
    QStringList args;
    for (const ApiParam &p : params) {
        names << p.name;
        args << QStringLiteral("%1 => :%1").arg(p.name);
    }
    if (!checkIdentifiers(names, error))
        return false;
    return run(QStringLiteral("SELECT * FROM api.%1(%2)").arg(function, args.join(QStringLiteral(", "))),
            params, query, error);
}
```

Рисунок 3.1 – Вызов функции схемы api с именованными параметрами (ApiCall::select)

Модель PagedTableModel загружает страницу списка: к параметрам отбора добавляются p_limit и p_offset, общее число строк берётся из колонки total_count первой строки результата, итоговая строка «Итого…» формируется из полей, посчитанных сервером оконными функциями по всему отбору (рисунок 3.2). Код строки (id) в колонки не выводится, а возвращается моделью в роли Qt::UserRole — идентификаторы записей пользователю не показываются.

```
bool PagedTableModel::reload()
{
    QSqlQuery q;
    bool ok;
    if (m_isView) {
        ok = ApiCall::selectView(m_source, m_orderBy, m_limit, m_offset, &q, &m_error);
    } else {
        QList<ApiParam> params = m_filter;
        params << ApiParam{QStringLiteral("p_limit"), m_limit} << ApiParam{QStringLiteral("p_offset"), m_offset};
        ok = ApiCall::select(m_source, params, &q, &m_error);
    }
    if (!ok) {
        clear();
        emit failed(m_error);
        return false;
    }

    beginResetModel();
    m_rows.clear();
    m_totalsText.clear();
    m_total = 0;
    const QSqlRecord rec = q.record();
    while (q.next()) {
        QVariantMap row;
        for (int i = 0; i < rec.count(); ++i)
            row.insert(rec.fieldName(i), q.value(i));
        m_rows << row;
    }
    if (!m_rows.isEmpty()) {
        m_total = m_rows.first().value(QStringLiteral("total_count")).toLongLong();
        …
}
```

Рисунок 3.2 – Загрузка страницы списка (PagedTableModel::reload, начало)

Класс AppConfig читает и сохраняет параметры подключения в config.ini, класс RoleConfig по имени группы возвращает набор разделов главного окна и ссылки панели навигации каждого раздела, класс ExcelExport сохраняет табличный документ отчёта в файл .xlsx.

## 3.2 Визуальные компоненты отображения данных

Интерфейс выполнен по макету, согласованному с преподавателем в лабораторной работе № 4. Окно входа «Доступ к информационной базе» содержит поля пользователя и пароля и раскрывающийся блок параметров подключения (рисунок 3.3).

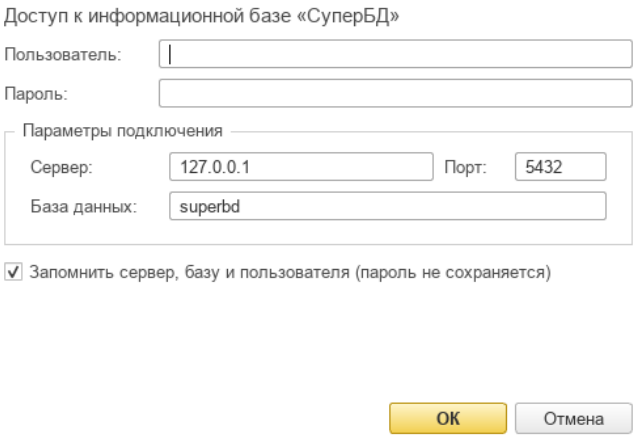



Рисунок 3.3 – Окно входа в программу

Главное окно (рисунок 3.4) состоит из панели разделов слева (набор разделов зависит от группы пользователя), панели навигации раздела, вкладок открытых форм и строки состояния. Начальная страница показывает сведения о пользователе и подключении, описание роли и блок «Текущие дела» — счётчики, открывающие соответствующий список или отчёт.

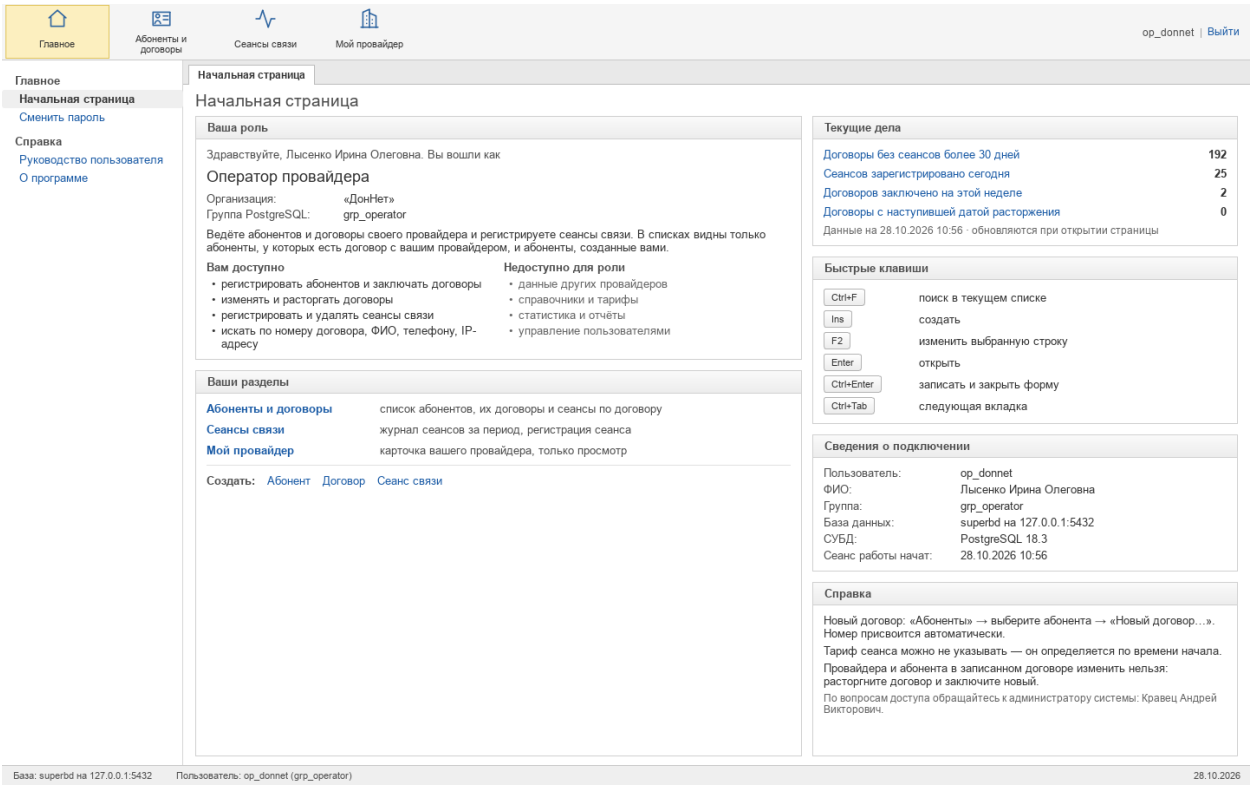



Рисунок 3.4 – Главное окно программы (оператор, начальная страница)

Общие визуальные компоненты:

– DataGrid — таблица (QTableView) с моделью PagedTableModel и панелью страниц; выделяет первую строку после загрузки, сообщает о выборе строки и двойном щелчке, показывает ошибки сервера;

– PagerBar — панель «Показывать по: 20 / 100 / 200, Страница N из M, Записи A–B из N»;

– FormKit — заголовок формы, командная панель с кнопками и полем поиска, подписи, поле с кнопкой выбора «…», сообщения в строке состояния;

– PickerDialog — форма выбора записи (абонента, договора, провайдера) из постраничного списка;

– ReportChart — диаграмма отчёта на основе Qt Charts: накопительные столбцы, линии или горизонтальные столбцы и легенда в отдельной рамке.

## 3.3 Разработка шаблонов приложений для работы с таблицами базы данных

Для работы с таблицами базы данных разработаны три шаблона страниц.

Шаблон списка: заголовок формы, командная панель, строка параметров отбора, таблица DataGrid, связанная с функцией api, и подсказка. Параметры отбора передаются модели как параметры функции, после изменения отбора загружается первая страница. По этому шаблону построены страницы «Договоры», «Сеансы связи», «Провайдеры», «Пользователи», «Общие абоненты», «Договоры без сеансов» и страницы администратора БД. Построение страницы по шаблону показано на рисунке 3.5.

```
IdleContractsPage::IdleContractsPage(const Whoami &me, QWidget *parent)
    : QWidget(parent)
{
    Q_UNUSED(me);
    QVBoxLayout *layout = FormKit::formLayout(this, QStringLiteral("Договоры без сеансов"));

    FormKit::CommandBar bar(this);
    QPushButton *open = bar.button(QStringLiteral("Открыть договор"), true);
    QPushButton *refresh = bar.button(QStringLiteral("Обновить"));
    bar.stretch();
    bar.more();
    layout->addLayout(bar.layout());

    auto *params = new QHBoxLayout;
    params->setSpacing(6);
    params->addWidget(FormKit::caption(QStringLiteral("Без сеансов более:"), this));
    m_days = new QSpinBox(this);
    m_days->setRange(1, 3650);
    m_days->setValue(30);
    m_days->setFixedWidth(80);
    params->addWidget(m_days);
    params->addWidget(FormKit::caption(QStringLiteral("дней"), this));
    params->addStretch(1);
    layout->addLayout(params);

    m_grid = new DataGrid(this, true, 20);
    m_grid->setFunction(QStringLiteral("q_idle_contracts"),
        {{QStringLiteral("contract_no"), QStringLiteral("Номер договора"), ColumnSpec::Text, 0, 130},
         {QStringLiteral("subscriber_name"), QStringLiteral("Абонент"), ColumnSpec::Text, 0, 300},
         {QStringLiteral("phone"), QStringLiteral("Телефон"), ColumnSpec::Text, 0, 150},
         {QStringLiteral("connected_on"), QStringLiteral("Дата подключения"), ColumnSpec::Date, 0, 130},
         {QStringLiteral("last_session"), QStringLiteral("Последний сеанс"), ColumnSpec::DateTime, 0, 140},
         {QStringLiteral("idle_days"), QStringLiteral("Дней без сеансов"), ColumnSpec::Integer, 0, 0}});
    layout->addWidget(m_grid, 1);
    layout->addWidget(FormKit::hint(QStringLiteral("Действующие договоры, заключённые раньше границы и
без сеансов после неё; "
                                                   "пустой «Последний сеанс» — сеансов не было."),
                                    this, true));

    auto openCard = [this] {
        const QVariant id = m_grid->currentId();
        if (id.isValid() && !id.isNull())
            ContractDialog(id, this).exec();
    };
    connect(open, &QPushButton::clicked, this, openCard);
    connect(m_grid, &DataGrid::activated, this, openCard);
    connect(refresh, &QPushButton::clicked, this, &IdleContractsPage::reload);
    connect(m_days, &QSpinBox::editingFinished, this, &IdleContractsPage::reload);
    reload();
}
```

Рисунок 3.5 – Страница списка по шаблону (IdleContractsPage)

Составная форма «один-ко-многим» содержит несколько DataGrid, связанных выбором строки: при выборе абонента загружаются его договоры, при

выборе договора — сеансы договора с итогами. По этому шаблону построена страница «Абоненты» (подраздел 5.1).

Шаблон отчёта: строка параметров (период, периодичность, провайдер, тип абонента), табличный документ с итоговой строкой и диаграмма ReportChart, команды «Сохранить в Excel…» и «Печать». Данные отчёта целиком формирует функция api.report_*; клиент только раскладывает строки результата по колонкам и рядам диаграммы. По этому шаблону построены отчёты раздела «Статистика» (подраздел 5.6).

Исходный код шаблонов приведён в приложении А.

# 4 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ В ВЫБРАННОЙ СУБД

## 4.1 Проектирование концептуальной модели БД

Модель построена по методике IDEF1X [2]. Исходное отношение, содержащее все сведения задания в одной таблице сеансов, приводит к аномалиям обновления (смена телефона провайдера требует изменения всех строк его сеансов), включения (невозможно зарегистрировать провайдера без сеансов) и удаления (удаление последнего сеанса уничтожает сведения о договоре) [1]. Последовательное приведение к первой нормальной форме (атомарность значений, развёртывание повторяющейся группы сеансов), второй (устранение зависимостей реквизитов провайдера и договора от части составного ключа) и третьей (вынесение абонента и тарифа, от которых транзитивно зависят реквизиты абонента и скидка) даёт восемь сущностей предметной области. Концептуальная модель показана на рисунке 4.1.

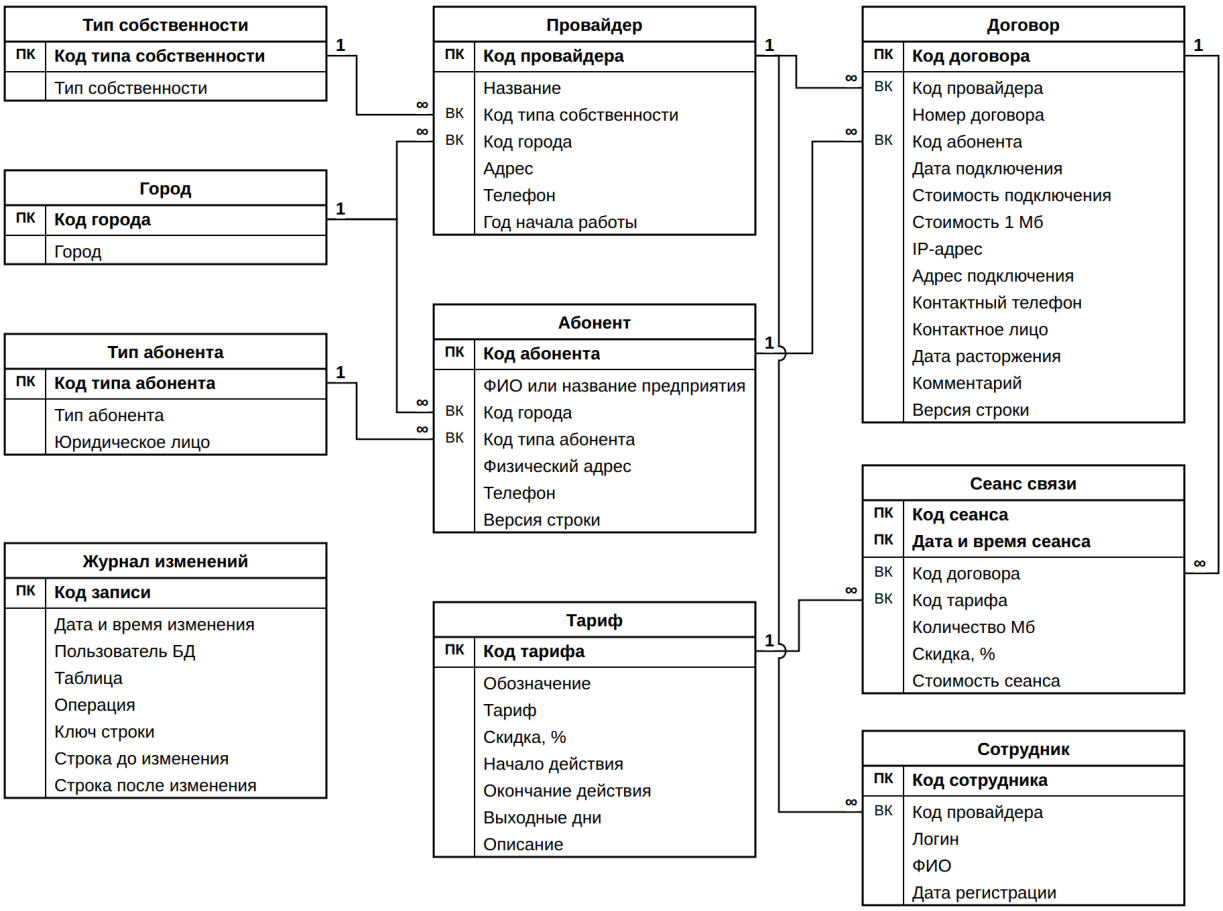



Рисунок 4.1 – Концептуальная модель базы данных

Справочники: тип собственности, город, тип абонента, тариф. Базовые таблицы: провайдер, абонент, договор, сеанс связи. Абоненты образуют общий реестр объединения: одно лицо или предприятие, обслуживаемое несколькими провайдерами, представлено одной записью, а дубли исключает уникальность наименования (без учёта регистра) вместе с цифрами телефона. Провайдер и абонент связаны «многие ко многим» через договор — таблицу пересечения, в которой, кроме номера, цен и IP-адреса, хранятся сведения, различающиеся у разных провайдеров: адрес подключения, контактный телефон и контактное лицо. Для требований к системе добавлены две служебные таблицы: «Сотрудник» связывает учётную запись PostgreSQL с провайдером для разграничения доступа к строкам (паролей и ролей таблица не содержит — роль определяется членством в группе PostgreSQL), «Журнал изменений» предназначен для версий изменённых строк в формате jsonb и внешними

ключами не связан, чтобы записи сохранялись после удаления строк. Пользователи и группы хранятся в системных таблицах pg_authid и pg_auth_members. Таблицы базы данных и связи между ними показаны на рисунке 4.2, описание связей приведено в таблице 4.1.

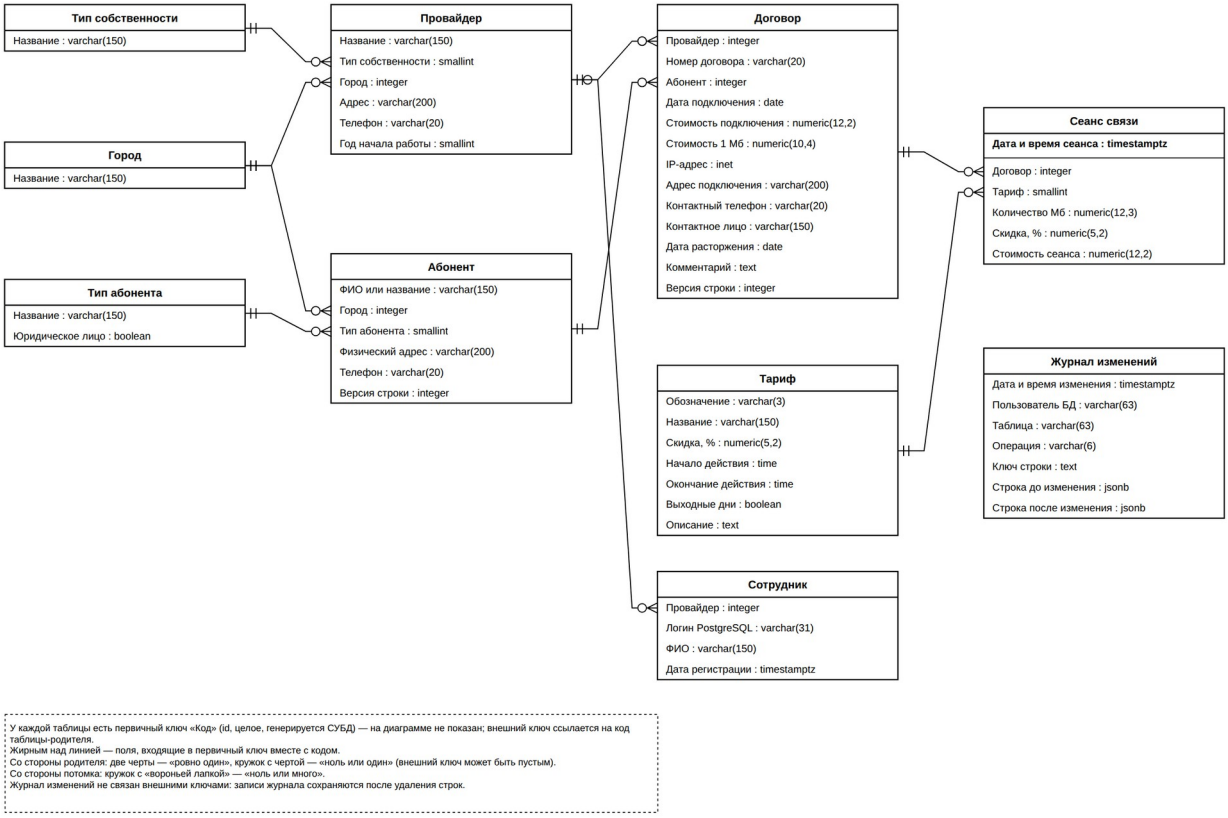



Рисунок 4.2 – Таблицы базы данных и связи между ними

Таблица 4.1 – Связи между сущностями

| № | Родитель | Потомок | Мощность | При удалении родителя |
|---|---|---|---|---|
| R1 | Тип собственности | Провайдер | 1 : 0..N | запрет |
| R2 | Город | Провайдер | 1 : 0..N | запрет |
| R3 | Город | Абонент | 1 : 0..N | запрет |
| R4 | Тип абонента | Абонент | 1 : 0..N | запрет |
| R5 | Провайдер | Договор | 1 : 0..N | запрет |
| R6 | Абонент | Договор | 1 : 0..N | каскадное удаление |
| R7 | Договор | Сеанс | 1 : 0..N | каскадное удаление |
| R8 | Тариф | Сеанс | 1 : 0..N | запрет |
| R9 | Провайдер | Сотрудник | 0..1 : 0..N | запрет |

Первичные ключи всех таблиц — суррогатные целочисленные, генерируемые СУБД. Естественные ключи сохранены как альтернативные: название провайдера и наименования в справочниках уникальны, номер

договора уникален в пределах провайдера, IP-адрес уникален среди действующих договоров провайдера (частичный уникальный индекс). Первичный ключ таблицы сеансов составной — идентификатор и дата сеанса: таблица предназначена для секционирования по дате сеанса (подраздел 4.6), а PostgreSQL требует включения ключа секционирования в первичный ключ.

Модель содержит обоснованную денормализацию: в сеансе хранятся скидка тарифа и стоимость сеанса на момент его регистрации. Скидка транзитивно зависит от тарифа, а стоимость вычисляется из объёма, цены договора и скидки, однако изменение справочника тарифов или условий договора не должно менять стоимость уже оказанных услуг, а отчёты о выручке по сотням тысяч сеансов не должны пересчитывать стоимость с соединением трёх таблиц. Значения задаются при вставке сеанса и не редактируются, поэтому аномалии обновления не возникают; соответствие скидки тарифу и стоимости формуле проверяется автоматическим тестом 3.4.

## 4.2 Создание таблиц, доменов, индексов, сиквенсов

База данных superbd создана в кодировке UTF-8 и разделена на схемы по назначению: ref — справочники и домены, core — основные таблицы предметной области, sec — сотрудники и функции разграничения доступа, audit — журнал изменений, api — функции и представления, через которые работает клиентское приложение. Все объекты принадлежат роли app_owner, у которой нет права входа в систему; права на схемы у роли PUBLIC отозваны, группам пользователей выдаётся доступ только к схеме api.

Структура создаётся последовательностью сценариев миграции (приложение Б), которые выполняются по порядку из сценария db/build.sql: схемы и группы пользователей, домены, справочники, основные таблицы, триггер проверки логина сотрудника, индексы, функции разграничения доступа, представления и функции api, после чего таблицы заполняются данными. Таблицы создаются в порядке «сначала сторона «один», затем подчинённые»;

каждый сценарий можно выполнить повторно. Создание основной таблицы «Договор» показано на рисунке 4.3.

```
CREATE TABLE IF NOT EXISTS core.contract (
    id              integer          GENERATED ALWAYS AS IDENTITY,
    provider_id     integer          NOT NULL,
    contract_no     ref.d_contract_no NOT NULL
        DEFAULT ('Д-' || to_char(current_date, 'YYYY') || '-' || lpad(nextval('core.contract_no_seq')::text, 6, '0')),
    subscriber_id   integer          NOT NULL,
    connected_on    date             NOT NULL DEFAULT current_date,
    connection_fee ref.d_money       NOT NULL,
    price_per_mb    ref.d_price_mb   NOT NULL,
    ip_address      inet             NOT NULL,
    connection_address ref.d_address NOT NULL,
    contact_phone   ref.d_phone,
    contact_person ref.d_name,
    closed_on       date,
    note            text,
    row_version     integer          NOT NULL DEFAULT 1,
    CONSTRAINT pk_contract PRIMARY KEY (id),
    CONSTRAINT uq_contract_provider_id_contract_no UNIQUE (provider_id, contract_no),
    CONSTRAINT fk_contract_provider_id FOREIGN KEY (provider_id)
        REFERENCES core.provider (id) ON DELETE RESTRICT,
    CONSTRAINT fk_contract_subscriber_id FOREIGN KEY (subscriber_id)
        REFERENCES core.subscriber (id) ON DELETE CASCADE,
    CONSTRAINT ck_contract_closed_after_connected CHECK (closed_on >= connected_on),
    CONSTRAINT ck_contract_row_version CHECK (row_version > 0)
);
```

Рисунок 4.3 – Создание таблицы «Договор»

Для каждой таблицы и колонки задан комментарий (COMMENT ON) с названием из концептуальной модели. Ограничения значений заданы доменами и проверочными ограничениями: дата расторжения договора не раньше даты подключения, время начала и окончания действия тарифа задаются вместе. Допустимые символы в доменах проверяются функцией translate, без регулярных выражений. Связи между таблицами реализованы внешними ключами с явно выбранным правилом удаления (таблица 4.1). Внешних ключей на системные таблицы PostgreSQL поставить нельзя, поэтому соответствие логина сотрудника существующей учётной записи PostgreSQL проверяет триггер (подраздел 4.3). Схема данных, построенная в pgAdmin по фактической базе, показана на рисунке 4.4, колонки таблицы «Сеанс связи» в pgAdmin — на рисунке 4.5.

[ВСТАВИТЬ СНИМОК: coursework/pz/img/pga_erd.png]

Рисунок 4.4 – Схема данных базы superbd в pgAdmin

[ВСТАВИТЬ СНИМОК: coursework/pz/img/pga_session_columns.png]

Рисунок 4.5 – Колонки таблицы «Сеанс связи» в pgAdmin

Кроме индексов, которые PostgreSQL создаёт для первичных ключей и ограничений уникальности, проиндексированы все внешние ключи, дата и время сеанса и пара «договор, дата сеанса» для выборок за период, пара «провайдер, дата подключения» договора и пара «абонент, провайдер» договора, по которой проверяется, есть ли у провайдера договор с абонентом. Уникальный индекс по выражениям «наименование в нижнем регистре с классом операторов text_pattern_ops, цифры телефона» не допускает повторной записи абонента и одновременно служит для поиска абонента по началу наименования. Уникальность IP-адреса среди действующих договоров провайдера обеспечивает частичный уникальный индекс с условием closed_on IS NULL. Номер договора по умолчанию формируется из явной последовательности core.contract_no_seq в виде «Д-<год>-<шестизначный номер>»; остальные последовательности созданы СУБД для колонок идентификаторов.

Таблицы заполнены сценариями db/seed: справочники — осмысленными значениями (от 10 до 12 записей), основные таблицы — псевдослучайными данными, сгенерированными средствами SQL (generate_series, random с фиксированным начальным значением setseed, поэтому результат воспроизводим): 30 провайдеров, 8 000 абонентов, 12 000 договоров и 100 000 сеансов связи — всего более 120 000 строк. Сценарий seed/040 добавляет провайдера без договоров и сотрудников, два договора и сеансы за текущий день — без них часть запросов и счётчиков начальной страницы возвращала бы пустой результат. Генератор соблюдает ограничения предметной области: сеансы попадают в период действия договора, тариф выбирается по времени сеанса и типу дня, стоимость сеанса вычисляется по формуле; 5 % абонентов не имеют договоров, а два тарифа не имеют сеансов — это нужно для демонстрации внешних соединений.

Домены приведены в таблице 4.2, структура таблиц — в таблицах 4.3 – 4.12, индексы — в таблице 4.13, последовательности — в таблице 4.14. Таблицы сформированы по фактической схеме базы данных (комментарии COMMENT ON и системный каталог) сценарием tools/schema2md.py.

Таблица 4.2 – Домены

| Домен | Базовый тип | Ограничение | Описание |
|---|---|---|---|
| d_address | varchar(200) | (btrim(VALUE) <> '') | Адрес в пределах города: непустая строка |
| d_contract_no | varchar(20) | ((VALUE = btrim(VALUE)) AND (VALUE <> '')) | Номер договора: непустая строка без пробелов по краям |
| d_login | varchar(31) | ((length(VALUE) >= 3) AND (translate("left"(VALUE, 1), 'abcdefghijklmnopqrstuvwxyz', '') = '') AND (translate(VALUE, 'abcdefghijklmnopqrstuvwxyz0123 456789_', '') = '')) | Логин сотрудника — имя LOGIN-роли PostgreSQL: латиница в нижнем регистре, цифры, «_», 3–31 символ |
| d_mb | numeric(12,3) | (VALUE > 0) | Объём информации, Мб: больше 0 |
| d_money | numeric(12,2) | (VALUE >= 0) | Денежная сумма, руб.: не меньше 0, два знака после запятой |
| d_name | varchar(150) | ((VALUE = btrim(VALUE)) AND (VALUE <> '')) | Наименование: непустая строка без пробелов по краям |
| d_percent | numeric(5,2) | ((VALUE >= 0) AND (VALUE <= 100)) | Процент от 0 до 100 |
| d_phone | varchar(20) | ((translate(VALUE, '+0123456789 ()-', '') = '') AND (strpos(substr(VALUE, 2), '+') = 0) AND (length(ltrim(VALUE, '+')) >= 7)) | Телефон: цифры, пробелы, скобки, дефис, необязательный «+» в начале; 7–20 символов |
| d_price_mb | numeric(10,4) | (VALUE > 0) | Стоимость 1 Мб, руб.: больше 0, четыре знака после запятой |
| d_year | smallint | ((VALUE >= 1970) AND (VALUE <= 2100)) | Год от 1970 до 2100 (не позже текущего — проверка триггером) |

Таблица 4.3 – Структура таблицы «Тип собственности» (ref.ownership_type)

| Колонка | Тип | NULL | Ключ | По умолчанию | Описание |
|---|---|---|---|---|---|
| id | smallint | NOT NULL | PK | identity | Код типа собственности |
| name | d_name | NOT NULL | AK | | Тип собственности |

Таблица 4.4 – Структура таблицы «Город» (ref.city)

| Колонка | Тип | NULL | Ключ | По умолчанию | Описание |
|---|---|---|---|---|---|
| id | integer | NOT NULL | PK | identity | Код города |
| name | d_name | NOT NULL | AK | | Город |

Таблица 4.5 – Структура таблицы «Тип абонента» (ref.subscriber_type)

| Колонка | Тип | NULL | Ключ | По умолчанию | Описание |
|---|---|---|---|---|---|
| id | smallint | NOT NULL | PK | identity | Код типа абонента |
| name | d_name | NOT NULL | AK | | Тип абонента |
| is_legal | boolean | NOT NULL | | false | Юридическое лицо |

Таблица 4.6 – Структура таблицы «Тариф» (ref.tariff)

| Колонка | Тип | NULL | Ключ | По умолчанию | Описание |
|---|---|---|---|---|---|
| id | smallint | NOT NULL | PK | identity | Код тарифа |
| code | varchar(3) | NOT NULL | AK | | Обозначение |
| name | d_name | NOT NULL | AK | | Тариф |
| discount_pct | d_percent | NOT NULL | | 0 | Скидка, % |
| time_from | time without time zone | NULL | | | Начало действия |
| time_to | time without time zone | NULL | | | Окончание действия |
| is_weekend | boolean | NULL | | | Выходные дни |
| description | text | NULL | | | Описание |

Таблица 4.7 – Структура таблицы «Провайдер» (core.provider)

| Колонка | Тип | NULL | Ключ | По умолчанию | Описание |
|---|---|---|---|---|---|
| id | integer | NOT NULL | PK | identity | Код провайдера |
| name | d_name | NOT NULL | AK | | Название |
| ownership_type_id | smallint | NOT NULL | FK | | Код типа собственности |
| city_id | integer | NOT NULL | FK | | Код города |
| address | d_address | NOT NULL | | | Адрес |
| phone | d_phone | NOT NULL | | | Телефон |
| founded_year | d_year | NOT NULL | | | Год начала работы |

Таблица 4.8 – Структура таблицы «Абонент» (core.subscriber)

| Колонка | Тип | NULL | Ключ | По умолчанию | Описание |
|---|---|---|---|---|---|
| id | integer | NOT NULL | PK | identity | Код абонента |
| name | d_name | NOT NULL | | | ФИО или название предприятия |
| city_id | integer | NOT NULL | FK | | Код города |
| subscriber_type_id | smallint | NOT NULL | FK | | Код типа абонента |
| address | d_address | NOT NULL | | | Физический адрес |
| phone | d_phone | NOT NULL | | | Телефон |
| row_version | integer | NOT NULL | | 1 | Версия строки |

Таблица 4.9 – Структура таблицы «Договор» (core.contract)

| Колонка | Тип | NULL | Ключ | По умолчанию | Описание |
|---|---|---|---|---|---|
| id | integer | NOT NULL | PK | identity | Код договора |
| provider_id | integer | NOT NULL | AK, FK | | Код провайдера |
| contract_no | d_contract_no | NOT NULL | AK | ((('Д-' \|\| to_char((CURRENT_DATE), 'YYYY')) \|\| '-') \|\| lpad((nextval('core.contract_no_seq'::regclass)), 6, '0')) | Номер договора |
| subscriber_id | integer | NOT NULL | FK | | Код абонента |
| connected_on | date | NOT NULL | | CURRENT_DATE | Дата подключения |
| connection_fee | d_money | NOT NULL | | | Стоимость подключения |
| price_per_mb | d_price_mb | NOT NULL | | | Стоимость 1 Мб |
| ip_address | inet | NOT NULL | | | IP-адрес |
| connection_address | d_address | NOT NULL | | | Адрес подключения |
| contact_phone | d_phone | NULL | | | Контактный телефон |
| contact_person | d_name | NULL | | | Контактное лицо |
| closed_on | date | NULL | | | Дата расторжения |
| note | text | NULL | | | Комментарий |
| row_version | integer | NOT NULL | | 1 | Версия строки |

Таблица 4.10 – Структура таблицы «Сеанс связи» (core.session)

| Колонка | Тип | NULL | Ключ | По умолчанию | Описание |
|---|---|---|---|---|---|
| id | bigint | NOT NULL | PK | identity | Код сеанса |
| started_at | timestamp with time zone | NOT NULL | PK | | Дата и время сеанса |
| contract_id | integer | NOT NULL | FK | | Код договора |
| tariff_id | smallint | NOT NULL | FK | | Код тарифа |
| volume_mb | d_mb | NOT NULL | | | Количество Мб |

| Колонка | Тип | NULL | Ключ | По умолчанию | Описание |
|---|---|---|---|---|---|
| discount_pct | d_percent | NOT NULL | | | Скидка, % |
| cost | d_money | NOT NULL | | | Стоимость сеанса |

Таблица 4.11 – Структура таблицы «Сотрудник» (sec.app_user)

| Колонка | Тип | NULL | Ключ | По умолчанию | Описание |
|---|---|---|---|---|---|
| id | integer | NOT NULL | PK | identity | Код сотрудника |
| provider_id | integer | NULL | FK | | Код провайдера |
| login | d_login | NOT NULL | AK | | Логин |
| full_name | d_name | NOT NULL | | | ФИО |
| created_at | timestamp with time zone | NOT NULL | | now() | Дата регистрации |
| reset_pending | boolean | NOT NULL | | false | Выдан временный пароль |
| reset_at | timestamp with time zone | NULL | | | Когда выдан временный пароль |
| reset_by | d_login | NULL | | | Кто выдал временный пароль |

Таблица 4.12 – Структура таблицы «Журнал изменений» (audit.change_log)

| Колонка | Тип | NULL | Ключ | По умолчанию | Описание |
|---|---|---|---|---|---|
| id | bigint | NOT NULL | PK | identity | Код записи |
| changed_at | timestamp with time zone | NOT NULL | | now() | Дата и время изменения |
| db_user | varchar(63) | NOT NULL | | SESSION_USER | Пользователь БД |
| table_name | varchar(63) | NOT NULL | | | Таблица |
| operation | varchar(6) | NOT NULL | | | Операция |
| row_pk | text | NOT NULL | | | Ключ строки |
| old_row | jsonb | NULL | | | Строка до изменения |
| new_row | jsonb | NULL | | | Строка после изменения |

Таблица 4.13 – Индексы

| Индекс | Таблица | Колонки и условие | Вид | Назначение |
|---|---|---|---|---|
| pk_change_log | audit.change_log | (id) | первичный ключ | |
| pk_contract | core.contract | (id) | первичный ключ | |
| uq_contract_provider_id_contract_no | core.contract | (provider_id, contract_no) | уникальный | |
| uq_contract_provider_id_ip_address_active | core.contract | (provider_id, ip_address) WHERE (closed_on IS NULL) | уникальный | IP-адрес уникален среди действующих договоров провайдера (D1) |

| Индекс | Таблица | Колонки и условие | Вид | Назначение |
|---|---|---|---|---|
| ix_contract_provider_id_connected_on | core.contract | (provider_id, connected_on) | | Договоры провайдера, заключённые в периоде |
| ix_contract_subscriber_id_provider_id | core.contract | (subscriber_id, provider_id) | | Договоры абонента; есть ли у абонента договор с провайдером (видимость абонента, D25) |
| pk_provider | core.provider | (id) | первичный ключ | |
| uq_provider_name | core.provider | name | уникальный | |
| ix_provider_city_id | core.provider | (city_id) | | Индекс внешнего ключа |
| ix_provider_ownership_type_id | core.provider | (ownership_type_id) | | Индекс внешнего ключа |
| pk_session | core.session | (id, started_at) | первичный ключ | |
| ix_session_contract_id_started_at | core.session | (contract_id, started_at) | | Сеансы договора за период |
| ix_session_started_at | core.session | (started_at) | | Сеансы за период (ключ будущего секционирования) |
| ix_session_tariff_id | core.session | (tariff_id) | | Индекс внешнего ключа |
| pk_subscriber | core.subscriber | (id) | первичный ключ | |
| uq_subscriber_lower_name_phone_digits | core.subscriber | (lower(name) text_pattern_ops, translate((phone), '+ ()-', '')) | уникальный | Абонент в реестре один (наименование без учёта регистра + цифры телефона, D25); поиск по началу наименования |
| ix_subscriber_city_id | core.subscriber | (city_id) | | Индекс внешнего ключа |
| ix_subscriber_subscriber_type_id | core.subscriber | (subscriber_type_id) | | Индекс внешнего ключа |

| Индекс | Таблица | Колонки и условие | Вид | Назначение |
|---|---|---|---|---|
| pk_city | ref.city | (id) | первичный ключ | |
| uq_city_name | ref.city | name | уникальный | |
| pk_ownership_type | ref.ownership_type | (id) | первичный ключ | |
| uq_ownership_type_name | ref.ownership_type | name | уникальный | |
| pk_subscriber_type | ref.subscriber_type | (id) | первичный ключ | |
| uq_subscriber_type_name | ref.subscriber_type | name | уникальный | |
| pk_tariff | ref.tariff | (id) | первичный ключ | |
| uq_tariff_code | ref.tariff | (code) | уникальный | |
| uq_tariff_name | ref.tariff | name | уникальный | |
| pk_app_user | sec.app_user | (id) | первичный ключ | |
| uq_app_user_login | sec.app_user | (login) | уникальный | |
| ix_app_user_provider_id | sec.app_user | (provider_id) | | Индекс внешнего ключа |

Таблица 4.14 – Последовательности

| Последовательность | Тип | Колонка | Вид | Назначение |
|---|---|---|---|---|
| core.contract_no_seq | integer | core.contract.contract_no | явная (OWNED BY) | Последовательность номеров договоров (D12) |
| audit.change_log_id_seq | bigint | audit.change_log.id | identity | Значения суррогатного первичного ключа |
| core.contract_id_seq | integer | core.contract.id | identity | Значения суррогатного первичного ключа |
| core.provider_id_seq | integer | core.provider.id | identity | Значения суррогатного первичного ключа |
| core.session_id_seq | bigint | core.session.id | identity | Значения суррогатного первичного ключа |
| core.subscriber_id_seq | integer | core.subscriber.id | identity | Значения суррогатного первичного ключа |

| Последовательность | Тип | Колонка | Вид | Назначение |
|---|---|---|---|---|
| ref.city_id_seq | integer | ref.city.id | identity | Значения суррогатного первичного ключа |
| ref.ownership_type_id_seq | smallint | ref.ownership_type.id | identity | Значения суррогатного первичного ключа |
| ref.subscriber_type_id_seq | smallint | ref.subscriber_type.id | identity | Значения суррогатного первичного ключа |
| ref.tariff_id_seq | smallint | ref.tariff.id | identity | Значения суррогатного первичного ключа |
| sec.app_user_id_seq | integer | sec.app_user.id | identity | Значения суррогатного первичного ключа |

## 4.3 Разработка триггеров

Триггер — процедура, которую сервер вызывает автоматически при вставке, изменении или удалении строк таблицы. В системе реализован триггер trg_app_user_biu_check_role таблицы «Сотрудник» (рисунок 4.6): перед вставкой строки и перед изменением логина он проверяет, что логин является существующей учётной записью PostgreSQL с правом входа. Триггер заменяет внешний ключ на системный каталог pg_roles, который поставить нельзя. Проверка выполняется автоматическим тестом 4.4: сотрудник с несуществующим логином отклоняется.

```
CREATE OR REPLACE FUNCTION sec.trg_app_user_biu_check_role()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = pg_catalog, pg_temp
AS $$
BEGIN
   IF NOT EXISTS (SELECT 1 FROM pg_catalog.pg_roles r
                  WHERE r.rolname = NEW.login AND r.rolcanlogin) THEN
      RAISE EXCEPTION 'Учётная запись «%» не найдена: сначала создайте пользователя PostgreSQL с
правом входа', NEW.login
         USING ERRCODE = 'P0001';
   END IF;
   RETURN NEW;
END $$;
COMMENT ON FUNCTION sec.trg_app_user_biu_check_role() IS
   'Триггерная функция: логин сотрудника должен существовать в pg_roles и иметь право входа (связь R10)';

DROP TRIGGER IF EXISTS trg_app_user_biu_check_role ON sec.app_user;
CREATE TRIGGER trg_app_user_biu_check_role
   BEFORE INSERT OR UPDATE OF login ON sec.app_user
   FOR EACH ROW EXECUTE FUNCTION sec.trg_app_user_biu_check_role();
COMMENT ON TRIGGER trg_app_user_biu_check_role ON sec.app_user IS
   'Проверка, что логин сотрудника — существующая LOGIN-роль PostgreSQL';
```

Рисунок 4.6 – Триггерная функция и триггер проверки логина сотрудника

[ЗАГЛУШКА: остальные триггеры не реализованы (ТЗ п. 3 — не менее шести, по одному на каждый тип события, для двух и более таблиц). Запланированы в docs/analysis/requirements.md, раздел 5: BEFORE INSERT сеанса — подбор тарифа по времени, фиксация скидки и расчёт стоимости (D2, D3, D13); BEFORE UPDATE договора — неизменность сторон и версия строки (D8, D14); BEFORE DELETE провайдера — запрет при недавних сеансах (D9); AFTER INSERT, UPDATE, DELETE договора — запись в журнал изменений audit.change_log (D10); INSTEAD OF — модифицируемое представление (подраздел 4.8). Описание, рисунки и тесты добавить после реализации]

## 4.4 Организация многоролевого доступа к данным

Роли пользователей реализованы средствами PostgreSQL. Пользователь системы — роль с правом входа (LOGIN), член ровно одной из пяти групп (таблица 4.15); пароль хранит только СУБД. Таблицы пользователей с паролями или ролями в базе нет: группа пользователя определяется по системному

каталогу pg_auth_members, а таблица «Сотрудник» хранит только ФИО и привязку к провайдеру.

Таблица 4.15 – Группы пользователей и их права

| Группа | Права |
|---|---|
| grp_operator | Функции и представления api своих экранов: абоненты, договоры, сеансы своего провайдера, изменение договоров и своих абонентов |
| grp_manager | Те же данные своего провайдера только для чтения, отчёты «Статистики» |
| grp_chief | Данные и отчёты по всем провайдерам или выбранному, только чтение |
| grp_admin | Пользователи операторов, руководителей и главных аналитиков, провайдеры, справочники, общие абоненты |
| grp_dba | Пользователи-администраторы приложения, журнал изменений, таблицы, подключения; встроенные роли pg_monitor, pg_maintain и pg_read_all_data (чтение для резервного копирования); переключение на владельца командой SET ROLE app_owner |

Права выдаются по принципу минимальных привилегий:

– у роли PUBLIC отозваны права на базу данных и все схемы проекта, а командой ALTER DEFAULT PRIVILEGES отозвано право выполнения любых новых функций владельца;

– группы получают право подключения к базе, USAGE на схему api и EXECUTE на функции (SELECT на представления) только своих экранов; прав на таблицы ни у одной группы нет, кроме чтения у администратора БД;

– функции api принадлежат app_owner и выполняются с его правами (SECURITY DEFINER) с фиксированным путём поиска search_path = pg_catalog, pg_temp, исключающим подмену объектов, и часовым поясом предметной области;

– первая операция каждой функции api — проверка группы вызывающего функцией sec.require_group; отказ возвращается ошибкой 42501 «Недостаточно прав для этого действия», а пока пользователь не сменил временный пароль — ошибкой SB001.

Группа определяется по прямому членству session_user в группах проекта (рисунок 4.7). Встроенная функция pg_has_role для этого не подходит: она

учитывает и косвенное членство, а администратор БД через членство в app_owner формально состоял бы во всех группах.

```
CREATE OR REPLACE FUNCTION sec.group_of(p_login name)
RETURNS text
LANGUAGE sql STABLE
SET search_path = pg_catalog, pg_temp
AS $$
   SELECT g.rolname::text
   FROM pg_catalog.pg_auth_members m
   JOIN pg_catalog.pg_roles g ON g.oid = m.roleid
   JOIN pg_catalog.pg_roles u ON u.oid = m.member
   WHERE u.rolname = p_login
     AND m.inherit_option
     AND g.rolname IN ('grp_admin', 'grp_dba', 'grp_operator', 'grp_manager', 'grp_chief')
   ORDER BY array_position(ARRAY['grp_admin', 'grp_dba', 'grp_operator', 'grp_manager', 'grp_chief'],
g.rolname::text)
   LIMIT 1
$$;
```

Рисунок 4.7 – Определение группы пользователя (sec.group_of)

Учётные записи создаются из приложения функцией api.user_create, принадлежащей app_owner с правом CREATEROLE: функция проверяет, что вызывающий администратор управляет указанной группой, выполняет CREATE ROLE и GRANT, добавляет строку «Сотрудник» и выдаёт временный пароль на сутки. Администратор приложения создаёт операторов, руководителей и главных аналитиков, администратор БД — администраторов приложения; первого администратора БД создаёт суперпользователь postgres. Вход пользователю запрещается и разрешается функцией api.user_set_login, пароль сбрасывается функцией api.user_reset_password: сервер генерирует временный пароль и до его смены пользователю доступны только сведения о себе и смена пароля. Группы и пользователи в pgAdmin показаны на рисунке 4.8.

[ВСТАВИТЬ СНИМОК: coursework/pz/img/pga_roles.png]

Рисунок 4.8 – Группы и учётные записи пользователей в pgAdmin

### 4.5 Разграничение доступа к данным на уровне строк (в зависимости от роли и логина)

Оператор и руководитель видят абонентов, договоры и сеансы только своего провайдера, главный аналитик — всех провайдеров или выбранного, администраторам данные абонентов, договоров и сеансов не выдаются. Провайдер пользователя определяется по его логину (session_user) в таблице «Сотрудник», группа — по членству в группах PostgreSQL.

Ограничение строк реализовано в функциях и представлениях api. Функции sec.scope_data и sec.scope_providers возвращают массив кодов провайдеров, данные которых видит пользователь, с учётом отбора по провайдеру из формы: свой провайдер, все провайдеры или пустой массив (рисунок 4.9). Каждый запрос к договорам и сеансам содержит одно условие «provider_id = ANY (массив)», вычисляемое один раз до выполнения запроса, что позволяет использовать индекс по провайдеру. Абонент общего реестра виден оператору и руководителю, пока у их провайдера есть с ним договор. Чужой договор не возвращается ни в списке, ни в карточке; договор с чужим абонентом по коду заключить нельзя. Эти правила проверены автоматическими тестами 7.х и 9.х (раздел 6).

```
CREATE OR REPLACE FUNCTION sec.provider_scope(p_sees_all boolean, p_provider_id integer)
RETURNS integer[]
LANGUAGE plpgsql STABLE
SET search_path = pg_catalog, pg_temp
AS $$
DECLARE
   v_own integer;
BEGIN
   PERFORM sec.assert_password_changed();
   IF p_sees_all THEN
      IF p_provider_id IS NOT NULL THEN
         RETURN ARRAY[p_provider_id];
      END IF;
      RETURN ARRAY(SELECT p.id FROM core.provider p ORDER BY p.id);
   END IF;
   v_own := sec.current_provider_id();
   IF v_own IS NULL OR v_own <> coalesce(p_provider_id, v_own) THEN
      RETURN '{}';
   END IF;
   RETURN ARRAY[v_own];
END $$;
```

Рисунок 4.9 – Видимые пользователю провайдеры (sec.provider_scope)

[ЗАГЛУШКА: политики защиты на уровне строк (ALTER TABLE … ENABLE и FORCE ROW LEVEL SECURITY, CREATE POLICY с session_user на core.contract, core.session, core.subscriber) не реализованы — план в docs/analysis/requirements.md, раздел 3. Сейчас строки ограничиваются только условиями в функциях и представлениях api. Описание политик, рисунок и тесты добавить после реализации]

## 4.6 Партицирование одной из основных таблиц БД

[ЗАГЛУШКА: подраздел будет дописан после ЛР 8]

## 4.7 Проектирование запросов к базе данных

Запросы проектировались как источники данных экранов клиентского приложения: для каждого запроса определены экран, роли пользователей и параметры отбора. Запросы без параметров оформлены представлениями схемы api, запросы с параметрами — функциями этой схемы. Функции-списки принимают размер и смещение страницы и возвращают общее число строк отбора (total_count), итоги «Итого…» вычисляются оконными функциями по

всему отбору. Соответствие видов запросов, требуемых заданием, запросам системы приведено в таблице 4.16.

Таблица 4.16 – Виды запросов задания и запросы системы

| Вид запроса по заданию | Запрос | Объект базы данных |
|---|---|---|
| Симметричное внутреннее соединение с условием по внешнему ключу | 1, 2 | api.q_provider_staff, api.q_contract_sessions |
| Симметричное внутреннее соединение с условием по датам | 3, 4 | api.q_sessions, api.q_contracts |
| Симметричное внутреннее соединение без условия | 5, 6, 7 | api.v_providers, api.v_subscribers, api.v_contracts |
| Левое внешнее соединение | 8 | api.q_subscribers |
| Правое внешнее соединение | 9 | api.q_tariff_usage |
| Запрос на запросе по принципу левого соединения | 10 | api.q_providers |
| Итоговый без условия, с итогами «всего», «в том числе» | 11 | api.v_user_groups |
| Итоговый с условием на данные по значению; с индексом и без индекса | 12 | api.report_new_contracts |
| Итоговый с условием на данные по маске; подзапрос с IN | 8 | api.q_subscribers |
| Итоговый с условием на группы | 13 | api.q_shared_subscribers |
| Итоговый с условием на данные и на группы | 14 | api.q_idle_contracts |
| Запрос с объединением | 15а | api.v_revenue_items |
| Запрос на запросе по принципу итогового запроса | 15б | api.report_revenue |
| Запрос с подзапросом NOT IN | 16 | api.q_providers |
| Подзапросы с CASE и операциями над итоговыми данными | 17 | api.report_provider_shares |
| Итоговый с условием на данные (отчёт) | — | api.report_top_subscribers |

Текст запросов приведён к форме, удобной для оптимизатора [1, 3]: условия записаны в канонической форме «столбец — оператор — константа», выражения от ввода пользователя (шаблоны поиска, цифры телефона) вычисляются один раз при входе в функцию, выборка выполняется до соединения, а для функций с необязательными отборами план строится по фактическим значениям параметров. В рисунках с текстом запросов функций приведена только выборка из тела функции: имена с префиксом p_ — параметры функции, v_scope — массив кодов видимых провайдеров, v_like и подобные — шаблоны поиска, вычисленные при входе в функцию.

### 4.7.1 Симметричное внутреннее соединение с условием

Запрос 1 выбирает сотрудников провайдера по коду провайдера (условие по внешнему ключу); группа и признак входа берутся из системных каталогов pg_roles и pg_auth_members. Используется на экранах «Провайдеры» и «Мой провайдер» (рисунки 4.10 и 4.11).

```
SELECT u.id, u.login::text, u.full_name::text, g.rolname::text,
    CASE g.rolname WHEN 'grp_admin' THEN 'Администраторы' WHEN 'grp_operator' THEN 'Операторы'
            WHEN 'grp_manager' THEN 'Руководители' WHEN 'grp_chief' THEN 'Главные аналитики' END,
    r.rolcanlogin, r.rolvaliduntil,
    count(*) OVER ()
FROM sec.app_user u
JOIN pg_catalog.pg_roles r ON r.rolname = u.login
JOIN pg_catalog.pg_auth_members m ON m.member = r.oid
JOIN pg_catalog.pg_roles g ON g.oid = m.roleid
                AND g.rolname IN ('grp_admin', 'grp_operator', 'grp_manager', 'grp_chief')
WHERE u.provider_id = ANY (v_scope)
ORDER BY array_position(ARRAY['grp_manager', 'grp_operator'], g.rolname::text), u.full_name
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.10 – Запрос 1 «Сотрудники провайдера» (api.q_provider_staff)

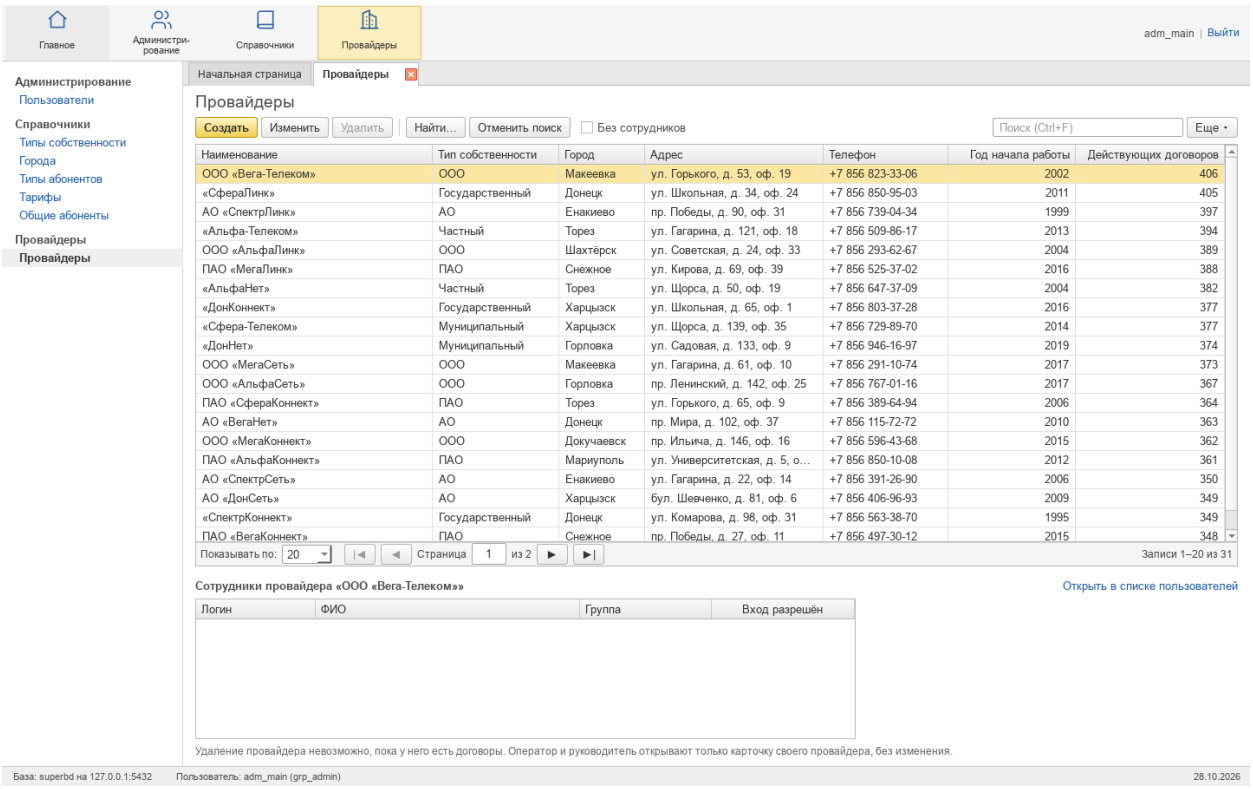



Рисунок 4.11 – Результат запроса 1 на экране «Провайдеры» (нижняя таблица)

Запрос 2 выбирает сеансы одного договора за период с тарифом и итогами по всему отбору (рисунки 4.12 и 4.13); используется на экране «Абоненты» и на вкладке «Сеансы» карточки договора.

```
SELECT s.id, s.started_at, t.code::text, t.name::text,
       s.volume_mb::numeric, s.discount_pct::numeric, s.cost::numeric,
       count(*) OVER (), sum(s.volume_mb) OVER (), sum(s.cost) OVER ()
FROM core.session s
JOIN core.contract c ON c.id = s.contract_id
JOIN ref.tariff t ON t.id = s.tariff_id
WHERE s.contract_id = p_contract_id
  AND c.provider_id = ANY (v_scope)
  AND (p_from IS NULL OR s.started_at >= p_from)
  AND (p_to IS NULL OR s.started_at < p_to + 1)
ORDER BY s.started_at DESC
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.12 – Запрос 2 «Сеансы договора за период» (api.q_contract_sessions)

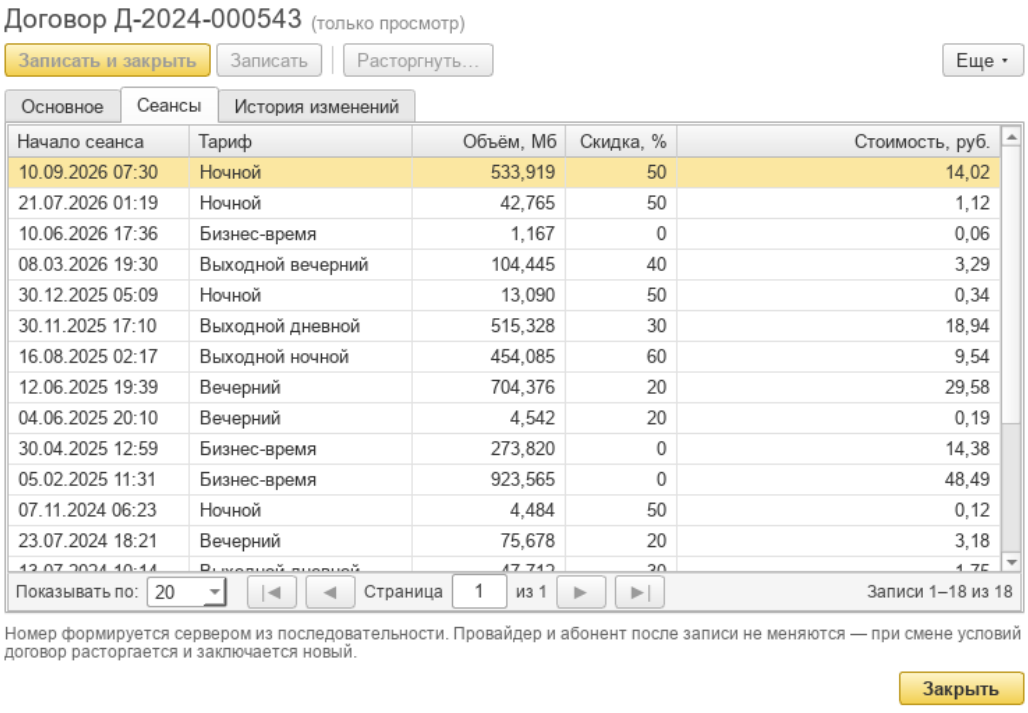



Рисунок 4.13 – Результат запроса 2: вкладка «Сеансы» карточки договора

Запрос 3 — журнал сеансов за период (условие по дате начала сеанса) с необязательными отборами по договору, IP-адресу или подсети и тарифу (рисунки 4.14 и 4.15); экран «Сеансы связи».

```
SELECT s.id, s.started_at, c.id, c.contract_no::text,
       sb.id, sb.name::text, host(c.ip_address),
       t.code::text, t.name::text, s.volume_mb::numeric, s.discount_pct::numeric,
       c.price_per_mb::numeric, s.cost::numeric,
       count(*) OVER (), sum(s.volume_mb) OVER (), sum(s.cost) OVER ()
FROM core.session s
JOIN core.contract c ON c.id = s.contract_id
JOIN core.subscriber sb ON sb.id = c.subscriber_id
JOIN ref.tariff t ON t.id = s.tariff_id
WHERE s.started_at >= p_from AND s.started_at < p_to + 1
  AND c.provider_id = ANY (v_scope)
  AND (p_contract_id IS NULL OR s.contract_id = p_contract_id)
  AND (v_ip IS NULL OR c.ip_address <<= v_ip)
  AND (p_tariff_id IS NULL OR s.tariff_id = p_tariff_id)
ORDER BY s.started_at DESC
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.14 – Запрос 3 «Журнал сеансов за период» (api.q_sessions)

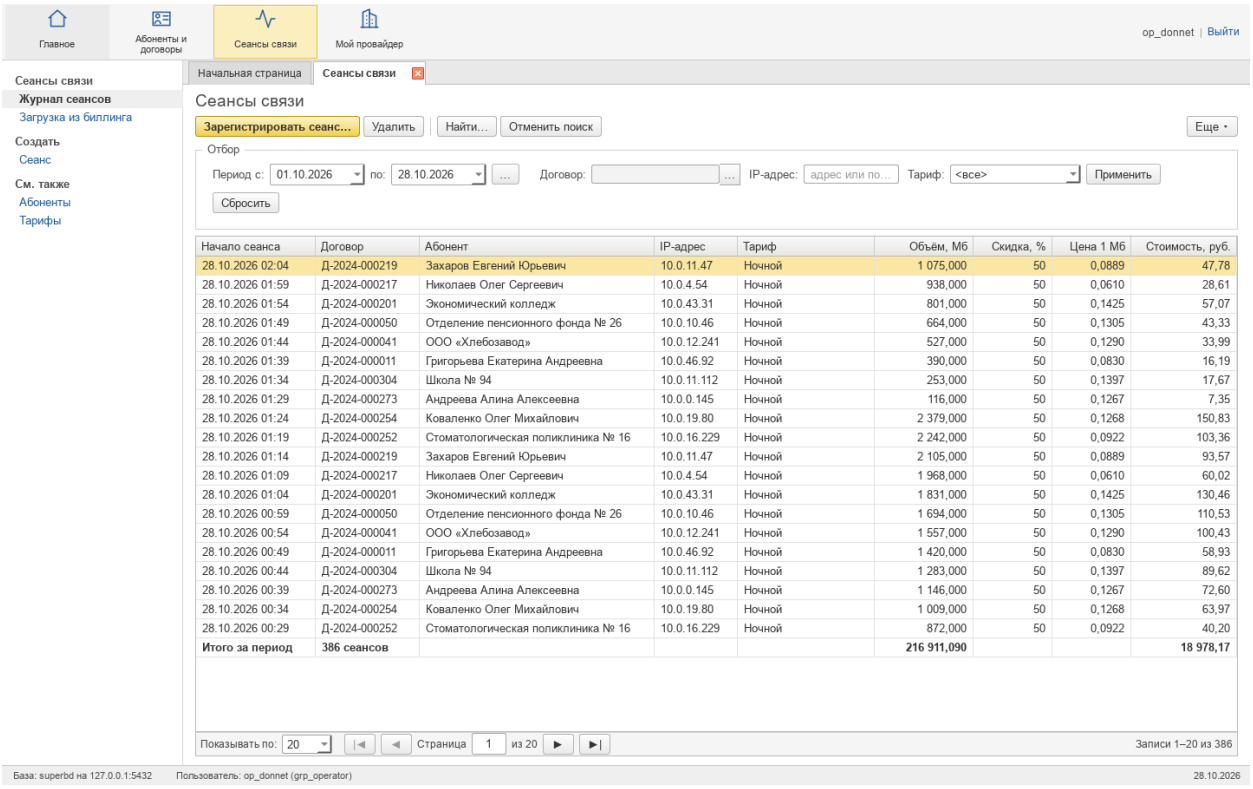



Рисунок 4.15 – Результат запроса 3 на экране «Сеансы связи»

Запрос 4 выбирает договоры, заключённые в периоде (условие по дате подключения), с абонентом, провайдером и состоянием; отборы — по абоненту, только действующие, поиск по началу номера, наименования или IP-адреса (рисунки 4.16 и 4.17); экраны «Договоры» и «Абоненты».

```
SELECT c.id, c.contract_no::text,
       CASE WHEN ot.name = upper(ot.name) THEN ot.name || ' «' || p.name || '»'
            ELSE '«' || p.name || '»' END,
       s.id, s.name::text, st.name::text,
       c.connected_on, c.connection_fee::numeric, c.price_per_mb::numeric,
       host(c.ip_address), c.closed_on,
       CASE WHEN c.closed_on IS NULL THEN 'Действует'
            ELSE 'Расторгнут ' || to_char(c.closed_on, 'DD.MM.YYYY') END,
       count(*) OVER ()
FROM core.contract c
JOIN core.subscriber s ON s.id = c.subscriber_id
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
JOIN core.provider p ON p.id = c.provider_id
JOIN ref.ownership_type ot ON ot.id = p.ownership_type_id
WHERE c.provider_id = ANY (v_scope)
  AND (p_from IS NULL OR c.connected_on >= p_from)
  AND (p_to IS NULL OR c.connected_on <= p_to)
  AND (p_subscriber_id IS NULL OR c.subscriber_id = p_subscriber_id)
  AND (NOT p_active_only OR c.closed_on IS NULL)
  AND (v_like IS NULL OR c.contract_no LIKE v_no_like OR lower(s.name) LIKE v_like
       OR host(c.ip_address) LIKE v_ip_like)
ORDER BY c.connected_on DESC, c.contract_no DESC
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.16 – Запрос 4 «Договоры, заключённые в периоде» (api.q_contracts)

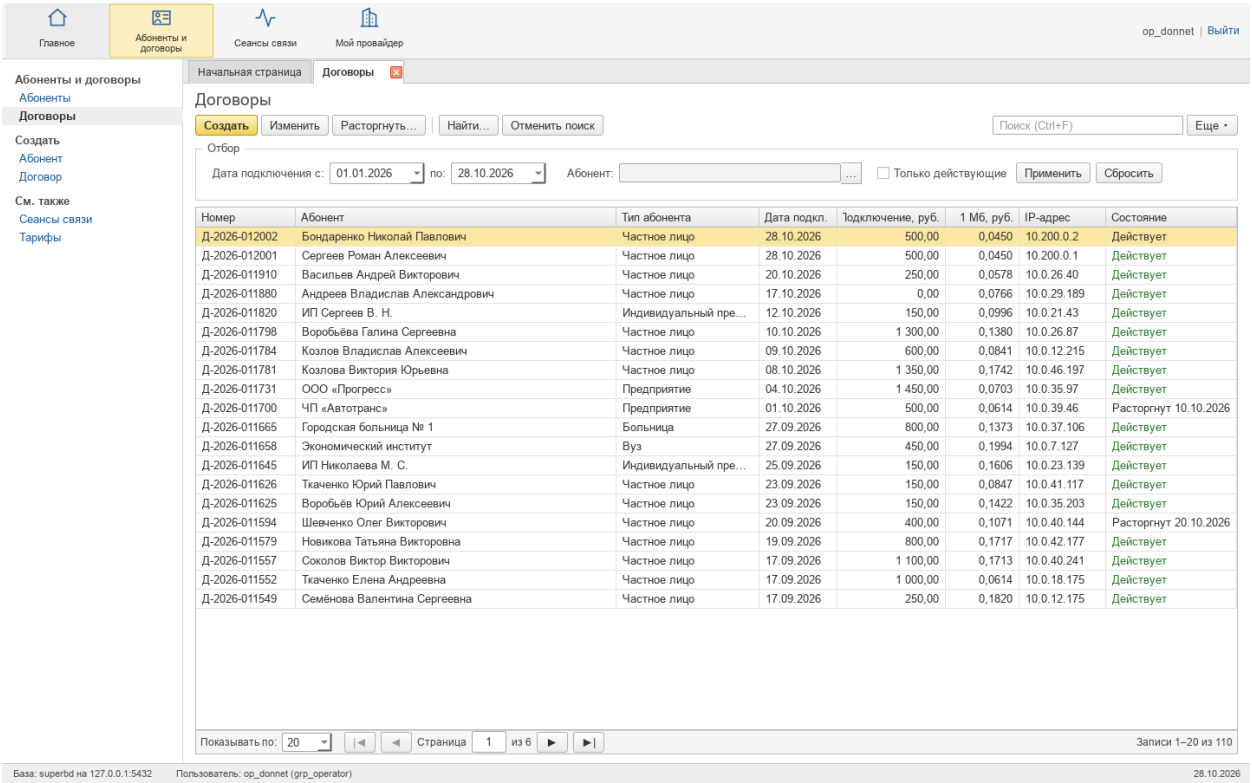



Рисунок 4.17 – Результат запроса 4 на экране «Договоры»

### 4.7.2 Симметричное внутреннее соединение без условия

Запросы 5–7 соединяют основную таблицу со справочниками без условий отбора и оформлены представлениями. Запрос 5 — провайдеры с типом

собственности и городом (рисунки 4.18 и 4.19); экран «Мой провайдер», формы выбора провайдера.

```
CREATE OR REPLACE VIEW api.v_providers WITH (security_barrier = true) AS
SELECT p.id,
    p.name::text AS name,
    CASE WHEN ot.name = upper(ot.name) THEN ot.name || ' «' || p.name || '»'
        ELSE '«' || p.name || '»' END AS display_name,
    p.ownership_type_id,
    ot.name::text AS ownership_type,
    p.city_id,
    c.name::text AS city,
    p.address::text AS address,
    p.phone::text AS phone,
    p.founded_year::integer AS founded_year
FROM core.provider p
JOIN ref.ownership_type ot ON ot.id = p.ownership_type_id
JOIN ref.city c ON c.id = p.city_id
WHERE p.id = ANY ((SELECT sec.scope_providers())::integer[]);
```

Рисунок 4.18 – Запрос 5 «Провайдеры» (api.v_providers)

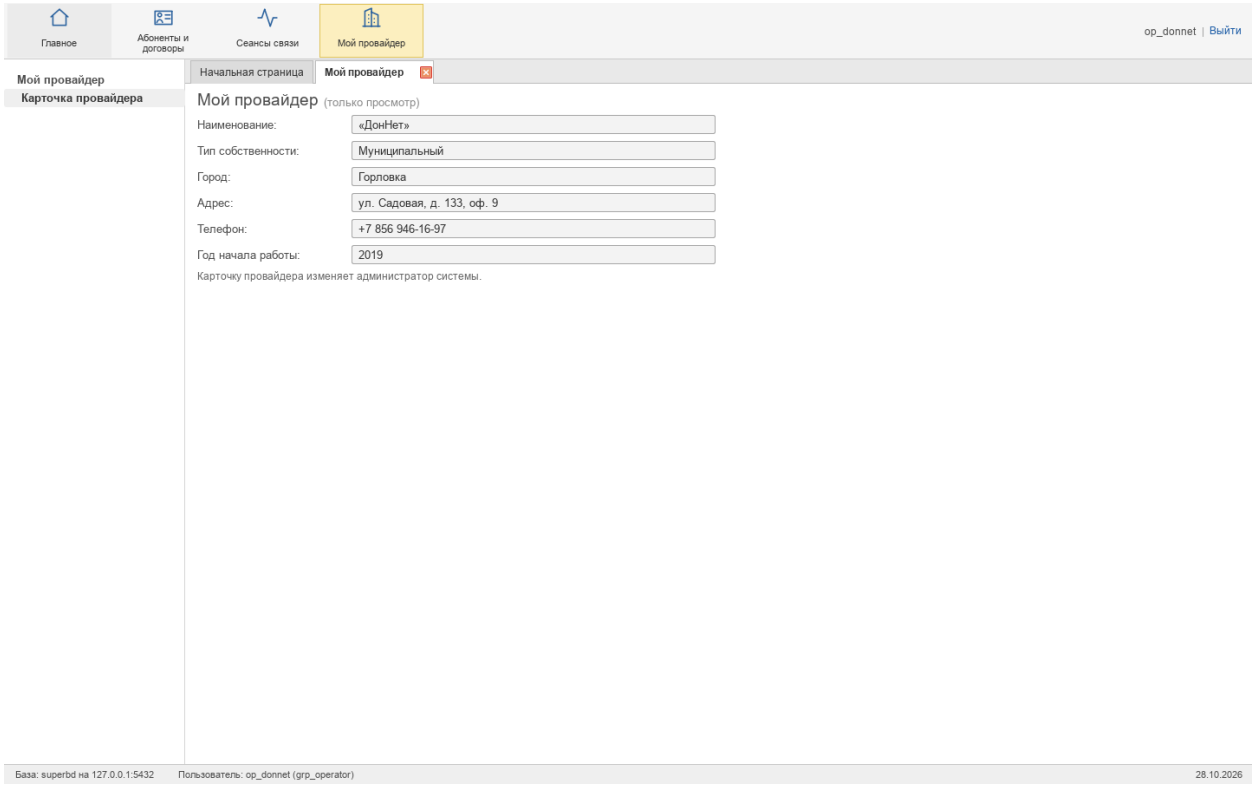



Рисунок 4.19 – Результат запроса 5 на экране «Мой провайдер»

Запрос 6 — абоненты с типом абонента и городом; оператору видны абоненты, у которых есть договор с его провайдером (рисунки 4.20 и 4.21); карточка абонента, форма выбора абонента.

```
CREATE OR REPLACE VIEW api.v_subscribers WITH (security_barrier = true) AS
SELECT s.id,
      s.name::text AS name,
      s.subscriber_type_id,
      st.name::text AS subscriber_type,
      st.is_legal,
      s.city_id,
      c.name::text AS city,
      s.address::text AS address,
      s.phone::text AS phone,
      s.row_version,
      (SELECT sec.current_group()) IS NOT DISTINCT FROM 'grp_admin'
         OR (SELECT sec.current_group()) IS NOT DISTINCT FROM 'grp_operator'
           AND NOT EXISTS (SELECT 1 FROM core.contract x
                     WHERE x.subscriber_id = s.id
                       AND x.provider_id IS DISTINCT FROM (SELECT sec.current_provider_id())) AS can_edit
FROM (SELECT s0.* FROM core.subscriber s0
     WHERE (SELECT sec.current_group()) IN ('grp_chief', 'grp_admin')
     UNION ALL
     SELECT s0.* FROM core.subscriber s0
     WHERE (SELECT sec.current_group()) NOT IN ('grp_chief', 'grp_admin')
       AND s0.id IN (SELECT v.subscriber_id FROM core.contract v
                  WHERE v.provider_id = ANY ((SELECT sec.scope_data())::integer[]))) s
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
JOIN ref.city c ON c.id = s.city_id;
```

Рисунок 4.20 – Запрос 6 «Абоненты» (api.v_subscribers)

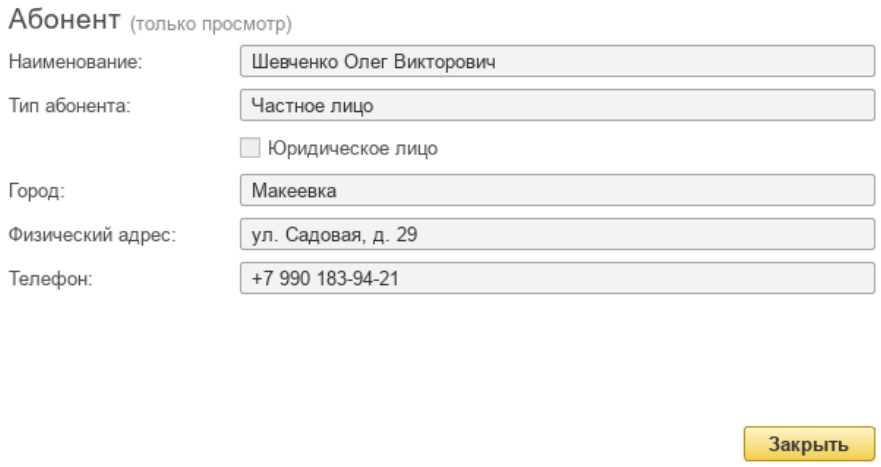



Рисунок 4.21 – Результат запроса 6: карточка абонента

Запрос 7 — договоры с абонентом, типом абонента, провайдером и данными подключения (рисунки 4.22 и 4.23); карточка договора.

```
CREATE OR REPLACE VIEW api.v_contracts WITH (security_barrier = true) AS
SELECT c.id,
    c.contract_no::text AS contract_no,
    c.provider_id,
    CASE WHEN ot.name = upper(ot.name) THEN ot.name || ' «' || p.name || '»'
        ELSE '«' || p.name || '»' END AS provider_name,
    c.subscriber_id,
    s.name::text AS subscriber_name,
    st.name::text AS subscriber_type,
    s.phone::text AS subscriber_phone,
    c.connected_on,
    c.connection_fee::numeric AS connection_fee,
    c.price_per_mb::numeric AS price_per_mb,
    host(c.ip_address) AS ip_address,
    c.closed_on,
    CASE WHEN c.closed_on IS NULL THEN 'Действует'
        ELSE 'Расторгнут ' || to_char(c.closed_on, 'DD.MM.YYYY') END AS state,
    c.note,
    c.row_version,
    c.connection_address::text AS connection_address,
    c.contact_phone::text AS contact_phone,
    c.contact_person::text AS contact_person
FROM core.contract c
JOIN core.subscriber s ON s.id = c.subscriber_id
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
JOIN core.provider p ON p.id = c.provider_id
JOIN ref.ownership_type ot ON ot.id = p.ownership_type_id
WHERE c.provider_id = ANY ((SELECT sec.scope_data())::integer[]);
```

Рисунок 4.22 – Запрос 7 «Договоры» (api.v_contracts)

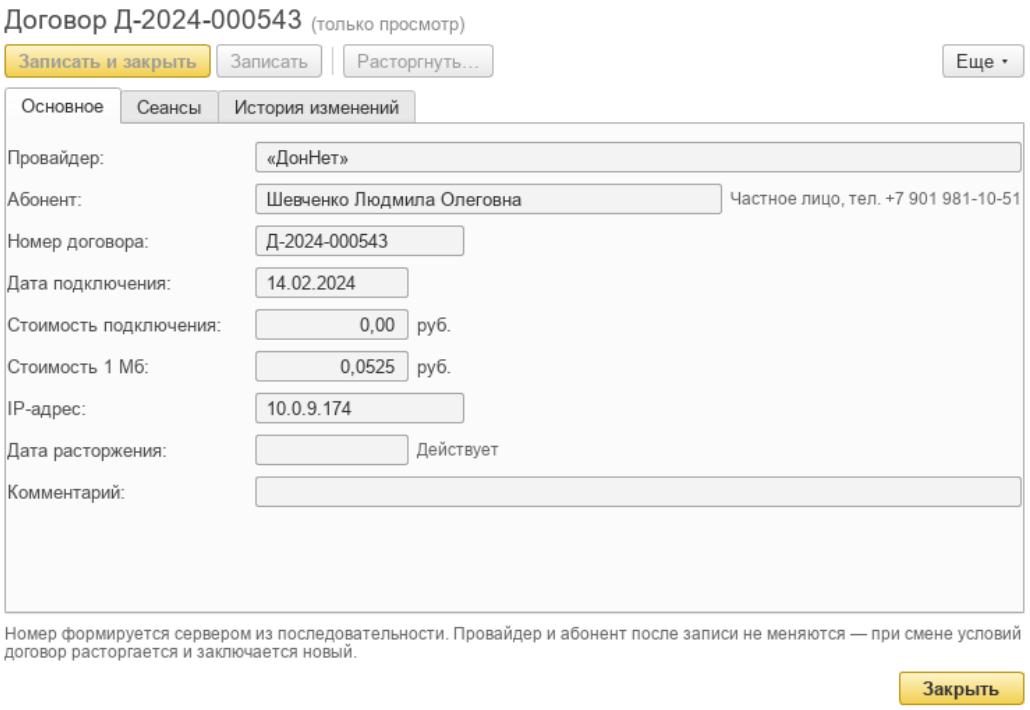



Рисунок 4.23 – Результат запроса 7: карточка договора

### 4.7.3 Внешние соединения и запрос на запросе

Запрос 8 присоединяет к абонентам их договоры левым внешним соединением и считает число всех и действующих договоров; абонент без договоров остаётся в результате с нулём (его видит главный аналитик). Поиск выполняется по маске — началу наименования (LIKE с префиксом) или цифрам телефона, видимость абонентов оператора — подзапросом IN по договорам его провайдера (рисунки 4.24 и 4.25); экран «Абоненты».

```
SELECT s.id, s.name::text, st.name::text, ci.name::text, s.address::text, s.phone::text,
       count(c.id), count(c.id) FILTER (WHERE c.closed_on IS NULL),
       count(*) OVER ()
FROM (SELECT s0.* FROM core.subscriber s0 WHERE v_all
      UNION ALL
      SELECT s0.* FROM core.subscriber s0
      WHERE NOT v_all
        AND s0.id IN (SELECT v.subscriber_id FROM core.contract v WHERE v.provider_id = ANY (v_scope))) s
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
JOIN ref.city ci ON ci.id = s.city_id
LEFT JOIN core.contract c ON c.subscriber_id = s.id
                         AND c.provider_id = ANY (v_scope)
WHERE (p_type_id IS NULL OR s.subscriber_type_id = p_type_id)
  AND (v_like IS NULL OR lower(s.name) LIKE v_like
       OR translate(s.phone, '+ ()-', '') LIKE v_phone_like)
GROUP BY s.id, s.name, s.address, s.phone, st.id, ci.id
HAVING NOT p_active_only OR count(c.id) FILTER (WHERE c.closed_on IS NULL) > 0
ORDER BY s.name, s.id
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.24 – Запрос 8 «Абоненты и число их договоров» (api.q_subscribers)

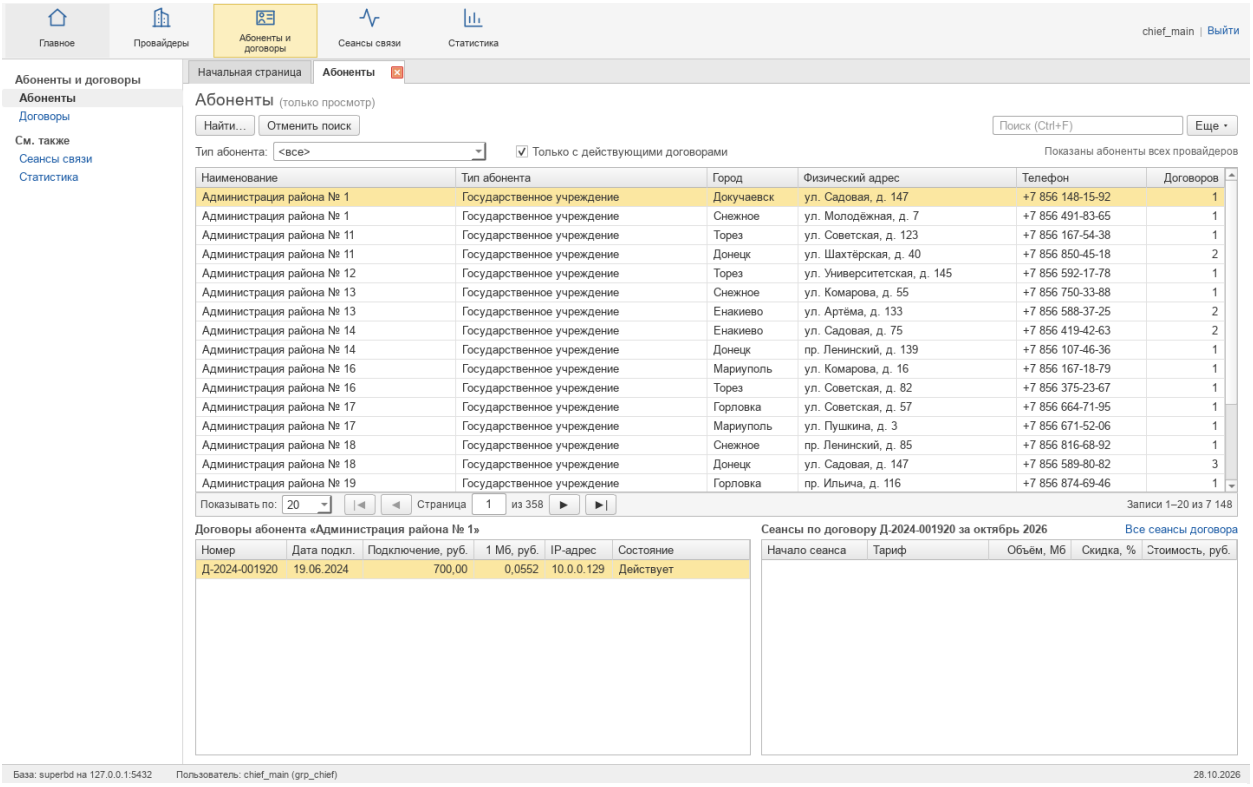



Рисунок 4.25 – Результат запроса 8 на экране «Абоненты» (главный аналитик)

Запрос 9 группирует сеансы периода по тарифу и правым внешним соединением дополняет результат всеми тарифами справочника — тариф без сеансов выводится с нулями (рисунки 4.26 и 4.27); отчёт «Трафик по тарифам».

```
SELECT t.id::integer, t.code::text, t.name::text, t.discount_pct::numeric,
       coalesce(u.sessions, 0), coalesce(u.volume_mb, 0)::numeric, coalesce(u.revenue, 0)::numeric,
       count(*) OVER ()
FROM (SELECT s.tariff_id, count(*) AS sessions, sum(s.volume_mb) AS volume_mb, sum(s.cost) AS revenue
      FROM core.session s
      JOIN core.contract c ON c.id = s.contract_id
      WHERE s.started_at >= p_from AND s.started_at < p_to + 1
        AND c.provider_id = ANY (v_scope)
      GROUP BY s.tariff_id) u
RIGHT JOIN ref.tariff t ON t.id = u.tariff_id
ORDER BY t.id
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.26 – Запрос 9 «Использование тарифов за период» (api.q_tariff_usage)

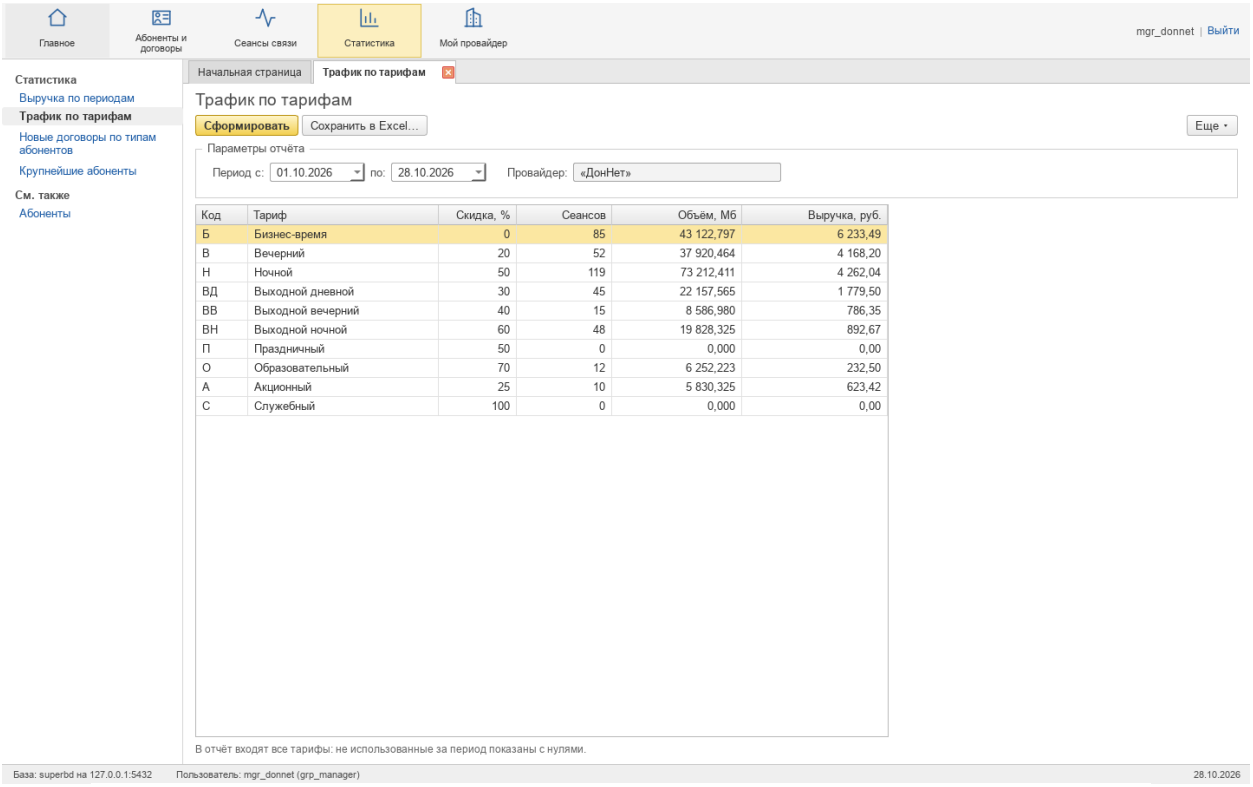



Рисунок 4.27 – Результат запроса 9 в отчёте «Трафик по тарифам»

Запрос 10 построен на запросе 5: представление провайдеров левым соединением дополняется подзапросом, который считает действующие договоры и договоры периода; провайдер без договоров остаётся в списке (рисунки 4.28 и 4.29); экран «Провайдеры».

```
SELECT v.id, v.display_name, v.ownership_type, v.city, v.address, v.phone, v.founded_year,
       coalesce(k.active, 0), coalesce(k.new_in_period, 0),
       count(*) OVER ()
FROM api.v_providers v
LEFT JOIN (
    SELECT c.provider_id,
           count(*) FILTER (WHERE c.closed_on IS NULL) AS active,
           count(*) FILTER (WHERE (p_from IS NULL OR c.connected_on >= p_from)
                            AND (p_to IS NULL OR c.connected_on <= p_to)) AS new_in_period
    FROM core.contract c
    WHERE c.provider_id = ANY (v_scope)
    GROUP BY c.provider_id) k ON k.provider_id = v.id
WHERE (v_like IS NULL OR lower(v.name) LIKE v_like)
  AND (NOT p_without_new OR coalesce(k.new_in_period, 0) = 0)
  AND (NOT p_without_staff
       OR v.id NOT IN (SELECT u.provider_id FROM sec.app_user u WHERE u.provider_id IS NOT NULL))
ORDER BY coalesce(k.active, 0) DESC, v.name
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.28 – Запрос 10 «Провайдеры с числом договоров» (api.q_providers)

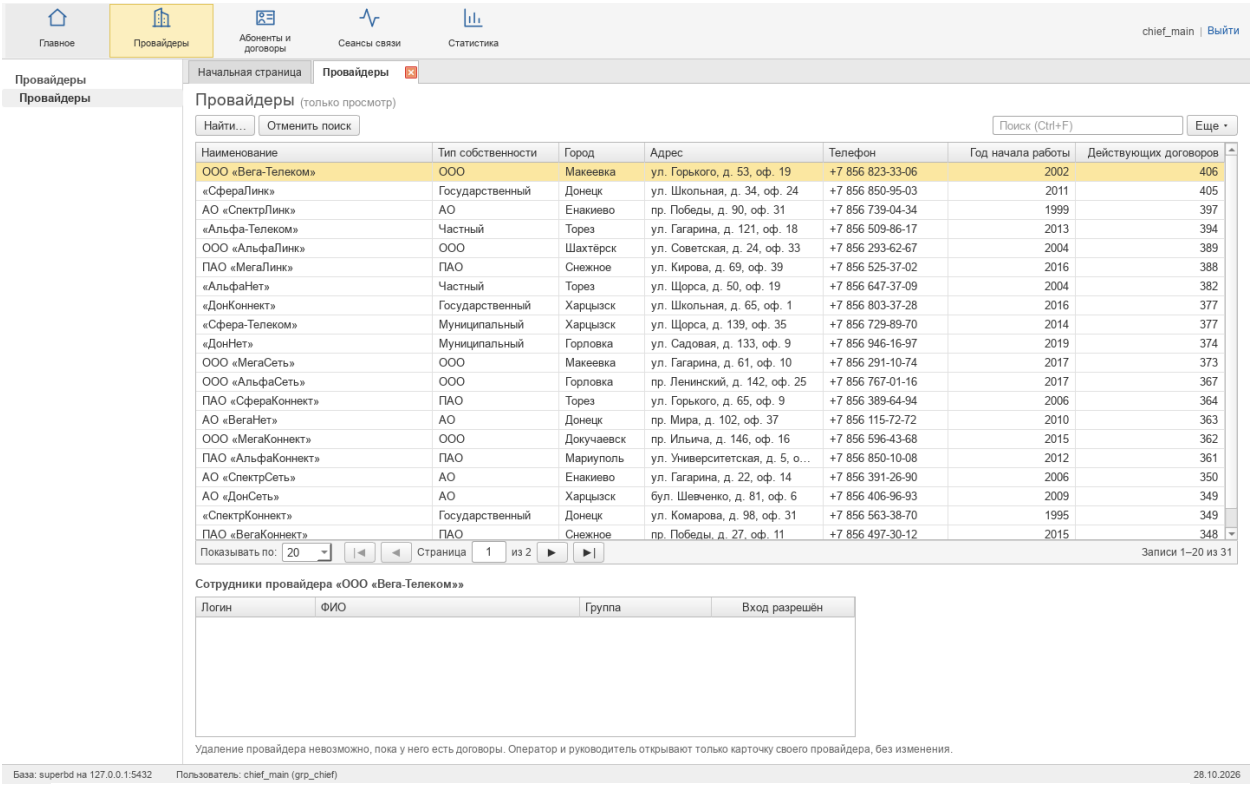



Рисунок 4.29 – Результат запроса 10 на экране «Провайдеры» (главный аналитик)

### 4.7.4 Итоговые запросы

Запрос 11 считает пользователей каждой группы и всего; строка «Все пользователи» получена группировкой ROLLUP, а не отдельным запросом (рисунки 4.30 и 4.31); панель групп экрана «Пользователи».

```
CREATE OR REPLACE VIEW api.v_user_groups WITH (security_barrier = true) AS
SELECT CASE WHEN grouping(g.rolname) = 1 THEN 'Все пользователи'
            ELSE sec.group_title(g.rolname::text) END AS title,
       g.rolname::text AS group_name,
       count(*) AS users,
       CASE WHEN grouping(g.rolname) = 1 THEN 0
            ELSE array_position(ARRAY['grp_dba', 'grp_admin', 'grp_chief', 'grp_manager', 'grp_operator']::name[],
                                g.rolname) END AS ord
FROM sec.app_user u
JOIN pg_catalog.pg_roles r        ON r.rolname = u.login
JOIN pg_catalog.pg_auth_members m ON m.member = r.oid AND m.inherit_option
JOIN pg_catalog.pg_roles g        ON g.oid = m.roleid
WHERE g.rolname = ANY ((SELECT sec.visible_groups(sec.current_group()))::name[])
  AND (SELECT sec.assert_password_changed())
GROUP BY ROLLUP (g.rolname)
ORDER BY ord;
```

Рисунок 4.30 – Запрос 11 «Пользователи по группам» (api.v_user_groups)

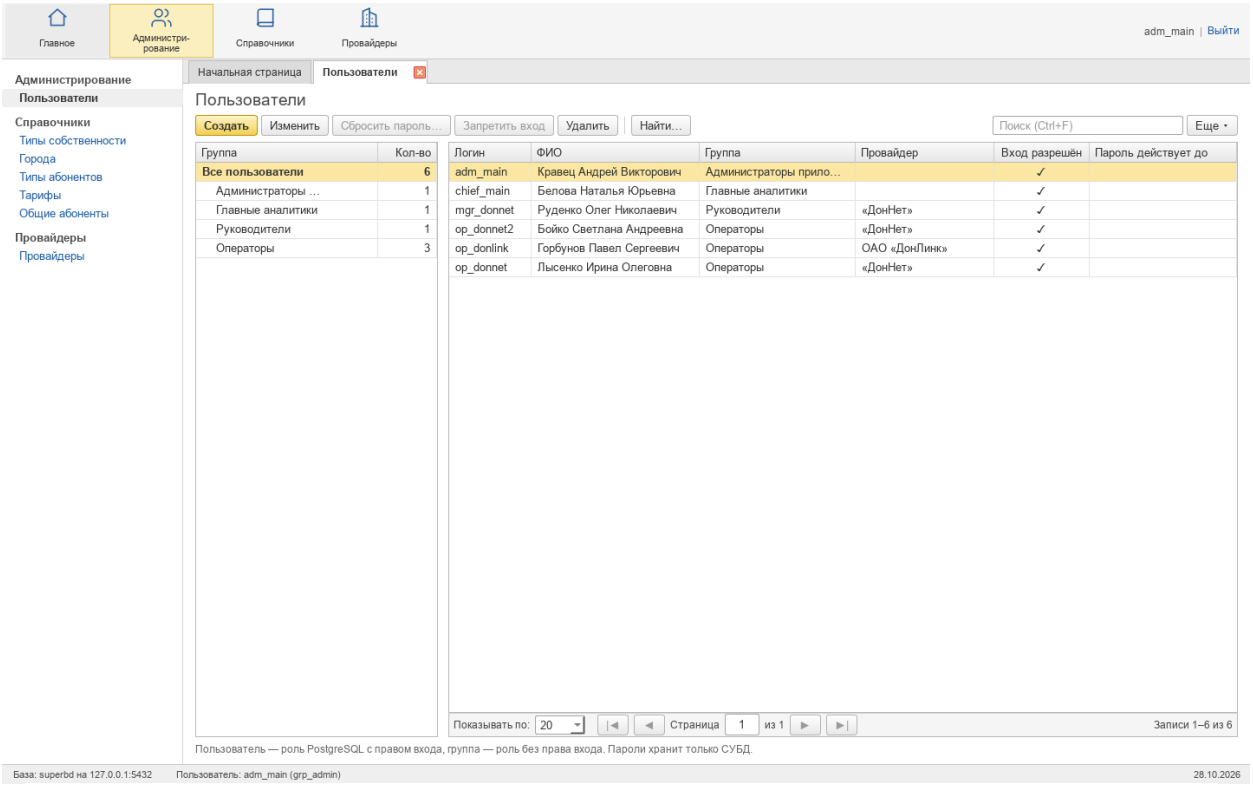



Рисунок 4.31 – Результат запроса 11: панель групп экрана «Пользователи»

Запрос 12 считает новые договоры по периодам (месяц, квартал, год) и типам абонентов с условием на данные — договоры видимых провайдеров, заключённые в периоде (рисунки 4.32 и 4.33); отчёт «Новые договоры по типам абонентов». Для руководителя провайдера за квартал договоры отбираются по составному индексу «провайдер, дата подключения», для главного аналитика за два года (73 % договоров) планировщик выбирает последовательное чтение таблицы как более дешёвое (рисунки 4.34 и 4.35, фрагменты планов, полученных модулем auto_explain).

```
SELECT k.period, sec.period_title(p_grain, k.period), st.id::integer, st.name::text, k.contracts
FROM (SELECT date_trunc(p_grain, c.connected_on::timestamp)::date AS period, s.subscriber_type_id,
          count(*) AS contracts
     FROM core.contract c
     JOIN core.subscriber s ON s.id = c.subscriber_id
     WHERE c.provider_id = ANY (v_scope) AND c.connected_on BETWEEN p_from AND p_to
     GROUP BY 1, s.subscriber_type_id) k
JOIN ref.subscriber_type st ON st.id = k.subscriber_type_id
ORDER BY k.period, st.id;
```

Рисунок 4.32 – Запрос 12 «Новые договоры по периодам и типам абонентов» (api.report_new_contracts)

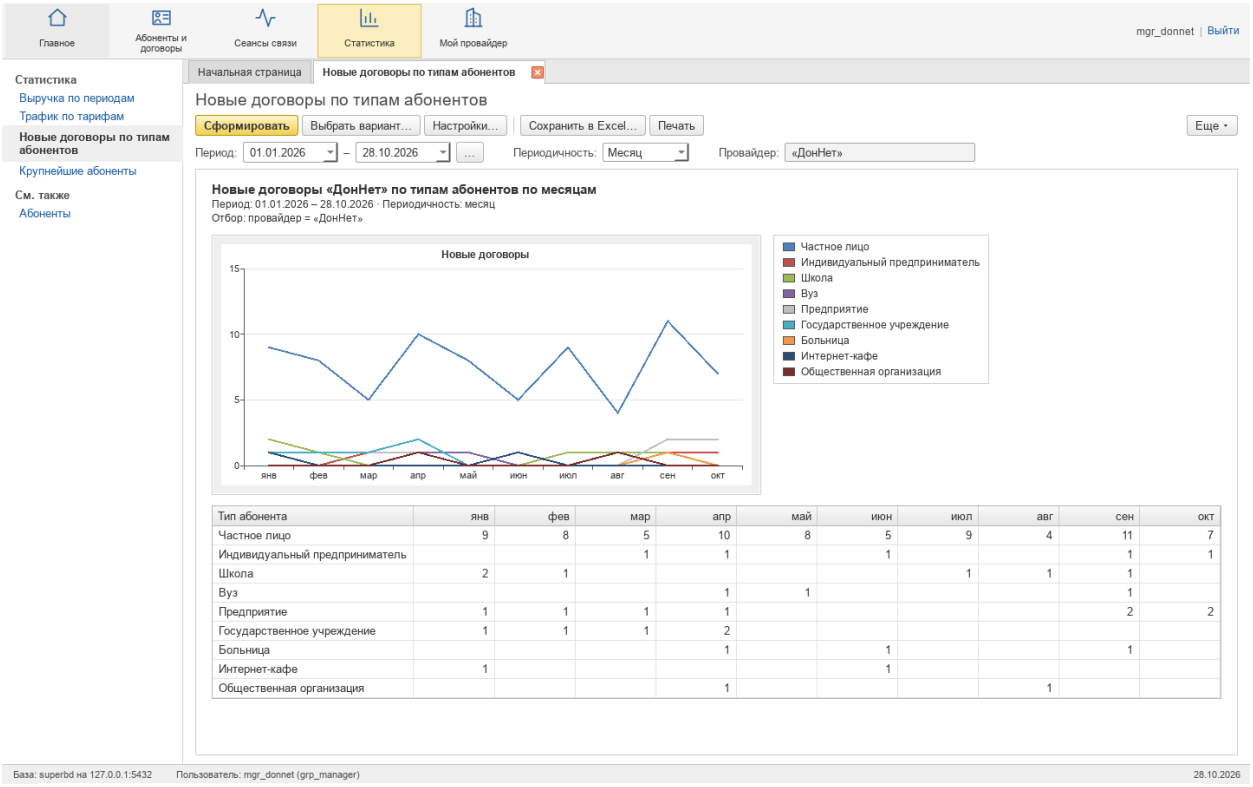



Рисунок 4.33 – Результат запроса 12 в отчёте «Новые договоры по типам абонентов»

```
->  Nested Loop  (cost=5.03..322.76 rows=36 width=6) (actual rows=36.00 loops=1)
      Buffers: shared hit=125
      ->  Bitmap Heap Scan on contract c  (cost=4.74..99.69 rows=36 width=8) (actual rows=36.00 loops=1)
            Recheck Cond: ((provider_id = ANY ('{1}'::integer[])) AND (connected_on >= '2026-07-01'::date) AND (connected_on <= '2026-09-
30'::date))
            Heap Blocks: exact=15
            Buffers: shared hit=17
            ->  Bitmap Index Scan on ix_contract_provider_id_connected_on  (cost=0.00..4.74 rows=36 width=0) (actual rows=36.00 loops=1)
                  Index Cond: ((provider_id = ANY ('{1}'::integer[])) AND (connected_on >= '2026-07-01'::date) AND (connected_on <= '2026-09-
30'::date))
                  Index Searches: 1
                  Buffers: shared hit=2
      ->  Index Scan using pk_subscriber on subscriber s  (cost=0.28..6.19 rows=1 width=6) (actual rows=1.00 loops=36)
            Index Cond: (id = c.subscriber_id)
```

Рисунок 4.34 – План запроса 12 с использованием индекса: руководитель, III квартал (фрагмент)

```
->  Hash Join  (cost=324.08..856.33 rows=8711 width=6) (actual rows=8714.00 loops=1)
      Hash Cond: (c.subscriber_id = s.id)
      Buffers: shared hit=348
      ->  Seq Scan on contract c  (cost=0.08..444.12 rows=8711 width=8) (actual rows=8714.00 loops=1)
            Filter: ((connected_on >= '2024-01-01'::date) AND (connected_on <= '2025-12-31'::date) AND (provider_id = ANY
('{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31}'::integer[])))
            Rows Removed by Filter: 3288
            Buffers: shared hit=204
```

Рисунок 4.35 – План запроса 12 без использования индекса: главный аналитик, 2024–2025 годы (фрагмент)

Запрос 13 выбирает общих абонентов — тех, у кого договоры не меньше чем с заданным числом провайдеров; условие на группы записано в HAVING (рисунки 4.36 и 4.37); экран «Общие абоненты» администратора приложения.

```
SELECT s.id, s.name::text, st.name::text, ci.name::text, s.address::text, s.phone::text,
       count(DISTINCT c.provider_id), count(*) OVER ()
FROM core.subscriber s
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
JOIN ref.city ci            ON ci.id = s.city_id
JOIN core.contract c        ON c.subscriber_id = s.id
GROUP BY s.id, st.id, ci.id
HAVING count(DISTINCT c.provider_id) >= p_min_providers
ORDER BY s.name, s.id
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.36 – Запрос 13 «Общие абоненты» (api.q_shared_subscribers)

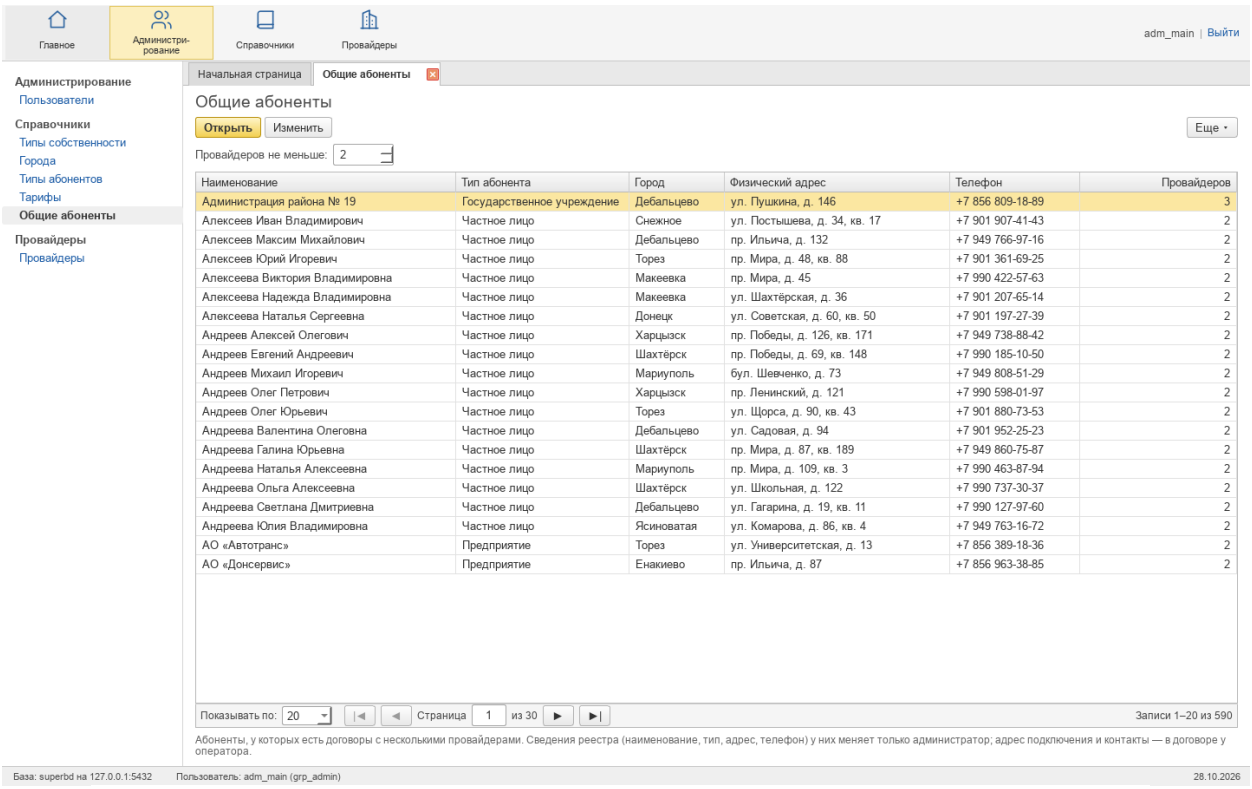



Рисунок 4.37 – Результат запроса 13 на экране «Общие абоненты»

Запрос 14 выбирает действующие договоры без сеансов больше заданного числа дней: условие на данные (WHERE) отбирает действующие договоры провайдера, условие на группы (HAVING) — договоры, у которых последний сеанс раньше границы (рисунки 4.38 и 4.39); счётчик начальной страницы и список «Договоры без сеансов».

```
SELECT c.id, c.contract_no::text, s.id, s.name::text, coalesce(c.contact_phone, s.phone)::text,
       c.connected_on, max(se.started_at),
       current_date - coalesce(max(se.started_at)::date, c.connected_on),
       count(*) OVER ()
FROM core.contract c
JOIN core.subscriber s     ON s.id = c.subscriber_id
LEFT JOIN core.session se ON se.contract_id = c.id
WHERE c.provider_id = ANY (v_scope) AND c.closed_on IS NULL
  AND c.connected_on < v_border
GROUP BY c.id, s.id
HAVING max(se.started_at) IS NULL
    OR max(se.started_at) < v_border::timestamptz
ORDER BY max(se.started_at) NULLS FIRST, c.connected_on, c.contract_no
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.38 – Запрос 14 «Договоры без сеансов» (api.q_idle_contracts)

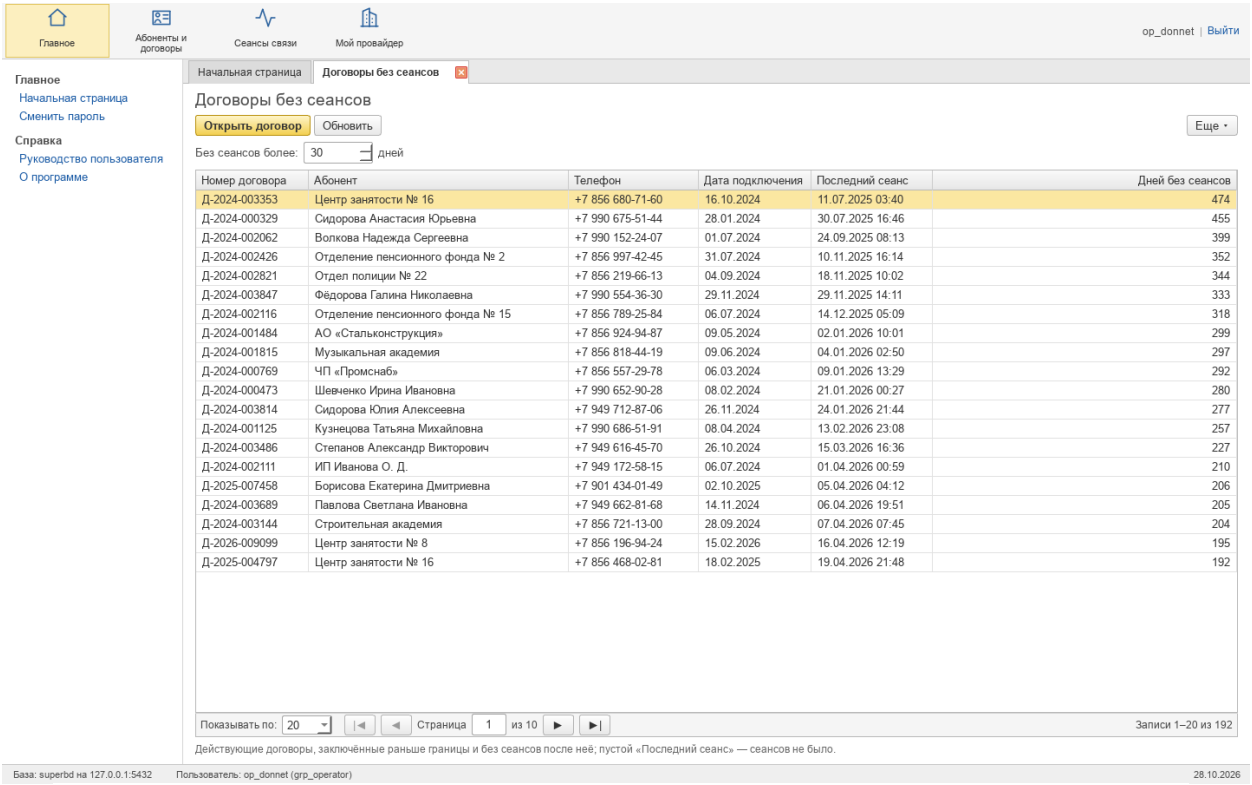



Рисунок 4.39 – Результат запроса 14 на экране «Договоры без сеансов»

Отчёт «Крупнейшие абоненты» выбирает первых N абонентов по выручке за период с долей от выручки периода (рисунки 4.40 и 4.41).

```
SELECT k.rnk::integer, s.id, s.name::text, st.name::text, k.volume_mb, k.revenue,
       round(100 * k.revenue / nullif(k.total, 0), 1)
FROM (SELECT r.subscriber_id, sum(r.volume_mb) AS volume_mb, sum(r.amount) AS revenue,
             row_number() OVER (ORDER BY sum(r.amount) DESC, r.subscriber_id) AS rnk,
             sum(sum(r.amount)) OVER () AS total
      FROM api.v_revenue_items r
      WHERE r.at >= p_from AND r.at < p_to + 1
        AND (p_provider_id IS NULL OR r.provider_id = p_provider_id)
      GROUP BY r.subscriber_id
      ORDER BY sum(r.amount) DESC, r.subscriber_id
      LIMIT least(greatest(coalesce(p_top, 10), 1), 100)) k
JOIN core.subscriber s ON s.id = k.subscriber_id
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
ORDER BY k.rnk;
```

Рисунок 4.40 – Отчёт «Крупнейшие абоненты» (api.report_top_subscribers)

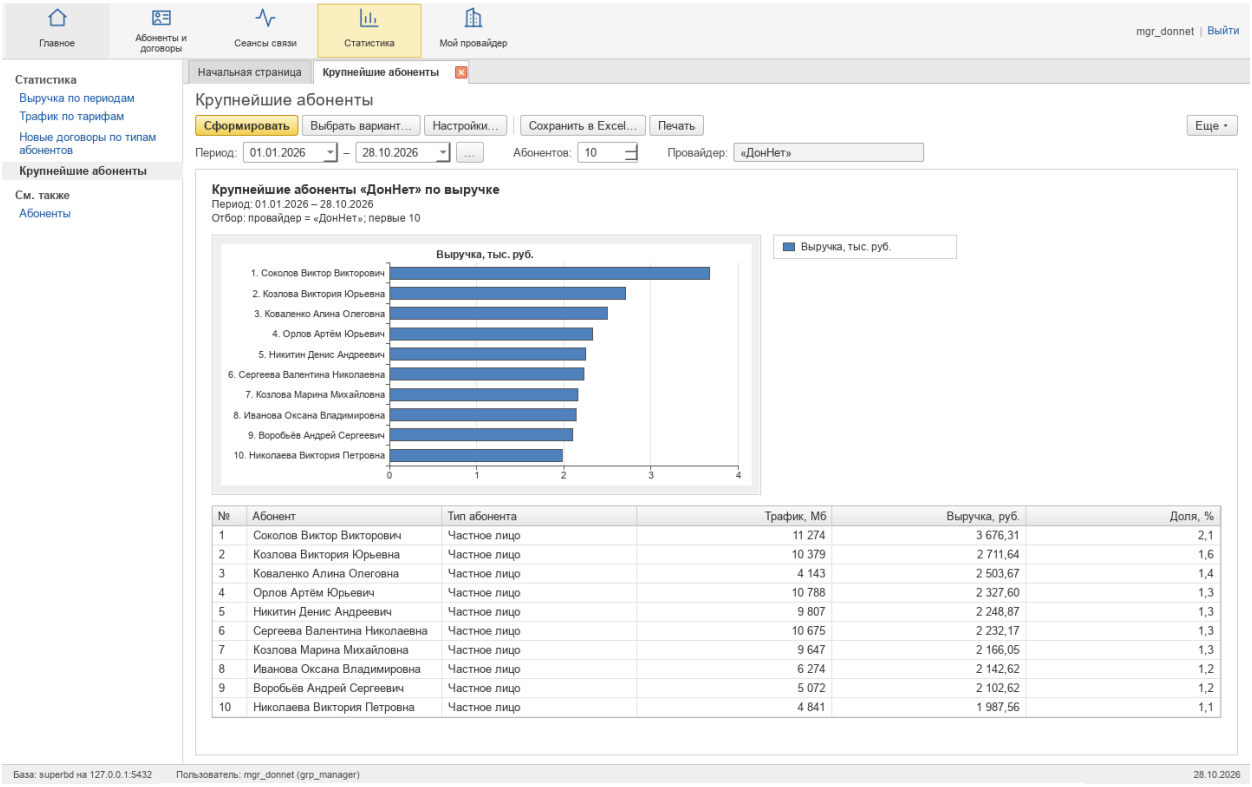



Рисунок 4.41 – Результат отчёта «Крупнейшие абоненты»

### 4.7.5 Объединение, запрос на запросе и запросы с подзапросами

Запрос 15a объединяет оператором UNION ALL две доходные операции с разными датами — плату за подключение (дата заключения договора) и оплату трафика (начало сеанса); совпадающие суммы являются разными операциями и не удаляются (рисунки 4.42 и 4.43). Представление служит основой отчётов о выручке.

```
CREATE OR REPLACE VIEW api.v_revenue_items WITH (security_barrier = true) AS
SELECT c.provider_id,
       c.subscriber_id,
       s.subscriber_type_id,
       c.connected_on::timestamptz AS at,
       'подключение'::text AS kind,
       c.connection_fee::numeric AS amount,
       0::numeric AS volume_mb
FROM core.contract c
JOIN core.subscriber s ON s.id = c.subscriber_id
WHERE c.provider_id = ANY ((SELECT sec.scope_data())::integer[])
UNION ALL
SELECT c.provider_id,
       c.subscriber_id,
       sb.subscriber_type_id,
       se.started_at,
       'трафик'::text,
       se.cost::numeric,
       se.volume_mb::numeric
FROM core.session se
JOIN core.contract c ON c.id = se.contract_id
JOIN core.subscriber sb ON sb.id = c.subscriber_id
WHERE c.provider_id = ANY ((SELECT sec.scope_data())::integer[]);
```

Рисунок 4.42 – Запрос 15а «Доходные операции» (api.v_revenue_items)

[ВСТАВИТЬ СНИМОК: coursework/pz/img/pga_v_revenue_items.png]

Рисунок 4.43 – Результат запроса 15а в pgAdmin

Запрос 15б построен на запросе 15а по принципу итогового запроса: группирует доходные операции по периодам с ROLLUP и добавляет строку «Итого» (рисунки 4.44 и 4.45); отчёт «Выручка по периодам» и счётчик «Выручка за месяц» начальной страницы.

```
SELECT k.period, CASE WHEN k.is_total THEN 'Итого' ELSE sec.period_title(p_grain, k.period) END,
       k.new_contracts, k.volume_mb, k.connection_fees, k.revenue - k.connection_fees, k.revenue
FROM (SELECT date_trunc(p_grain, r.at)::date AS period,
             grouping(date_trunc(p_grain, r.at)::date) = 1                    AS is_total,
             count(*) FILTER (WHERE r.kind = 'подключение')                   AS new_contracts,
             coalesce(sum(r.volume_mb), 0)                                    AS volume_mb,
             coalesce(sum(r.amount) FILTER (WHERE r.kind = 'подключение'), 0)  AS connection_fees,
             coalesce(sum(r.amount), 0)                                       AS revenue
      FROM api.v_revenue_items r
      WHERE r.at >= p_from AND r.at < p_to + 1
        AND (p_provider_id IS NULL OR r.provider_id = p_provider_id)
        AND (p_subscriber_type_id IS NULL OR r.subscriber_type_id = p_subscriber_type_id)
      GROUP BY ROLLUP (date_trunc(p_grain, r.at)::date)) k
ORDER BY k.is_total, k.period;
```

Рисунок 4.44 – Запрос 15б «Выручка по периодам» (api.report_revenue)

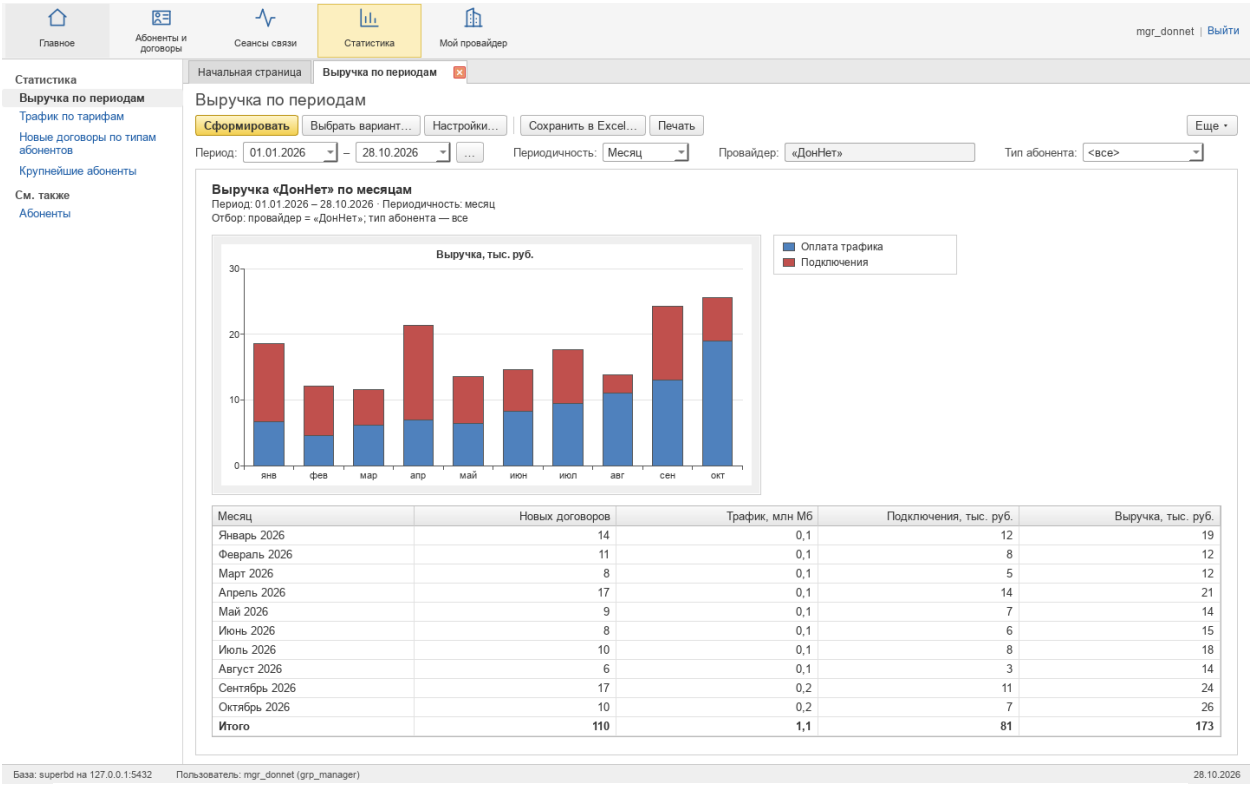



Рисунок 4.45 – Результат запроса 15б в отчёте «Выручка по периодам» (руководитель)

Запрос 16 — отбор «Без сотрудников» в функции api.q_providers: провайдеры, коды которых не входят (NOT IN) в подзапрос по сотрудникам; значения NULL в подзапросе исключаются явно, иначе разность множеств была бы пустой (рисунок 4.46). Текст функции приведён на рисунке 4.28.

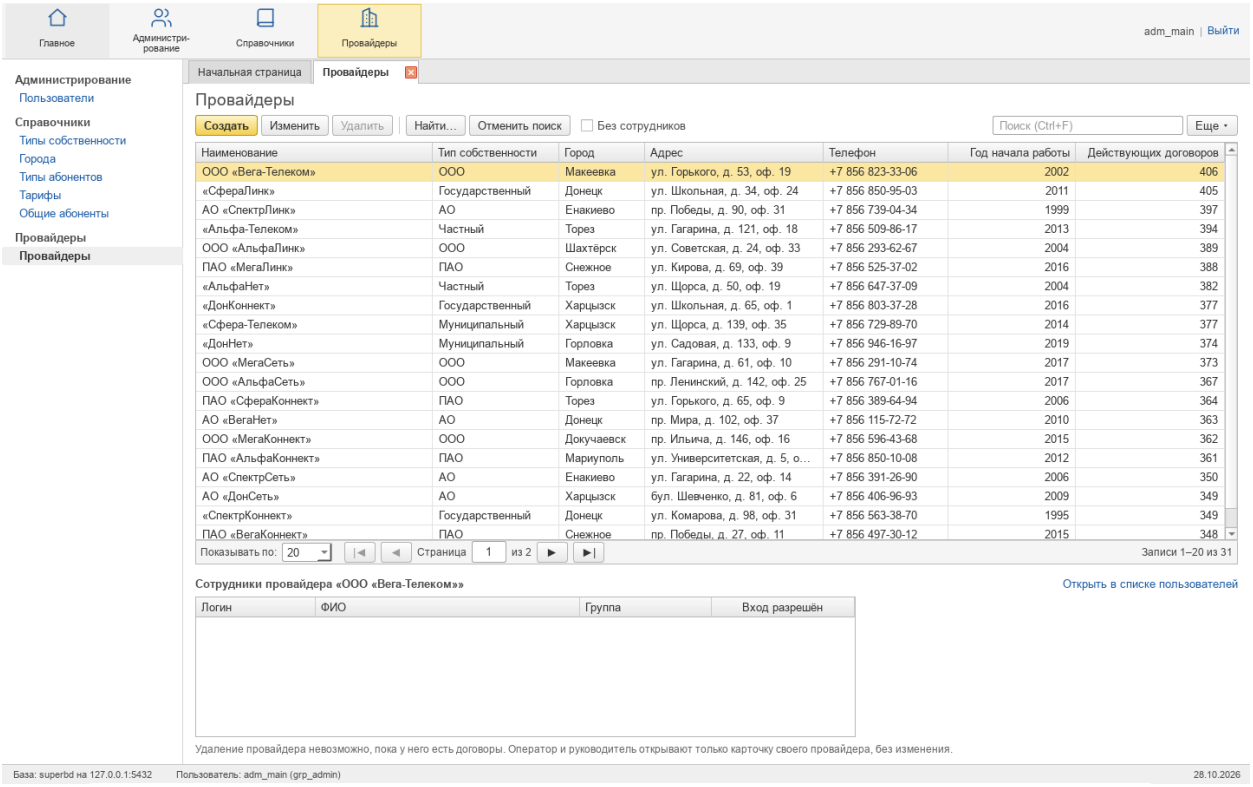



Рисунок 4.46 – Результат запроса 16: провайдеры без сотрудников (администратор приложения)

Запрос 17 вычисляет доли провайдеров в выручке: внутренний запрос считает выручку провайдера, его место и общий итог, внешний относит провайдера к группе крупнейших или к «Прочим» выражением CASE и вычисляет долю группы (рисунки 4.47 и 4.48); отчёт главного аналитика.

```
WITH x AS (
   SELECT p.id, p.display_name,
          coalesce(k.contracts, 0)                                  AS contracts,
          coalesce(r.volume_mb, 0)                                     AS volume_mb,
          coalesce(r.revenue, 0)                                    AS revenue,
          least(row_number() OVER (ORDER BY r.revenue DESC NULLS LAST, p.id), v_top + 1)::integer AS grp,
          sum(r.revenue) OVER ()                                       AS total
   FROM api.v_providers p
   LEFT JOIN (SELECT i.provider_id, sum(i.volume_mb) AS volume_mb, sum(i.amount) AS revenue
              FROM api.v_revenue_items i
              WHERE i.at >= p_from AND i.at < p_to + 1
                AND (p_subscriber_type_id IS NULL OR i.subscriber_type_id = p_subscriber_type_id)
              GROUP BY i.provider_id) r ON r.provider_id = p.id
   LEFT JOIN (SELECT c.provider_id, count(*) AS contracts
              FROM core.contract c
              WHERE c.connected_on <= p_to AND (c.closed_on IS NULL OR c.closed_on > p_to)
                AND (p_subscriber_type_id IS NULL
                     OR c.subscriber_id IN (SELECT s.id FROM core.subscriber s
                                            WHERE s.subscriber_type_id = p_subscriber_type_id))
              GROUP BY c.provider_id) k ON k.provider_id = p.id),
t AS (
   SELECT CASE WHEN grouping(x.grp) = 1 THEN v_top + 2 ELSE x.grp END AS ord,
          CASE WHEN grouping(x.grp) = 1 THEN 'Итого'
               WHEN x.grp <= v_top THEN min(x.display_name)
               ELSE 'Прочие (' || sec.count_title(count(*), 'провайдер', 'провайдера', 'провайдеров') || ')'
          END AS provider_group,
          count(*) AS providers, sum(x.contracts)::bigint AS contracts,
          sum(x.volume_mb) AS volume_mb, sum(x.revenue) AS revenue,
          round(100 * sum(x.revenue) / nullif(max(x.total), 0), 1) AS share_pct,
          grouping(x.grp) = 1 AS is_total
   FROM x
   GROUP BY ROLLUP (x.grp))
SELECT t.ord, t.provider_group, t.providers, NULL::date, NULL::text,
       t.contracts, t.volume_mb, t.revenue, t.share_pct, t.is_total
FROM t
UNION ALL
SELECT x.grp, t.provider_group, t.providers, d.period, sec.period_title(p_grain, d.period),
       NULL::bigint, sum(i.volume_mb), sum(i.amount), NULL::numeric, false
FROM x
JOIN t ON t.ord = x.grp
JOIN api.v_revenue_items i ON i.provider_id = x.id AND i.at >= p_from AND i.at < p_to + 1
                          AND (p_subscriber_type_id IS NULL OR i.subscriber_type_id = p_subscriber_type_id)
CROSS JOIN LATERAL (SELECT date_trunc(p_grain, i.at)::date AS period) d
WHERE p_grain IS NOT NULL
GROUP BY x.grp, t.provider_group, t.providers, d.period
ORDER BY 4 NULLS FIRST, 1;
```

Рисунок 4.47 – Запрос 17 «Доли провайдеров в выручке» (api.report_provider_shares)

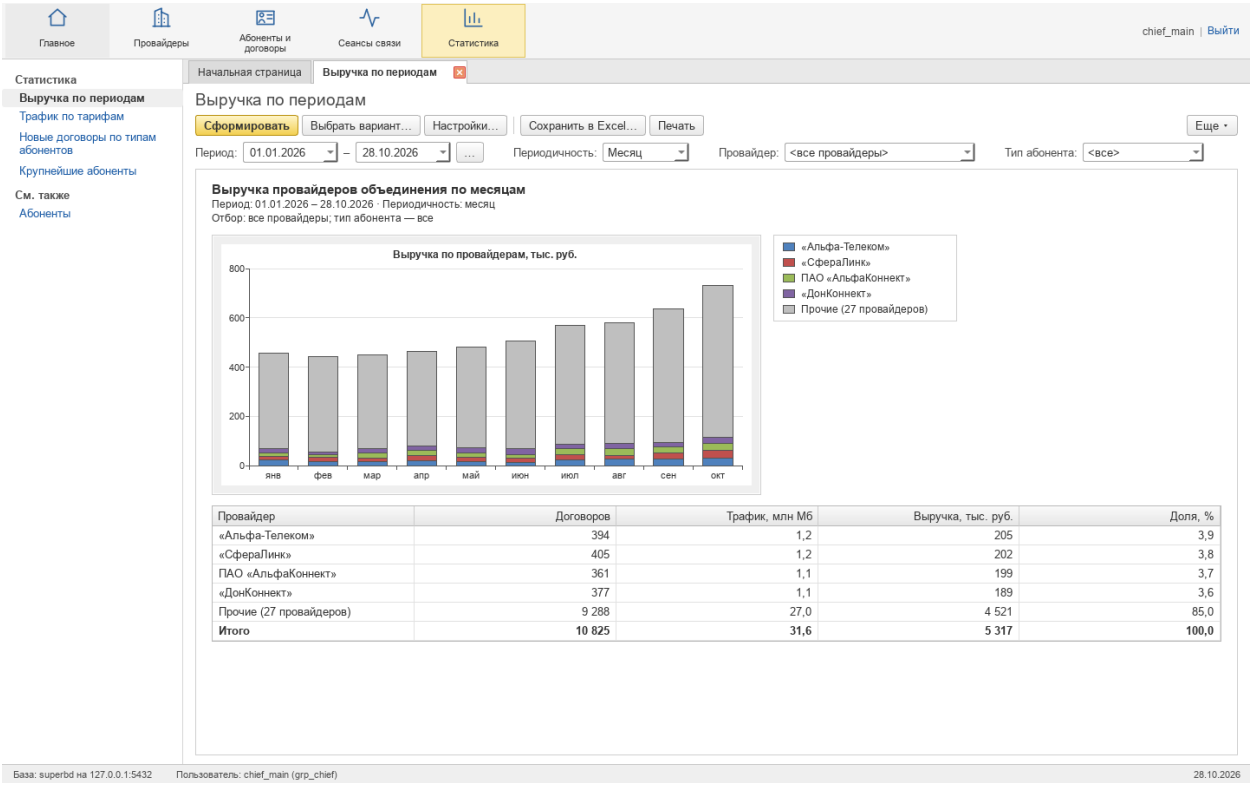



Рисунок 4.48 – Результат запроса 17 в отчёте «Выручка по периодам» (главный аналитик)

Корректность запросов проверена автоматическими тестами (раздел 6): результат каждого запроса сравнивается с независимым расчётом, проверяются видимость строк для каждой роли, итоги, постраничный вывод и поиск.

## 4.8 Создание представлений и хранимых процедур, функций

Клиентское приложение обращается к данным только через схему api: шесть представлений (запросы без параметров) и функции (запросы с параметрами, изменение данных, управление пользователями, сведения для начальной страницы и администратора БД). Представления с данными предметной области создаются с параметром security_barrier и ограничивают строки условием на массив видимых провайдеров; функции оформлены по единому шаблону (рисунок 4.49): SECURITY DEFINER, фиксированные search_path и часовой пояс, проверка группы первой операцией, проверка параметров с понятным пользователю сообщением, выборка с ограничением

страницы. Перечень объектов схемы api с группами, которым они доступны, приведён в таблице 4.17 (О — оператор, Р — руководитель, ГА — главный аналитик, АП — администратор приложения, АБД — администратор БД).

```
CREATE OR REPLACE FUNCTION api.q_shared_subscribers(
    p_min_providers integer DEFAULT 2,
    p_limit integer DEFAULT 20,
    p_offset integer DEFAULT 0)
RETURNS TABLE (
    id integer, name text, subscriber_type text, city text, address text, phone text,
    providers bigint, total_count bigint)
LANGUAGE plpgsql STABLE SECURITY DEFINER
SET search_path = pg_catalog, pg_temp
SET TimeZone = 'Europe/Moscow'
AS $$
BEGIN
    PERFORM sec.require_group(ARRAY['grp_admin']);
    IF p_min_providers IS NULL OR p_min_providers < 2 THEN
        RAISE EXCEPTION 'Общий абонент — абонент не меньше чем двух провайдеров' USING ERRCODE = 'P0001';
    END IF;
    RETURN QUERY
    SELECT s.id, s.name::text, st.name::text, ci.name::text, s.address::text, s.phone::text,
           count(DISTINCT c.provider_id), count(*) OVER ()
    FROM core.subscriber s
    JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
    JOIN ref.city ci            ON ci.id = s.city_id
    JOIN core.contract c        ON c.subscriber_id = s.id
    GROUP BY s.id, st.id, ci.id
    HAVING count(DISTINCT c.provider_id) >= p_min_providers
    ORDER BY s.name, s.id
    LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
END $$;
```

Рисунок 4.49 – Функция-список по единому шаблону (api.q_shared_subscribers)

Таблица 4.17 – Представления и функции схемы api

| Объект | Вид | Назначение | Группы |
|---|---|---|---|
| api.v_contracts | представление | Запрос 7. Договоры с абонентом, типом абонента, провайдером и данными подключения | О, Р, ГА |
| api.v_manageable_groups | представление | Группы, учётные записи которых создаёт и блокирует текущий администратор | АП, АБД |
| api.v_providers | представление | Запрос 5. Провайдеры с типом собственности и городом | О, Р, ГА, АП |
| api.v_revenue_items | представление | Запрос 15а. Доходные операции | Р, ГА |
| api.v_subscribers | представление | Запрос 6. Абоненты с типом абонента и городом | О, Р, ГА, АП |
| api.v_user_groups | представление | Запрос 11. Пользователи по группам | АП, АБД |
| api.change_own_password | функция | Смена своего пароля | О, Р, ГА, АП |
| api.contract_create | функция | Новый договор своего провайдера с абонентом | О |

| Объект | Вид | Назначение | Группы |
|---|---|---|---|
| api.contract_update | функция | Изменение договора своего провайдера | О |
| api.db_connections | функция | Подключения к базе | АБД |
| api.db_tables | функция | Состояние таблиц проекта | АБД |
| api.home_todo | функция | Начальная страница, блок «Текущие дела» | О, Р, ГА, АП, АБД |
| api.lookup_subscriber_types | функция | Список выбора «Тип абонента» | О, Р, ГА, АП |
| api.lookup_tariffs | функция | Список выбора «Тариф» | О, Р, ГА, АП |
| api.q_change_log | функция | Журнал изменений за период | АБД |
| api.q_change_log_fields | функция | Поля записи журнала изменений | АБД |
| api.q_contract_sessions | функция | Запрос 2. Сеансы выбранного договора за период с тарифом, скидкой, стоимостью и итогами | О, Р, ГА |
| api.q_contracts | функция | Запрос 4. Договоры, заключённые в периоде, с абонентом, типом абонента, провайдером и состоянием | О, Р, ГА |
| api.q_idle_contracts | функция | Запрос 14. Действующие договоры своего провайдера без сеансов больше N дней | О, Р |
| api.q_provider_staff | функция | Запрос 1. Сотрудники провайдера | Р, ГА, АП |
| api.q_providers | функция | Запрос 10. Провайдеры (представление v_providers) с числом действующих и новых за период договоров из подзапроса, включая провайдеров без договоров (запрос на запросе, левое соединение) | Р, ГА, АП |
| api.q_sessions | функция | Запрос 3. Сеансы за период с договором, абонентом, IP, тарифом и стоимостью | О, Р, ГА |
| api.q_shared_subscribers | функция | Запрос 13. Общие абоненты — с договорами не меньше чем N провайдеров (HAVING, N ≥ 2), их сведения меняет администратор приложения | АП |
| api.q_subscribers | функция | Запрос 8. Абоненты с типом, городом и числом договоров видимых провайдеров | О, Р, ГА |
| api.q_tariff_usage | функция | Запрос 9. Использование тарифов за период | Р, ГА |
| api.q_users | функция | Пользователи системы видимых групп | АП, АБД |
| api.report_new_contracts | функция | Запрос 12. Новые договоры по периодам | Р, ГА |
| api.report_provider_shares | функция | Запрос 17. Доли провайдеров в выручке за период | ГА |
| api.report_revenue | функция | Запрос 15б. Выручка по периодам | Р, ГА |
| api.report_top_subscribers | функция | Отчёт «Крупнейшие абоненты». Первые N абонентов по выручке за период | Р, ГА |
| api.subscriber_delete | функция | Удаление абонента вместе с его договорами и сеансами | О |
| api.subscriber_find_exact | функция | Абонент реестра с точно таким наименованием | О |
| api.subscriber_update | функция | Изменение сведений абонента в реестре | О, АП |
| api.user_create | функция | Новый пользователь | АП, АБД |
| api.user_reset_password | функция | Сброс пароля сотрудника управляемой группы | АП, АБД |
| api.user_set_login | функция | Разрешить или запретить вход пользователю управляемой группы | АП, АБД |
| api.whoami | функция | Текущий пользователь | О, Р, ГА, АП, АБД |

Функции изменения данных (api.contract_create, api.contract_update, api.subscriber_update, api.subscriber_delete) проверяют права на конкретную строку, используют оптимистическую блокировку по версии строки row_version и возвращают новую версию; при одновременном изменении второй пользователь получает ошибку 40001 и перечитывает запись.

[ЗАГЛУШКА: модифицируемое представление (ТЗ п. 6: INSTEAD OF-триггеры, план — api.v_contract_card «абонент + договор», docs/analysis/requirements.md, раздел 5) не реализовано. Описание, рисунок и тест добавить после реализации]

# 5 РАЗРАБОТКА КЛИЕНТСКОГО ПРИЛОЖЕНИЯ

Одна программа обслуживает все роли. После входа сервер сообщает группу пользователя функцией api.whoami, и главное окно строит панель разделов по группе (таблица 5.1). Инструменты, которые роли не нужны, не показываются; руководитель и главный аналитик работают с теми же страницами, что и оператор, в режиме только просмотра — отдельных страниц одной тематики для разных ролей нет. Скрытие инструментов на клиенте — удобство; запрет выполняет сервер.

Таблица 5.1 – Разделы главного окна по группам пользователей

| **Группа** | **Разделы и формы** |
|---|---|
| Оператор | Главное; Абоненты и договоры (абоненты, договоры); Сеансы связи (журнал сеансов); Мой провайдер |
| Руководитель | Главное; Абоненты и договоры, Сеансы связи — просмотр; Статистика (выручка, трафик по тарифам, новые договоры, крупнейшие абоненты); Мой провайдер |
| Главный аналитик | Главное; Провайдеры; Абоненты и договоры, Сеансы связи — просмотр; Статистика с отбором по провайдеру |
| Администратор приложения | Главное; Администрирование (пользователи); Справочники (типы собственности, города, типы абонентов, тарифы, общие абоненты); Провайдеры |
| Администратор БД | Главное; Администрирование (пользователи, журнал изменений); База данных (таблицы, подключения) |

Начальная страница у всех ролей содержит сведения о пользователе и подключении, описание возможностей роли и блок «Текущие дела» — счётчики, которые считает функция api.home_todo; щелчок по счётчику открывает соответствующий список или отчёт. Числа счётчиков получены теми же функциями, что и открываемые списки, поэтому счётчик и список не расходятся.

[ЗАГЛУШКА: формы создания и изменения записей в клиенте не реализованы: «Абонент (создание)», «Договор (создание)», «Сеанс связи (создание)», изменение и удаление абонента и договора, справочники (типы собственности, города, типы абонентов, тарифы), «Провайдер (создание)», изменение и удаление пользователя — по нажатию кнопок выводится сообщение «Форма … будет добавлена на следующем этапе». Серверные функции для

абонентов и договоров (api.contract_create, api.contract_update, api.subscriber_update, api.subscriber_delete) реализованы и проверены тестами 9.x. Описание форм и снимки добавить после реализации]

## 5.1 Формы и компоненты для работы в роли «Оператор»

Начальная страница оператора показана на рисунке 3.4 (подраздел 3.2): счётчики «Договоры без сеансов более 30 дней», «Сеансов зарегистрировано сегодня», «Договоров заключено на этой неделе», «Договоры с наступившей датой расторжения».

Страница «Абоненты» — составная форма «один-ко-многим» (рисунок 5.1): верхняя таблица — абоненты с поиском по началу наименования или цифрам телефона и отбором по типу абонента и наличию действующих договоров (запрос 8), средняя — договоры выбранного абонента у своего провайдера (запрос 4), нижняя — сеансы выбранного договора с итогами (запрос 2). Двойной щелчок открывает карточку абонента или договора (рисунки 4.21 и 4.23).

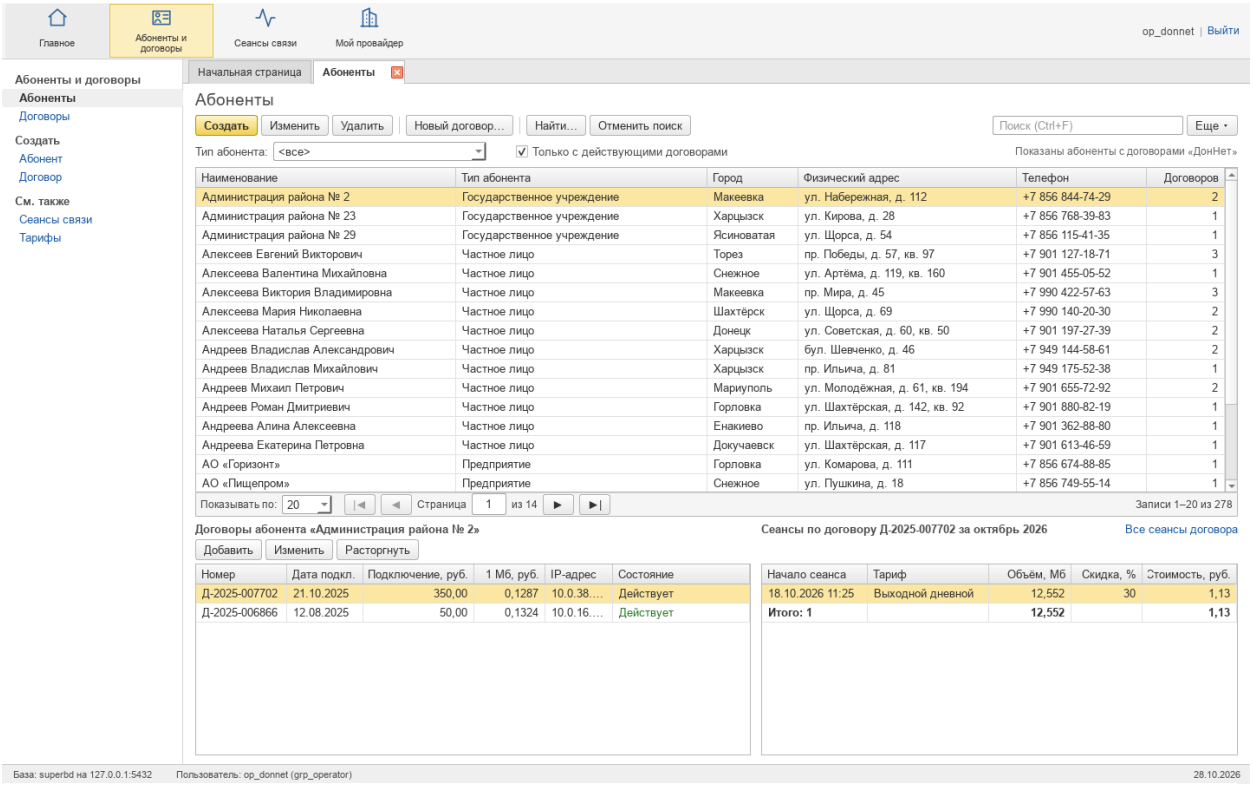



Рисунок 5.1 – Составная форма «Абоненты»: абонент → договоры → сеансы (оператор)

Страница «Договоры» выводит договоры, заключённые в периоде, с отбором по абоненту (форма выбора) и признаком «Только действующие» (рисунок 4.17); страница «Сеансы связи» — журнал сеансов за период с отбором по договору, IP-адресу или подсети и тарифу и строкой «Итого за период» (рисунок 4.15). Период задаётся полями дат или меню «…» (сегодня, эта неделя, этот месяц, прошлый месяц, этот год). Страница «Мой провайдер» — карточка своего провайдера и список его сотрудников (рисунок 4.19). Список «Договоры без сеансов» открывается счётчиком начальной страницы (рисунок 4.39).

## 5.2 Формы и компоненты для работы в роли «Руководитель»

Руководитель провайдера видит данные своего провайдера в режиме только просмотра: в заголовке формы выводится пометка «(только просмотр)», кнопки изменения не показываются (рисунок 5.2). Начальная страница руководителя содержит счётчики выручки и новых договоров за текущий месяц,

действующих договоров и договоров без сеансов (рисунок 5.3). Раздел «Статистика» описан в подразделе 5.6.

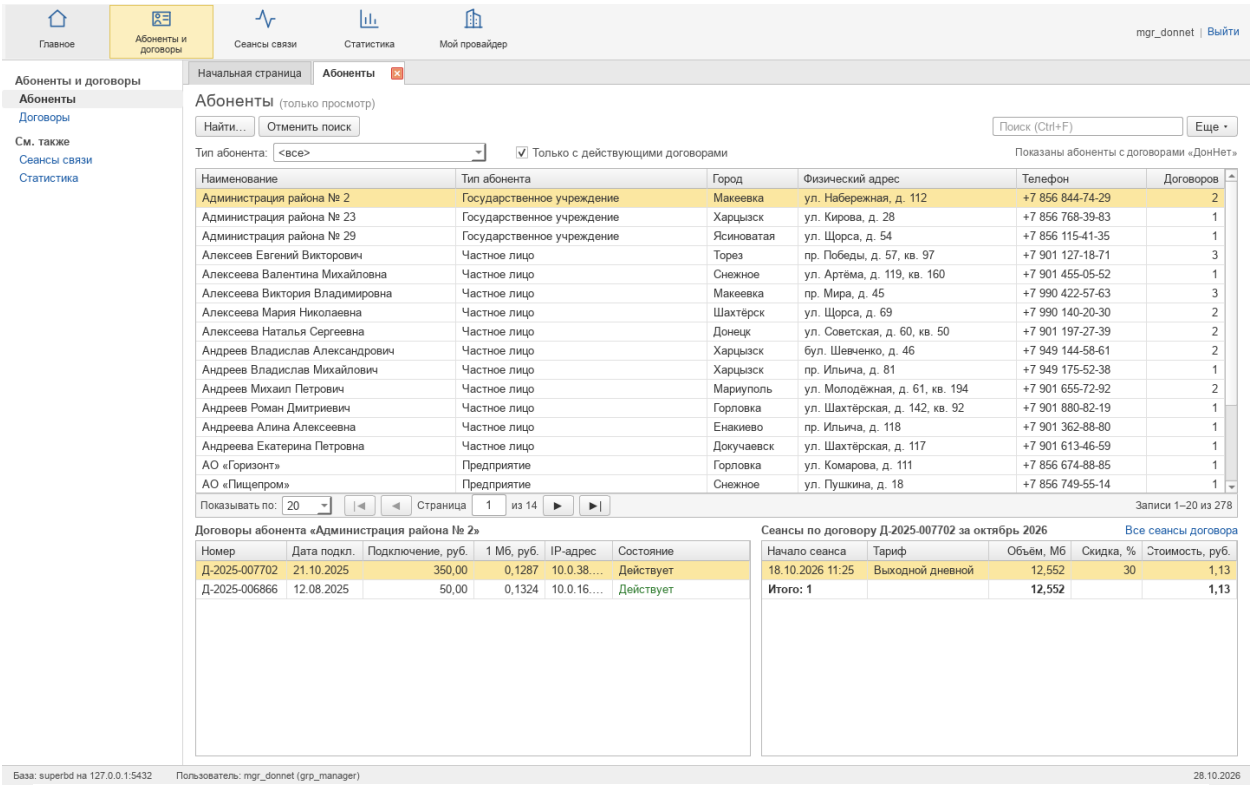



Рисунок 5.2 – Страница «Абоненты» в режиме просмотра (руководитель)

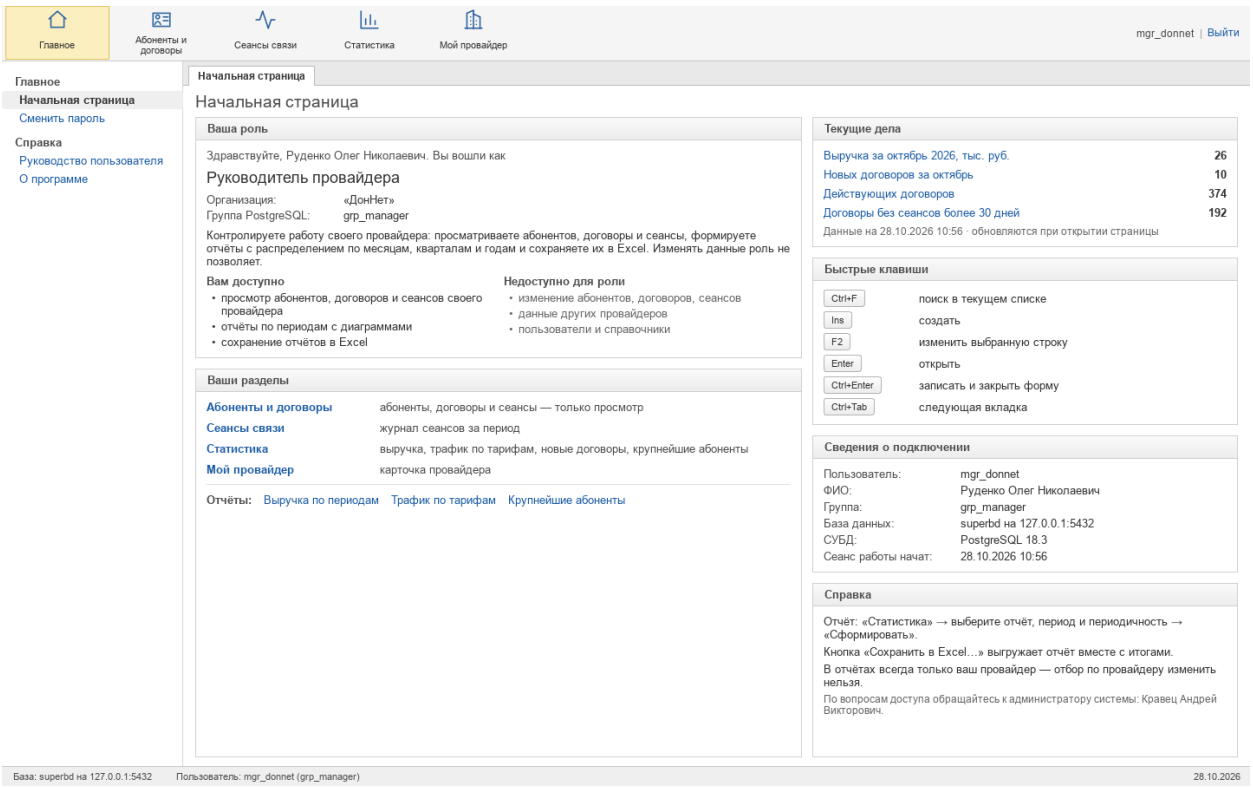



Рисунок 5.3 – Начальная страница руководителя провайдера

## 5.3 Формы и компоненты для работы в роли «Главный аналитик»

Главный аналитик объединения видит данные всех провайдеров: раздел «Провайдеры» со списком провайдеров и числом действующих и новых договоров (рисунок 4.29), абонентов всего реестра, включая абонентов без договоров (рисунок 4.25), журнал сеансов по всем провайдерам с отбором по провайдеру. В отчётах «Статистики» доступен отбор по провайдеру; без отбора отчёт о выручке показывает доли провайдеров (рисунок 4.48). Начальная страница главного аналитика показана на рисунке 5.4.

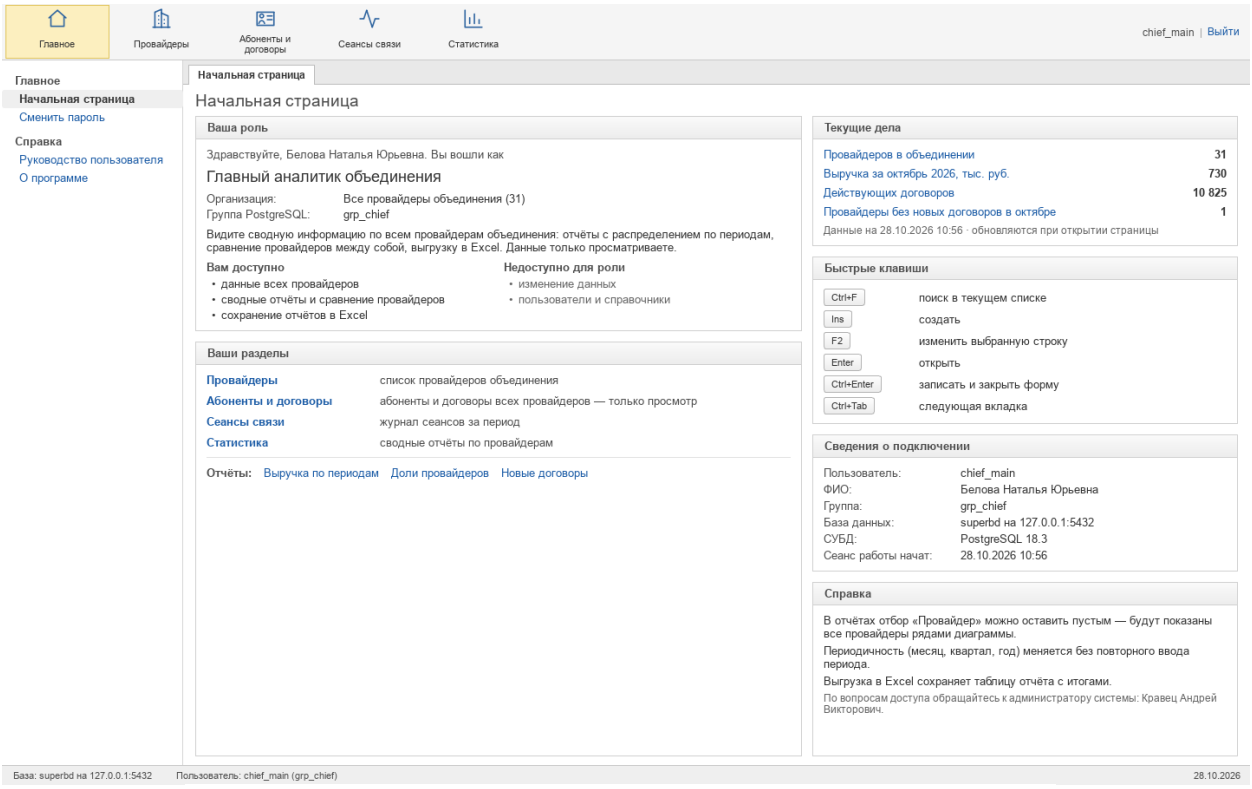



Рисунок 5.4 – Начальная страница главного аналитика

## 5.4 Формы и компоненты для работы в роли «Администратор приложения»

Администратор приложения ведёт учётные записи операторов, руководителей и главных аналитиков на странице «Пользователи» (рисунок 4.31): слева — группы с числом пользователей (запрос 11), справа —

пользователи выбранной группы с логином, ФИО, провайдером, признаком «Вход разрешён» и сроком действия пароля. Команды страницы:

– «Создать» — форма «Пользователь (создание)»: логин, ФИО, группа, провайдер (обязателен для оператора и руководителя), признак «Вход разрешён»; после записи сервер возвращает временный пароль, который показывается один раз;

– «Запретить вход» / «Разрешить вход» — функция api.user_set_login;

– «Сбросить пароль…» — подтверждение (рисунок 5.5) и окно с временным паролем на сутки; до смены временного пароля пользователю доступна только форма «Смена пароля» (рисунок 5.6).

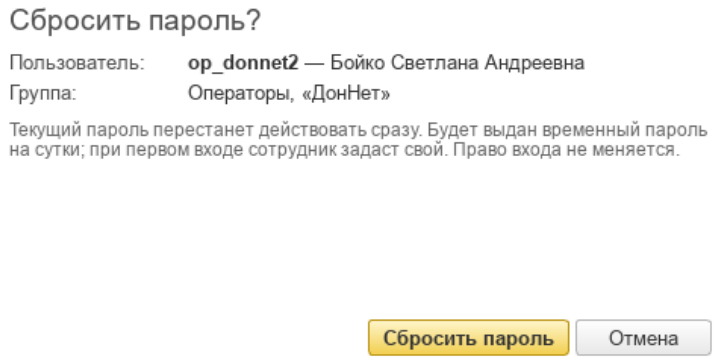



Рисунок 5.5 – Подтверждение сброса пароля

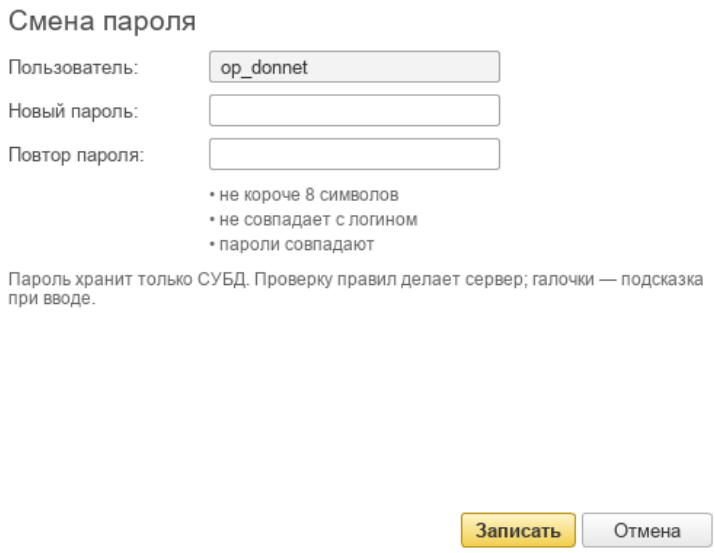



Рисунок 5.6 – Форма «Смена пароля»

В разделе «Провайдеры» администратор видит всех провайдеров и сотрудников выбранного провайдера (рисунок 4.11); отбор «Без сотрудников» показывает провайдеров, которым ещё не назначены сотрудники (рисунок 4.46). Список «Общие абоненты» раздела «Справочники» выводит абонентов, обслуживаемых несколькими провайдерами, сведения которых меняет только администратор (рисунок 4.37). Начальная страница администратора приложения показана на рисунке 5.7.

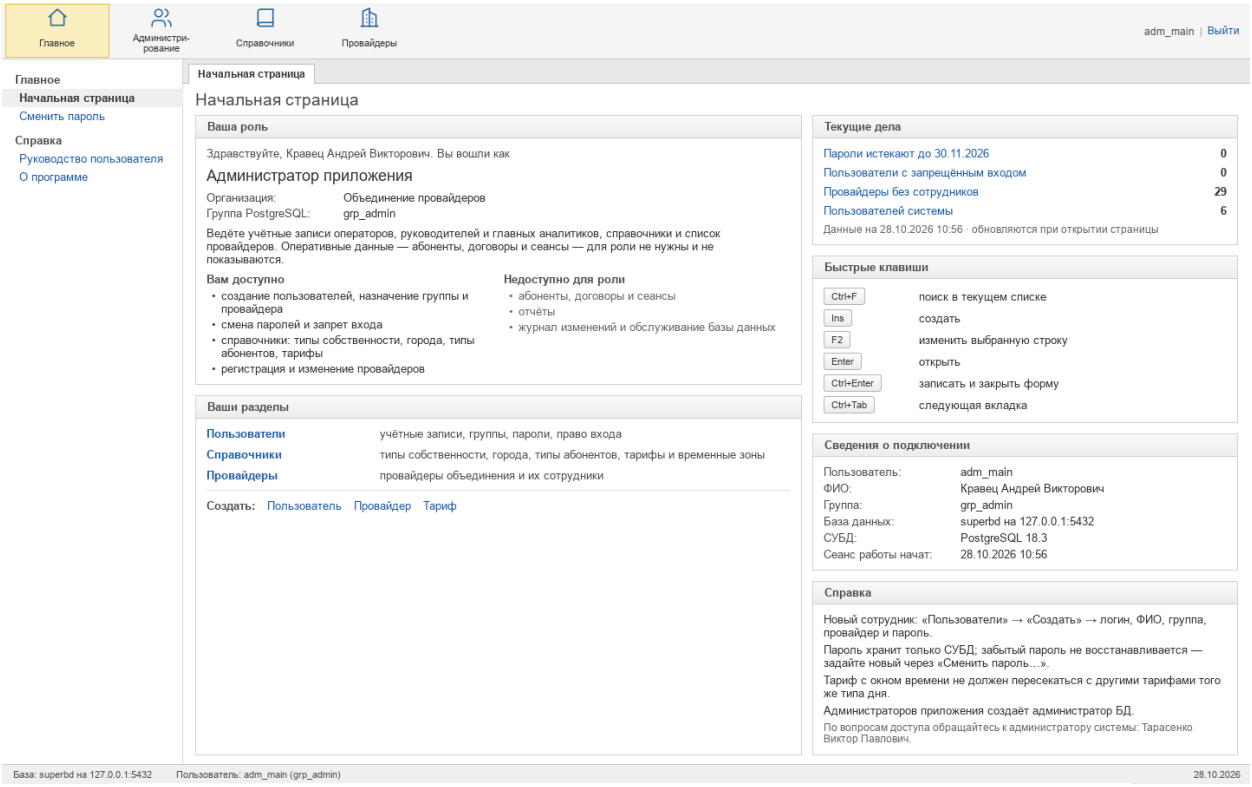



Рисунок 5.7 – Начальная страница администратора приложения

## 5.5 Формы и компоненты для работы в роли «Администратор БД»

Администратор базы данных создаёт и блокирует учётные записи администраторов приложения на той же странице «Пользователи» и обслуживает базу данных. Начальная страница показывает размер данных, число подключений, изменений в журнале за сегодня и таблиц без свежей статистики (рисунок 5.8).

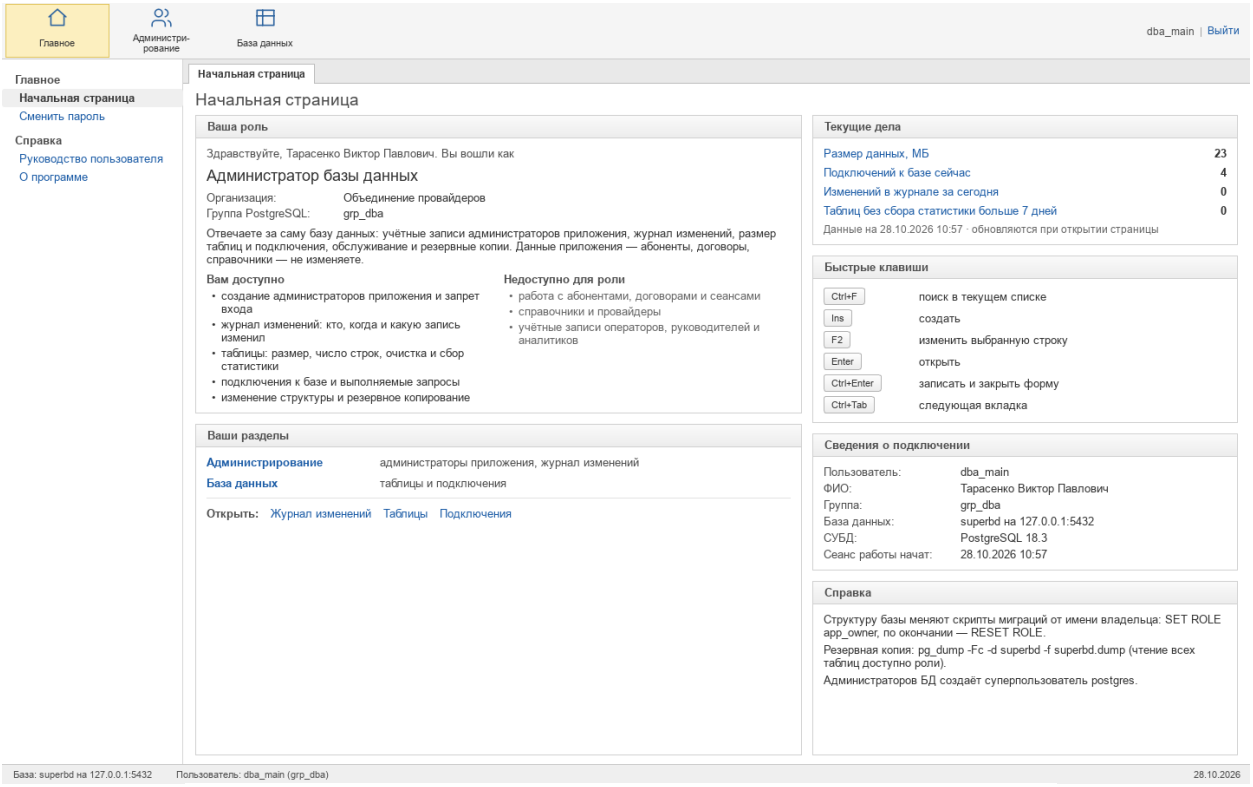



Рисунок 5.8 – Начальная страница администратора БД

Страница «Таблицы базы данных» выводит для каждой таблицы проекта назначение, оценку числа строк, размер с индексами, время последней очистки и сбора статистики, число последовательных и индексных чтений (функция api.db_tables, рисунок 5.9); страница «Подключения к базе» — текущие сеансы сервера: пользователь, программа, адрес, начало сеанса, состояние и выполняемый запрос (функция api.db_connections, рисунок 5.10).

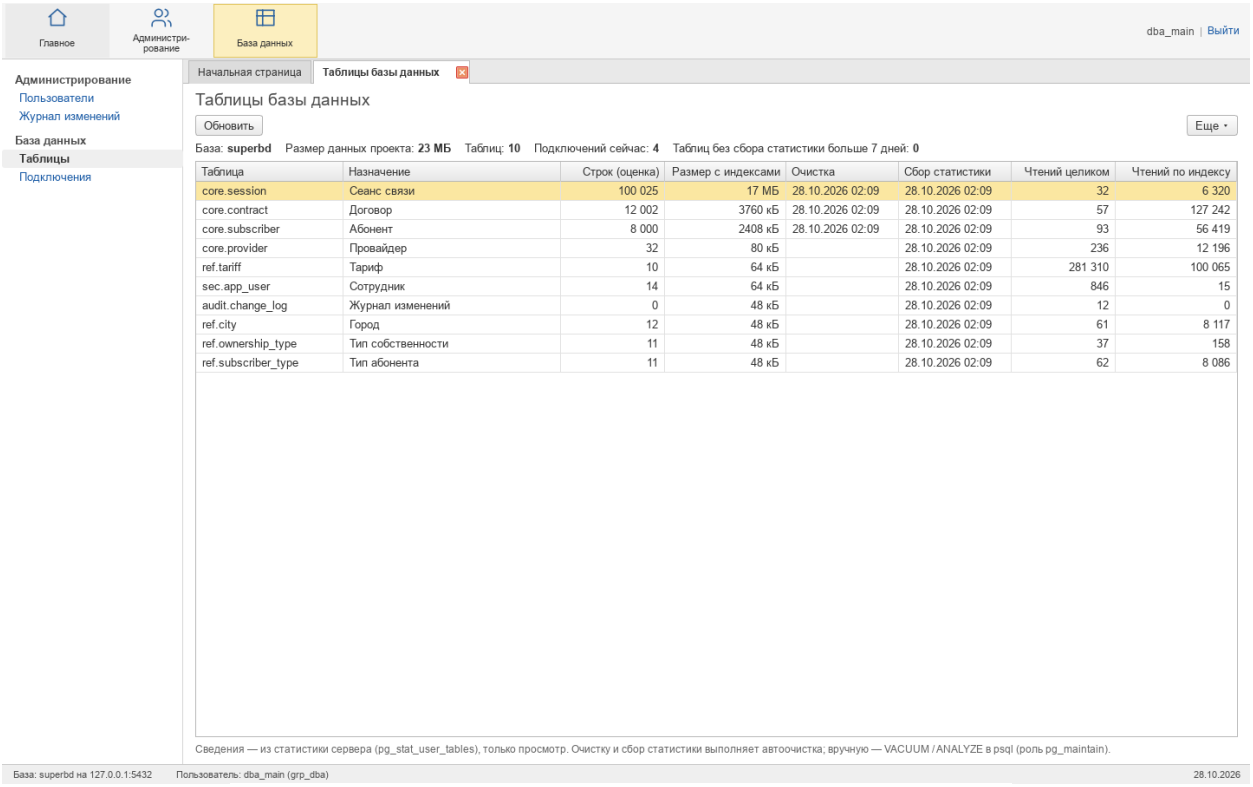



Рисунок 5.9 – Страница «Таблицы базы данных»

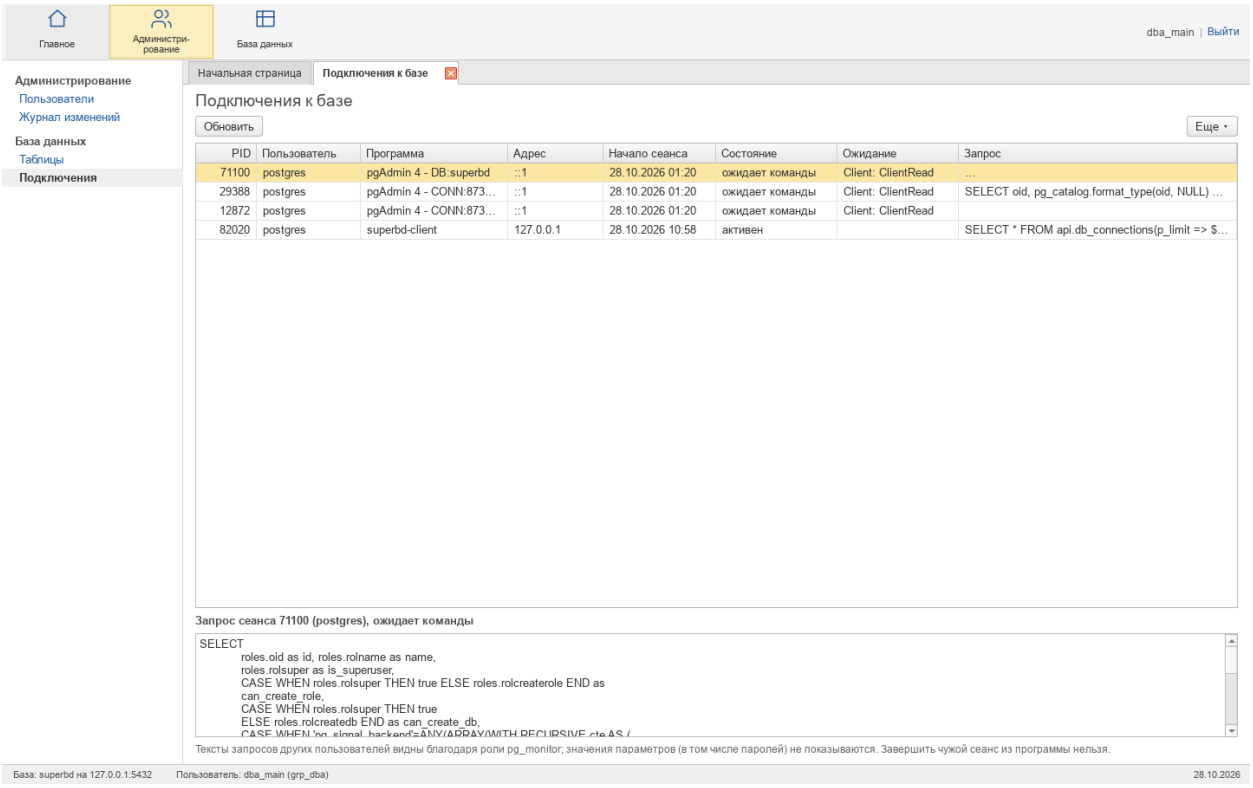



Рисунок 5.10 – Страница «Подключения к базе»

Страница «Журнал изменений» выводит записи журнала за период с отбором по таблице, операции и пользователю и показывает поля записи «было / стало» (функции api.q_change_log и api.q_change_log_fields, рисунок 5.11).

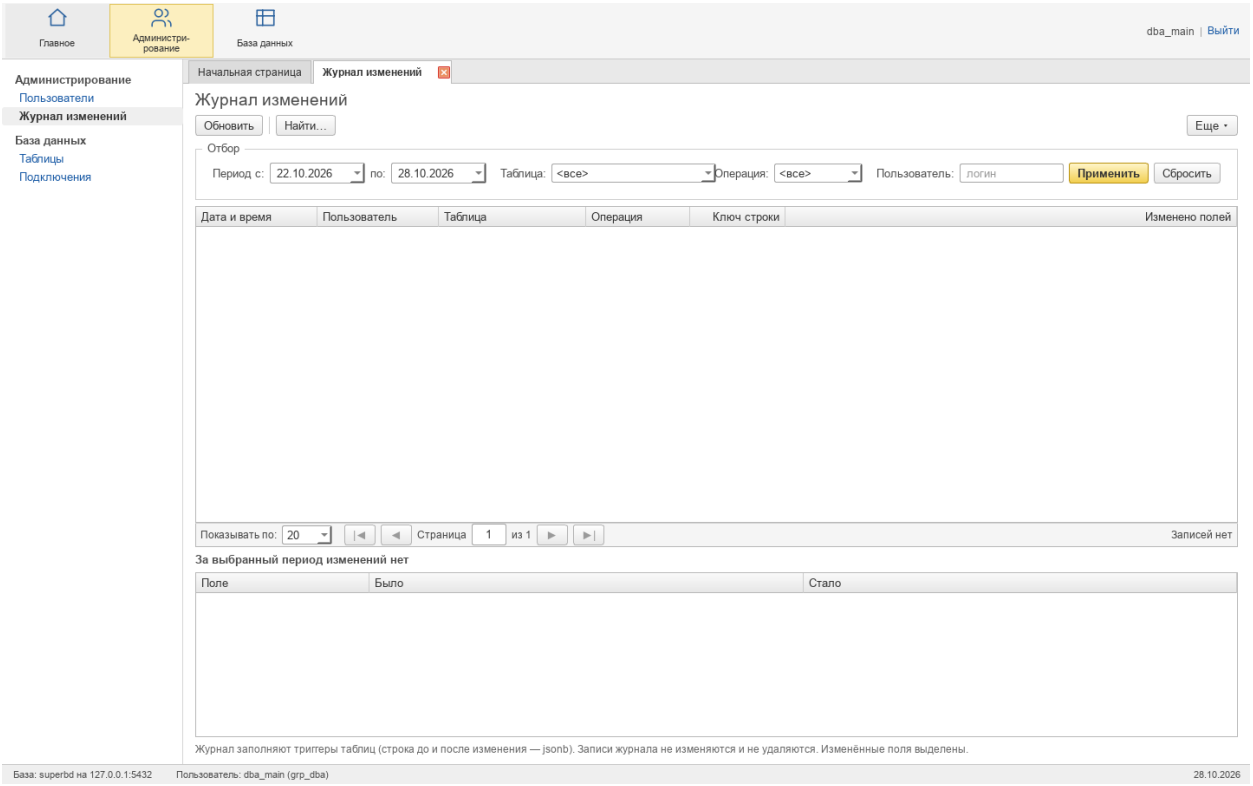



Рисунок 5.11 – Страница «Журнал изменений»

[ЗАГЛУШКА: журнал изменений пуст: записи в audit.change_log должны добавлять AFTER-триггеры таблиц (подраздел 4.3), которые ещё не реализованы. Переснять рисунок после реализации триггеров журнала]

5.6 Генерация результатов итоговых запросов (диаграммы, экспорт в Excel)

Раздел «Статистика» руководителя и главного аналитика содержит четыре отчёта, построенные по шаблону отчёта (подраздел 3.3): «Выручка по периодам» (запрос 15б, у главного аналитика без отбора — доли провайдеров, запрос 17), «Трафик по тарифам» (запрос 9), «Новые договоры по типам абонентов» (запрос 12) и «Крупнейшие абоненты». В строке параметров задаются период, периодичность (месяц, квартал, год), провайдер (у главного аналитика) и тип

абонента — обязательное распределение по временным отрезкам выполняет сервер. Отчёт состоит из табличного документа с итоговой строкой и диаграммы с легендой:

– «Выручка по периодам» — накопительные столбцы «плата за подключение» и «оплата трафика» по периодам (рисунок 4.45); у главного аналитика — столбцы по группам провайдеров: крупнейшие и «Прочие» (рисунок 4.48);

– «Новые договоры по типам абонентов» — линии по типам абонентов (рисунок 4.33);

– «Крупнейшие абоненты» — горизонтальные столбцы выручки абонентов (рисунок 4.41);

– «Трафик по тарифам» — таблица использования тарифов, включая неиспользованные (рисунок 4.27).

Команда «Сохранить в Excel…» записывает табличный документ отчёта в файл .xlsx средствами библиотеки QXlsx с заголовком отчёта, параметрами, шапкой таблицы и итоговой строкой; команда «Печать» выводит документ и диаграмму на принтер. Все вычисления выполняет сервер: клиент только раскладывает строки результата по колонкам таблицы и рядам диаграммы.

# 6 ТЕСТИРОВАНИЕ РАЗРАБОТАННОЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ

Серверная часть проверяется набором автоматических тестов db/tests, который выполняется после каждой пересборки базы данных сценарием db/build_and_test.cmd: база создаётся заново из миграций и данных, затем выполняется db/tests/run_all.sql. Тест выполняет действие от имени нужного пользователя (SET SESSION AUTHORIZATION), сравнивает результат с ожидаемым или с независимым расчётом и передаёт итог функции pg_temp.check, которая записывает его во временную таблицу результатов и выводит строку PASS или FAIL с номером и описанием проверки; изменяющие данные тесты выполняются в транзакции с откатом. В конце прогона при хотя бы одном FAIL сценарий завершается ошибкой. Состав тестов приведён в таблице 6.1.

Таблица 6.1 – Автоматические тесты серверной части

| **Группа** | **Что проверяется** | **Файл** | **Тестов** |
|---|---|---|---|
| 1 | Число таблиц, отсутствие своих таблиц пользователей и паролей, комментарии ко всем таблицам и колонкам | 010 | 5 |
| 2 | Внешние ключи и правила удаления по модели, индексы на всех внешних ключах | 010 | 3 |
| 3 | Объём и согласованность данных: не менее 10 записей в справочниках и 10 000 в основных таблицах, сеансы в периоде договора, скидка и стоимость сеанса по формуле, тариф по окну времени | 010 | 10 |
| 4 | Сотрудники: логин — роль PostgreSQL, ровно одна группа, провайдер у операторов и руководителей; триггер проверки логина | 010 | 4 |
| 5 | Каскадное удаление абонента, запрет удаления тарифа и провайдера, на которые есть ссылки, запрет дубля абонента | 010 | 5 |
| 6 | Сведения о пользователе, запрет прямого чтения таблиц, отказ пользователю без группы, функции недоступны PUBLIC, счётчики начальной страницы | 020 | 7 |
| 7 | Запросы 1–10: результат, видимость строк для ролей, постраничный вывод, поиск, экранирование символов шаблона | 030 | 19 |
| 8 | Отсутствие регулярных выражений, проверки доменов телефона, логина и обозначения тарифа | 010 | 3 |

| Группа | Что проверяется | Файл | Тестов |
|---|---|---|---|
| 9 | Общий реестр абонентов: видимость, права на изменение и удаление, оптимистическая блокировка | 040 | 16 |
| 10 | Разделение администратора приложения и администратора БД: права, создание и блокировка пользователей | 050 | 17 |
| 11 | Сброс и смена пароля, временный пароль, отказ SB001 до смены пароля | 070 | 8 |
| 12 | Запросы 11–17 и отчёт «Крупнейшие абоненты»: сравнение с независимым расчётом, итоги, права ролей | 060 | 20 |
| Всего | | | 117 |

Результат прогона на рабочей базе 28.10.2026 — пройдено 117 из 117 тестов, по файлам — на рисунке 6.1; пересборка базы со всеми данными занимает около 4 с, прогон тестов — около 1,5 с.

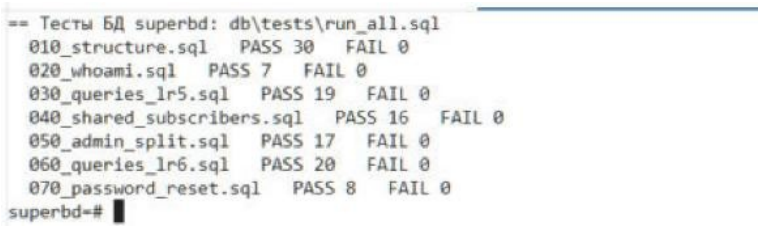



Рисунок 6.1 – Итог прогона автоматических тестов по файлам

## 6.1 Защита от несанкционированного доступа

Проверяется, что пользователь не может получить данные в обход функций api и сверх прав своей группы:

– оператор не читает таблицы схемы core напрямую — ошибка «нет доступа к таблице» (тест 6.2);

– пользователь без группы не получает доступа к схеме api (6.4), функции api и sec не выполняются от имени PUBLIC (6.5), внутреннюю функцию sec.provider_scope оператор вызвать не может (7.18);

– администратору БД закрыты экраны данных абонентов и провайдеров (10.2), администратору приложения — журнал изменений и состояние базы (10.7); администратор приложения не создаёт администраторов (10.10);

– оператор другого провайдера не видит сеансы и договоры чужого провайдера (7.5, 9.8), договор с чужим абонентом по коду не заключить — перебор кодов ничего не даёт (9.5);

– с временным паролем доступны только сведения о себе и смена пароля, остальные функции отказывают с кодом SB001 (11.3); временный пароль не сохраняется ни в одной таблице проекта (11.5);

– ввод служебных символов шаблона «%» и «_» в поиск не выбирает все строки (7.14); значения передаются параметрами, поэтому ввод вида «' OR 1=1 --» обрабатывается как обычная строка поиска.

## 6.2 Одновременная работа с данными

Изменение абонента и договора выполняется с оптимистической блокировкой: клиент передаёт прочитанную версию строки, функция изменяет строку только при совпадении версии и увеличивает её. Тест 9.11 моделирует двух пользователей: первое изменение проходит (версия 1 → 2), повторное изменение с устаревшей версией отклоняется с кодом 40001 и сообщением «Абонент изменён другим пользователем — обновите карточку». Одновременное создание абонента с тем же наименованием и телефоном не создаёт дубль благодаря уникальному индексу (тесты 5.5, 9.6, 9.10).

## 6.3 Каскадное удаление

Удаление абонента каскадно удаляет его договоры и сеансы (тест 5.1: договоров 6 → 0, сеансов 68 → 0), после отката транзакции абонент на месте (5.2). Удаление тарифа, на который ссылаются сеансы, и провайдера с договорами отклоняется ограничением RESTRICT (5.3, 5.4). Удалить общего абонента, обслуживаемого несколькими провайдерами, нельзя никому: каскадом были бы удалены чужие договоры (9.16).

## 6.4 Корректность запросов и отчётов

Результат каждого запроса сравнивается с независимым расчётом другим способом: например, список договоров без сеансов — с расчётом через NOT EXISTS (12.5), выручка за месяц — с прямой суммой операций (12.7), строка «Итого» — с суммой периодов (12.8); число строк объединения — с числом договоров и сеансов (12.9). Отдельно проверены совпадение счётчиков начальной страницы со списками (12.6, 12.10), независимость результата от часового пояса клиента (6.7), понятные сообщения при неверных параметрах (7.13, 12.16).

[ЗАГЛУШКА: не выполнены проверки плана T1–T10 (docs/analysis/requirements.md, раздел 10), требующие нереализованных частей: T7 — запрет удаления провайдера с недавними сеансами триггером, T8 — подбор тарифа и расчёт стоимости триггером при вводе сеанса, T10 — отсечение секций при запросе за квартал (после ЛР 8), а также одновременная работа двух экземпляров клиента и тесты форм ввода. Добавить после реализации триггеров, секционирования и форм]

# ЗАКЛЮЧЕНИЕ

В курсовом проекте разработана клиент-серверная информационная система учёта услуг связи интернет-провайдеров (вариант Б-16), в которой логика программного обеспечения размещена на сервере PostgreSQL, а настольное приложение на Qt 6 выполняет функции тонкого клиента.

По задачам, поставленным во введении, получены следующие результаты.

1) Построена концептуальная модель предметной области по методике IDEF1X: восемь сущностей предметной области в третьей нормальной форме и две служебные таблицы; обоснована денормализация — хранение скидки и стоимости сеанса на момент регистрации. Абоненты образуют общий реестр объединения провайдеров, договор является таблицей пересечения провайдера и абонента.

2) Создана база данных из пяти схем с десятью доменами, внешними ключами с выбранными правилами удаления, индексами под условия экранов и явной последовательностью номеров договоров; таблицы заполнены воспроизводимыми псевдослучайными данными — более 120 000 строк.

3) Реализованы пять групп пользователей средствами PostgreSQL, доступ только через функции и представления схемы api с проверкой группы, ограничение видимости строк провайдером пользователя, создание и блокировка учётных записей из приложения, сброс пароля с временным паролем.

4) Разработаны запросы всех видов, перечисленных в задании: запросы без параметров оформлены шестью представлениями, запросы с параметрами — функциями; каждый запрос является источником данных экрана, отчёта или счётчика клиентского приложения. Тексты запросов оптимизированы, использование индексов подтверждено планами выполнения.

5) Разработано клиентское приложение: интерфейс по группе пользователя, постраничный вывод, поиск и отбор на сервере, составная форма

«абонент — договоры — сеансы», отчёты с диаграммами, выгрузкой в Excel и печатью, страницы администраторов.

6) Корректность серверной части подтверждена 117 автоматическими тестами, в том числе проверками защиты от несанкционированного доступа, оптимистической блокировки и каскадного удаления.

Часть требований технического задания на момент оформления записки не реализована: триггеры (кроме проверки логина сотрудника) и модифицируемое представление, политики защиты на уровне строк, секционирование таблицы сеансов (лабораторная работа № 8), формы создания и изменения абонентов, договоров, сеансов и справочников в клиенте. Для них в записке оставлены отметки, а порядок реализации определён в документации проекта.

Предложения по развитию системы: перевести ограничение строк с условий в функциях на политики RLS с сохранением функций api как единственной точки доступа, заполнять журнал изменений триггерами, секционировать таблицу сеансов по году и кварталу, добавить загрузку учётных записей сеансов из биллинговых систем провайдеров в формате JSON.

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1) Карпова, Т. С. Базы данных: модели, разработка, реализация : учебник для вузов / Т. С. Карпова. — Санкт-Петербург : Питер, 2002. — 304 с.

2) Ильиных, Т. Е. Проектирование реляционных баз данных в нотациях IDEF1X / Т. Е. Ильиных, Л. И. Шустова. — Москва : МИФИ, 2000. — 136 с.

3) Лузанов, П. В. Postgres. Первое знакомство / П. В. Лузанов, Е. В. Рогов, И. В. Лёвшин. — 12-е изд. — Москва : Постгрес Профессиональный, 2026.

4) Шёниг, Г.-Ю. PostgreSQL 11. Мастерство разработки / Г.-Ю. Шёниг. — Москва : ДМК Пресс, 2019. — 353 с.

5) Методические рекомендации по выполнению курсового проекта по дисциплине «Программирование систем с серверами баз данных» / А. В. Чернышова, С. В. Щедрин, С. С. Бердюкова, Д. А. Филипишин. — Донецк : ДонНТУ, 2024.

6) PostgreSQL 18 Documentation / The PostgreSQL Global Development Group. — URL: https://www.postgresql.org/docs/18/ (дата обращения: 28.10.2026). — Текст : электронный.

7) Qt 6 Documentation / The Qt Company. — URL: https://doc.qt.io/qt-6/ (дата обращения: 28.10.2026). — Текст : электронный.

8) QXlsx — библиотека чтения и записи файлов Excel для Qt. — URL: https://github.com/QtExcel/QXlsx (дата обращения: 28.10.2026). — Текст : электронный.

# ПРИЛОЖЕНИЕ А.
# ЛИСТИНГ ШАБЛОНОВ

В приложении приведены общие классы клиента, из которых собираются страницы для работы с таблицами базы данных: вызов функций api (листинги А.1 и А.2), постраничная модель (листинги А.3 и А.4), таблица со страницами и панель страниц (листинги А.5 и А.6), страница-список, составная форма «один-ко-многим» и отчёт по шаблону (листинги А.7–А.9).

Листинг А.1 – Класс ApiCall (ApiCall.h)

```
class ApiCall {
public:
  static bool select(const QString &function, const QList<ApiParam> &params, QSqlQuery *query,
          QString *error);
  static bool exec(const QString &function, const QList<ApiParam> &params, QString *error);

  static bool selectView(const QString &view, const QString &orderBy, int limit, int offset,
          QSqlQuery *query, QString *error);
  static bool selectViewRow(const QString &view, const QString &keyField, const QVariant &key,
          QSqlQuery *query, QString *error);
};
```

Листинг А.2 – Вызов функции и страницы представления api (ApiCall.cpp)

```
bool ApiCall::select(const QString &function, const QList<ApiParam> &params, QSqlQuery *query, QString *error)
{
  QStringList names{function};
  QStringList args;
  for (const ApiParam &p : params) {
    names << p.name;
    args << QStringLiteral("%1 => :%1").arg(p.name);
  }
  if (!checkIdentifiers(names, error))
    return false;
  return run(QStringLiteral("SELECT * FROM api.%1(%2)").arg(function, args.join(QStringLiteral(", "))),
      params, query, error);
}

bool ApiCall::exec(const QString &function, const QList<ApiParam> &params, QString *error)
{
  QSqlQuery q;
  return select(function, params, &q, error);
}

bool ApiCall::selectView(const QString &view, const QString &orderBy, int limit, int offset,
          QSqlQuery *query, QString *error)
{
  if (!checkIdentifiers({view, orderBy}, error))
    return false;
  return run(QStringLiteral("SELECT *, count(*) OVER () AS total_count FROM api.%1 "
              "ORDER BY %2 LIMIT :p_limit OFFSET :p_offset").arg(view, orderBy),
      {{QStringLiteral("p_limit"), qBound(1, limit, 200)},
       {QStringLiteral("p_offset"), qMax(0, offset)}},
      query, error);
}
```

Листинг А.3 – Класс PagedTableModel (PagedTableModel.h)

```
class PagedTableModel : public QAbstractTableModel {
  Q_OBJECT
public:
  explicit PagedTableModel(QObject *parent = nullptr);
  …
  void setFilter(const QList<ApiParam> &params);
  void setTotals(const QList<TotalCell> &cells);
  void setPage(int offset, int limit);
  bool reload();
  void clear();
  …
  QVariant rowId(int row) const;
  QVariant value(int row, const QString &field) const;
  …
  bool isTotalsRow(int row) const;
  …
  static QString format(const ColumnSpec &col, const QVariant &value);
…
signals:
  void loaded();
  void failed(const QString &message);
…
private:
…
};
```

Листинг А.4 – Загрузка страницы списка (PagedTableModel::reload)

```
bool PagedTableModel::reload()
{
  QSqlQuery q;
  bool ok;
  if (m_isView) {
  …
  if (!ok) {
    clear();
    emit failed(m_error);
    return false;
  …
  beginResetModel();
  …
  m_total = 0;
  …
  while (q.next()) {
    …
    for (int i = 0; i < rec.count(); ++i)
  …
  if (!m_rows.isEmpty()) {
    m_total = m_rows.first().value(QStringLiteral("total_count")).toLongLong();
    const QLocale ru(QLocale::Russian, QLocale::Russia);
    for (const TotalCell &cell : m_totalCells) {
      …
      if (!cell.source.isEmpty()) {
        …
        for (const ColumnSpec &c : m_columns)
          if (c.field == cell.column)
        …
        if (text.isEmpty() && col)
        …
        if (cell.forms.size() == 3) {
  …
  endResetModel();
  …
  emit loaded();
  return true;
}
```

Листинг А.5 – Таблица со страницами (DataGrid::DataGrid)

```
DataGrid::DataGrid(QWidget *parent, bool withPager, int limit)
  : QWidget(parent)
  , m_model(new PagedTableModel(this))
  , m_view(new QTableView(this))
  …
  if (withPager) {
    m_pager = new PagerBar(this);
    …
    connect(m_pager, &PagerBar::pageRequested, this, [this](int offset, int lim) {
      …
      reload();
  …
  connect(m_model, &PagedTableModel::failed, this, [this](const QString &message) {
    FormKit::error(this, message);
  …
  connect(m_view->selectionModel(), &QItemSelectionModel::currentRowChanged, this,
      …
      emit currentChanged(current.isValid() ? m_model->rowId(current.row()) : QVariant());
  …
  connect(m_view, &QTableView::doubleClicked, this, [this](const QModelIndex &index) {
    if (index.isValid() && !m_model->isTotalsRow(index.row()))
      emit activated(m_model->rowId(index.row()));
…
}
```

Листинг А.6 – Панель страниц (PagerBar::setState)

```
void PagerBar::setState(int offset, int limitValue, qint64 total, int rowsOnPage)
{
  m_offset = offset;
  m_total = total;
  const int pages = qMax<qint64>(1, (total + limitValue - 1) / limitValue);
  const int page = offset / limitValue + 1;
  {
    const QSignalBlocker block(m_page);
    m_page->setMaximum(pages);
    m_page->setValue(page);
  }
  {
    const QSignalBlocker block(m_limit);
    const int idx = m_limit->findText(QString::number(limitValue));
    if (idx >= 0)
      m_limit->setCurrentIndex(idx);
  }
  const QLocale ru(QLocale::Russian, QLocale::Russia);
  m_pages->setText(QStringLiteral("из %1").arg(ru.toString(pages)));
  m_first->setEnabled(page > 1);
  m_prev->setEnabled(page > 1);
  m_next->setEnabled(page < pages);
  m_last->setEnabled(page < pages);
  if (total == 0 || rowsOnPage == 0)
    m_records->setText(QStringLiteral("Записей нет"));
  else
    m_records->setText(QStringLiteral("Записи %1–%2 из %3")
            .arg(ru.toString(offset + 1), ru.toString(offset + rowsOnPage),
ru.toString(total)));
}
```

Листинг А.7 – Страница-список по шаблону (IdleContractsPage)

```
IdleContractsPage::IdleContractsPage(const Whoami &me, QWidget *parent)
  : QWidget(parent)
{
  Q_UNUSED(me);
  …
  FormKit::CommandBar bar(this);
  …
  m_days = new QSpinBox(this);
  …
  m_grid = new DataGrid(this, true, 20);
    …
    if (id.isValid() && ! id.isNull())
      ContractDialog(id, this).exec();
  …
  connect(open, &QPushButton::clicked, this, openCard);
  connect(m_grid, &DataGrid::activated, this, openCard);
  connect(refresh, &QPushButton::clicked, this, &IdleContractsPage::reload);
  connect(m_days, &QSpinBox::editingFinished, this, &IdleContractsPage::reload);
  reload();
}
```

Листинг А.8 – Составная форма «один-ко-многим»: выбор абонента и договора (SubscribersPage)

```
void SubscribersPage::subscriberChanged(const QVariant &id)
{
  if (!id.isValid() || id.isNull()) {
    m_contractsCaption->setText(QStringLiteral("Договоры абонента"));
    m_contracts->clear();
    return;
  }
  m_contractsCaption->setText(QStringLiteral("Договоры абонента «%1»")
                .arg(m_subscribers->currentValue(QStringLiteral("name")).toString()));
  m_contracts->setFilter({{QStringLiteral("p_subscriber_id"), id}});
  m_contracts->reloadFromStart();
}

void SubscribersPage::contractChanged(const QVariant &id)
{
  const QDate today = QDate::currentDate();
  if (!id.isValid() || id.isNull()) {
    m_sessionsCaption->setText(QStringLiteral("Сеансы по договору за %1").arg(FormKit::monthTitle(today)));
    m_sessions->clear();
    return;
  }
  m_sessionsCaption->setText(QStringLiteral("Сеансы по договору %1 за %2")
                .arg(m_contracts->currentValue(QStringLiteral("contract_no")).toString(),

FormKit::monthTitle(today)));
  m_sessions->setFilter({{QStringLiteral("p_contract_id"), id},

{QStringLiteral("p_from"), QDate(today.year(), today.month(), 1)},
            {QStringLiteral("p_to"), today}});
  m_sessions->reloadFromStart();
}
```

Листинг А.9 – Отчёт по шаблону: выручка по периодам (ReportPage::buildRevenue)

```
void ReportPage::buildRevenue()
{
  const QString provider = providerTitle();

setHeading(QStringLiteral("Выручка %1 %2").arg(provider, byGrain()),
       QStringLiteral("провайдер = %1; %2").arg(provider, typeFilter()));
  …
  if (!ApiCall::select(QStringLiteral("report_revenue"), params, &q, &error)) {
    FormKit::error(this, error);
    return;
            …
QStringLiteral("Подключения, тыс. руб."), QStringLiteral("Выручка, тыс. руб.")});
  …
  while (q.next()) {
    …
    if (!total) {
      categories << periodLabel(period.toDate());
  …
  if (!categories.isEmpty())
…
}
```

# ПРИЛОЖЕНИЕ Б.
# ЛИСТИНГ СЕРВЕРНОГО ПРИЛОЖЕНИЯ

Серверное приложение состоит из сценариев каталога db: создание базы (create_db.sql), пересборка (build.sql), миграции, данные и тесты. Состав сценариев приведён в таблице Б.1; ниже приведены ключевые объекты в сокращении (листинги Б.1–Б.18), пропущенные строки отмечены знаком «…».

Таблица Б.1 – Состав сценариев серверной части

| Файл | Строк | Объекты |
|---|---|---|
| db/create_db.sql | 20 | Создание базы данных проекта и роли-владельца (выполняется один раз, из базы postgres) |
| db/build.sql | 36 | Пересборка БД курсового проекта с нуля (суперпользователем, в базе superbd) |
| db/migrations/ 010_schemas_roles_base.sql | 82 | ref, core, sec, audit, api |
| db/migrations/020_domains.sql | 47 | домены d_name, d_address, d_phone, d_year, d_contract_no, d_login, d_money, d_price_mb, d_percent, d_mb |
| db/migrations/030_ref_tables.sql | 65 | ref.ownership_type, ref.city, ref.subscriber_type, ref.tariff |
| db/migrations/ 040_core_tables.sql | 185 | core.provider, core.subscriber, core.contract_no_seq, core.contract, core.session, sec.app_user, audit.change_log |
| db/migrations/ 045_app_user_role_check.sql | 28 | sec.trg_app_user_biu_check_role, trg_app_user_biu_check_role |
| db/migrations/050_indexes.sql | 47 | индексов: 13 |
| db/migrations/ 055_sec_whoami.sql | 233 | sec.group_of, sec.current_group, sec.in_group, sec.manageable_groups, sec.visible_groups, sec.group_title, sec.current_provider_id, sec.assert_password_changed, sec.check_group, sec.require_group, api.whoami |
| db/migrations/056_api_home.sql | 156 | api.home_todo |
| db/migrations/057_sec_scope.sql | 109 | sec.sees_all_data, sec.sees_all_providers, sec.provider_scope, sec.scope_data, sec.scope_providers, sec.like_prefix |
| db/migrations/ 060_queries_simple.sql | 515 | api.v_providers, api.v_subscribers, api.v_contracts, api.q_provider_staff, api.q_contract_sessions, api.q_sessions, api.q_contracts, api.q_subscribers, api.q_tariff_usage, api.q_providers |
| db/migrations/ 061_queries_aggregate.sql | 453 | sec.check_report_period, sec.period_title, sec.count_title, api.v_revenue_items, api.v_user_groups, api.report_new_contracts, api.q_shared_subscribers, api.q_idle_contracts, api.report_revenue, api.report_provider_shares, api.report_top_subscribers |
| db/migrations/ 065_api_lookups.sql | 35 | api.lookup_subscriber_types, api.lookup_tariffs |
| db/migrations/ 066_api_subscriber_write.sql | 304 | sec.to_inet, api.subscriber_find_exact, api.contract_create, api.subscriber_update, api.subscriber_delete, sec.check_subscriber_edit, api.contract_update |
| db/migrations/ 070_api_admin.sql | 312 | api.q_users, api.v_manageable_groups, api.user_create, api.user_set_login, api.q_change_log, api.q_change_log_fields, api.db_tables, api.db_connections |
| db/migrations/ 075_password_reset.sql | 123 | sec.temp_password, sec.issue_temp_password, api.user_reset_password, api.change_own_password |
| db/seed/010_ref.sql | 41 | Справочники: не менее 10 осмысленных записей в каждом (ТЗ п. 4) |

| Файл | Строк | Объекты |
|---|---|---|
| db/seed/020_generate.sql | 228 | Псевдослучайные данные: провайдеры, абоненты, договоры, сеансы (ТЗ п. 4: 10 000 – 50 000+ строк) |
| db/seed/030_demo_users.sql | 59 | Демо-пользователи системы: по одному-два на группу |
| db/seed/040_for_queries.sql | 62 | Записи, согласованные с запросами ЛР 5 (задание ЛР 5, п. 3) и счётчиками начальной страницы |
| db/tests/010_structure.sql | 327 | тесты 1.1–8.3 (30) |
| db/tests/020_whoami.sql | 96 | тесты 6.1–6.7 (7) |
| db/tests/030_queries_lr5.sql | 130 | тесты 7.1–7.18 (19) |
| db/tests/ 040_shared_subscribers.sql | 178 | тесты 9.1–9.16 (16) |
| db/tests/050_admin_split.sql | 212 | тесты 10.1–10.16 (17) |
| db/tests/060_queries_lr6.sql | 248 | тесты 12.1–12.18 (20) |
| db/tests/070_password_reset.sql | 136 | тесты 11.1–11.8 (8) |
| db/tests/_helpers.sql | 30 | Общие помощники тестов (подключаются из run_all.sql первыми) |
| db/tests/run_all.sql | 44 | запуск всех тестов и итог прогона |

Листинг Б.1 – Создание базы данных и роли-владельца (create_db.sql)

```
\set ON_ERROR_STOP on

DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_catalog.pg_roles WHERE rolname = 'app_owner') THEN
    CREATE ROLE app_owner NOLOGIN NOSUPERUSER NOCREATEDB CREATEROLE NOINHERIT;
  END IF;
END $$;
COMMENT ON ROLE app_owner IS 'Владелец объектов БД superbd (схемы, таблицы, функции); входа в систему не имеет';

SELECT 'CREATE DATABASE superbd ENCODING "UTF8" TEMPLATE template0'
WHERE NOT EXISTS (SELECT 1 FROM pg_catalog.pg_database WHERE datname = 'superbd')
\gexec

COMMENT ON DATABASE superbd IS 'Учёт услуг связи интернет-провайдеров (курсовой проект, вариант Б-16)';
```

Листинг Б.2 – Пересборка базы данных (build.sql)

```
\set ON_ERROR_STOP on
SET client_min_messages = warning;
\echo 'Сборка БД superbd'

DROP SCHEMA IF EXISTS api, audit, sec, core, ref CASCADE;

\echo '  миграции'
\ir migrations/010_schemas_roles_base.sql
\ir migrations/020_domains.sql
\ir migrations/030_ref_tables.sql
\ir migrations/040_core_tables.sql
\ir migrations/045_app_user_role_check.sql
\ir migrations/050_indexes.sql
\ir migrations/055_sec_whoami.sql
\ir migrations/056_api_home.sql
\ir migrations/057_sec_scope.sql
\ir migrations/060_queries_simple.sql
\ir migrations/061_queries_aggregate.sql
\ir migrations/065_api_lookups.sql
\ir migrations/066_api_subscriber_write.sql
\ir migrations/070_api_admin.sql
\ir migrations/075_password_reset.sql

\echo '  данные'
\ir seed/010_ref.sql
\ir seed/020_generate.sql
\ir seed/030_demo_users.sql
\ir seed/040_for_queries.sql

ANALYZE;
\echo 'Готово'
```

Листинг Б.3 – Группы пользователей и права (010_schemas_roles_base.sql)

```
DO $$
DECLARE
  g text;
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_catalog.pg_roles WHERE rolname = 'app_owner') THEN
    CREATE ROLE app_owner NOLOGIN NOSUPERUSER NOCREATEDB CREATEROLE NOINHERIT;
    …
    IF NOT EXISTS (SELECT 1 FROM pg_catalog.pg_roles WHERE rolname = g) THEN
      EXECUTE format('CREATE ROLE %I NOLOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE INHERIT', g);
  …
```

```
  EXECUTE format('REVOKE ALL ON DATABASE %I FROM PUBLIC', current_database());
  EXECUTE format('GRANT CONNECT, TEMPORARY ON DATABASE %I TO grp_admin, grp_dba, grp_operator, grp_manager, grp_chief', current_database());
  …
  EXECUTE format('GRANT CREATE ON DATABASE %I TO app_owner', current_database());
…
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
…
CREATE SCHEMA IF NOT EXISTS ref AUTHORIZATION app_owner;
```

```
CREATE SCHEMA IF NOT EXISTS core AUTHORIZATION app_owner;
CREATE SCHEMA IF NOT EXISTS sec AUTHORIZATION app_owner;
CREATE SCHEMA IF NOT EXISTS audit AUTHORIZATION app_owner;
CREATE SCHEMA IF NOT EXISTS api AUTHORIZATION app_owner;
…
REVOKE ALL ON SCHEMA ref, core, sec, audit, api FROM PUBLIC;
GRANT USAGE ON SCHEMA api TO grp_admin, grp_dba, grp_operator, grp_manager, grp_chief;
```

Листинг Б.4 – Домены (020_domains.sql)

```
DO $$
DECLARE
  d record;
BEGIN
  FOR d IN
    SELECT * FROM (VALUES
      ('d_name',       'varchar(150)', $c$VALUE = btrim(VALUE) AND VALUE <> ''$c$),
      ('d_address',    'varchar(200)', $c$btrim(VALUE) <> ''$c$),
      ('d_phone',      'varchar(20)',  $c$translate(VALUE, '+0123456789 ()-', '') = '' AND strpos(substr(VALUE, 2), '+') = 0
                       AND length(ltrim(VALUE, '+')) >= 7$c$),
```

```
      ('d_year',       'smallint',     $c$VALUE BETWEEN 1970 AND 2100$c$),
      ('d_contract_no', 'varchar(20)',  $c$VALUE = btrim(VALUE) AND VALUE <> ''$c$),
      ('d_login',      'varchar(31)',  $c$length(VALUE) >= 3
                       AND translate(left(VALUE, 1), 'abcdefghijklmnopqrstuvwxyz', '') = ''
                       AND translate(VALUE, 'abcdefghijklmnopqrstuvwxyz0123456789_', '') = ''$c$),
      ('d_money',
'numeric(12,2)', $c$VALUE >= 0$c$),
```

```
      ('d_price_mb',
'numeric(10,4)', $c$VALUE > 0$c$),
      ('d_percent',    'numeric(5,2)', $c$VALUE BETWEEN 0 AND 100$c$),
      ('d_mb',         'numeric(12,3)', $c$VALUE > 0$c$)
    ) AS t(name, base, chk)
  LOOP
    IF NOT EXISTS (SELECT 1 FROM pg_catalog.pg_type t JOIN pg_catalog.pg_namespace n ON n.oid = t.typnamespace
          WHERE n.nspname = 'ref' AND t.typname = d.name) THEN
      EXECUTE format('CREATE DOMAIN ref.%I AS %s CONSTRAINT %I CHECK (%s)',
            d.name, d.base, 'ck_' || d.name, d.chk);
    END IF;
  END LOOP;
END $$;
```

Листинг Б.5 – Справочник «Тариф» (030_ref_tables.sql)

```
CREATE TABLE IF NOT EXISTS ref.tariff (
  id          smallint GENERATED ALWAYS AS IDENTITY,
  code        varchar(3)    NOT NULL,
  name        ref.d_name    NOT NULL,
  discount_pct ref.d_percent NOT NULL DEFAULT 0,
  time_from   time,
  time_to     time,
  is_weekend  boolean,
  description text,
  CONSTRAINT pk_tariff PRIMARY KEY (id),
  CONSTRAINT uq_tariff_name UNIQUE (name),
  CONSTRAINT uq_tariff_code UNIQUE (code),
  CONSTRAINT ck_tariff_code CHECK (code <> '' AND translate(code, 'АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ', '') = ''),
  CONSTRAINT ck_tariff_window CHECK ((time_from IS NULL) = (time_to IS NULL))
);
```

Листинг Б.6 – Таблица «Сеанс связи» (040_core_tables.sql)

```
CREATE TABLE IF NOT EXISTS core.session (
  id          bigint GENERATED ALWAYS AS IDENTITY,
  started_at  timestamptz   NOT NULL,
  contract_id integer       NOT NULL,
  tariff_id   smallint      NOT NULL,
  volume_mb   ref.d_mb      NOT NULL,
  discount_pct ref.d_percent NOT NULL,
  cost        ref.d_money   NOT NULL,
  CONSTRAINT pk_session PRIMARY KEY (id, started_at),
  CONSTRAINT fk_session_contract_id FOREIGN KEY (contract_id)
    REFERENCES core.contract (id) ON DELETE CASCADE,
  CONSTRAINT fk_session_tariff_id FOREIGN KEY (tariff_id)
    REFERENCES ref.tariff (id) ON DELETE RESTRICT
);
```

Листинг Б.7 – Таблица «Сотрудник» (040_core_tables.sql)

```
CREATE TABLE IF NOT EXISTS sec.app_user (
  id          integer GENERATED ALWAYS AS IDENTITY,
  provider_id integer,
  login       ref.d_login NOT NULL,
  full_name   ref.d_name  NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now(),
  reset_pending boolean   NOT NULL DEFAULT false,
  reset_at    timestamptz,
  reset_by    ref.d_login,
  CONSTRAINT pk_app_user PRIMARY KEY (id),
  CONSTRAINT uq_app_user_login UNIQUE (login),
  CONSTRAINT fk_app_user_provider_id FOREIGN KEY (provider_id)
    REFERENCES core.provider (id) ON DELETE RESTRICT
);
```

Листинг Б.8 – Индексы (050_indexes.sql)

```
SET ROLE app_owner;

CREATE INDEX IF NOT EXISTS ix_provider_ownership_type_id ON core.provider (ownership_type_id);
CREATE INDEX IF NOT EXISTS ix_provider_city_id ON core.provider (city_id);
CREATE INDEX IF NOT EXISTS ix_subscriber_city_id ON core.subscriber (city_id);
CREATE INDEX IF NOT EXISTS ix_subscriber_subscriber_type_id ON core.subscriber (subscriber_type_id);
CREATE INDEX IF NOT EXISTS ix_session_tariff_id ON core.session (tariff_id);
CREATE INDEX IF NOT EXISTS ix_app_user_provider_id ON sec.app_user (provider_id);
…
CREATE INDEX IF NOT EXISTS ix_session_contract_id_started_at ON core.session (contract_id, started_at);
CREATE INDEX IF NOT EXISTS ix_session_started_at ON core.session (started_at);
CREATE INDEX IF NOT EXISTS ix_contract_provider_id_connected_on ON core.contract (provider_id, connected_on);
CREATE INDEX IF NOT EXISTS ix_change_log_changed_at ON audit.change_log (changed_at);
…
CREATE INDEX IF NOT EXISTS ix_contract_subscriber_id_provider_id ON core.contract (subscriber_id, provider_id);
…
CREATE UNIQUE INDEX IF NOT EXISTS uq_subscriber_lower_name_phone_digits
…
CREATE UNIQUE INDEX IF NOT EXISTS uq_contract_provider_id_ip_address_active
…
RESET ROLE;
```

Листинг Б.9 – Проверка группы пользователя (sec.check_group)

```
CREATE OR REPLACE FUNCTION sec.check_group(p_groups text[])
RETURNS void
LANGUAGE plpgsql STABLE
SET search_path = pg_catalog, pg_temp
AS $$
DECLARE
  v_group text := sec.current_group();
BEGIN
  IF v_group IS NULL THEN
    RAISE EXCEPTION 'Нет доступа: учётная запись не входит ни в одну группу пользователей системы'
      USING ERRCODE = '42501';
  END IF;
  IF NOT v_group = ANY (p_groups) THEN
    RAISE EXCEPTION 'Недостаточно прав для этого действия' USING ERRCODE = '42501';
  END IF;
END $$;
```

Листинг Б.10 – Сведения о текущем пользователе (api.whoami)

```
CREATE OR REPLACE FUNCTION api.whoami()
RETURNS TABLE (
…
LANGUAGE plpgsql STABLE SECURITY DEFINER
SET search_path = pg_catalog, pg_temp
SET TimeZone = 'Europe/Moscow'
AS $$
  …
  RETURN QUERY
  SELECT session_user::text,
    u.full_name::text,

coalesce(split_part(u.full_name, ' ', 1)
        || coalesce(' ' || nullif(left(split_part(u.full_name, ' ', 2), 1), '') || '.', '')
        || coalesce(' ' || nullif(left(split_part(u.full_name, ' ', 3), 1), '') || '.', ''),
        session_user::text),
    v_group,
    CASE v_group WHEN 'grp_admin'    THEN 'Администратор приложения'
            WHEN 'grp_dba'    THEN 'Администратор базы данных'
            WHEN 'grp_operator'    THEN 'Оператор провайдера'
            WHEN 'grp_manager'    THEN 'Руководитель провайдера'
            WHEN 'grp_chief'    THEN 'Главный аналитик объединения' END,
    CASE v_group WHEN 'grp_admin'    THEN 'администратор приложения'
            WHEN 'grp_dba'    THEN 'администратор БД'
            WHEN 'grp_operator'    THEN 'оператор'
            WHEN 'grp_manager'    THEN 'руководитель'
            WHEN 'grp_chief'    THEN 'главный аналитик' END,
    CASE WHEN p.id IS NULL THEN NULL
        WHEN ot.name = upper(ot.name) THEN ot.name || ' «' || p.name || '»'
        ELSE '«' || p.name || '»' END,
    CASE WHEN v_group IN ('grp_admin', 'grp_dba') THEN 'Объединение провайдеров'
        WHEN v_group = 'grp_chief' THEN 'Все провайдеры объединения ('
                    || (SELECT count(*) FROM core.provider) || ')'
        ELSE CASE WHEN ot.name = upper(ot.name) THEN ot.name || ' «' || p.name || '»'
            ELSE '«' || p.name || '»' END END,
  …
  LEFT JOIN ref.ownership_type ot ON ot.id = p.ownership_type_id;
END $$;
```

Листинг Б.11 – Область видимости данных (sec.scope_data)

```
CREATE OR REPLACE FUNCTION sec.scope_data(p_provider_id integer DEFAULT NULL)
RETURNS integer[]
LANGUAGE sql STABLE SECURITY DEFINER
SET search_path = pg_catalog, pg_temp
AS $$
  SELECT sec.provider_scope(sec.sees_all_data(), p_provider_id)
$$;
```

Листинг Б.12 – Журнал сеансов (api.q_sessions)

```
CREATE OR REPLACE
FUNCTION api.q_sessions(
  p_from date,
…
RETURNS TABLE (
…
LANGUAGE plpgsql STABLE
SECURITY DEFINER
SET search_path = pg_catalog,
pg_temp
SET TimeZone =
'Europe/Moscow'
SET plan_cache_mode =
force_custom_plan
AS $$
  …
  RETURN QUERY
   SELECT s.id, s.started_at, c.id,
c.contract_no::text,
     sb.id, sb.name::text,
host(c.ip_address),
     t.code::text, t.name::text,
s.volume_mb::numeric,
s.discount_pct::numeric,
     c.price_per_mb::numeric,
s.cost::numeric,
     count(*) OVER (),
sum(s.volume_mb) OVER (),
sum(s.cost) OVER ()
   FROM core.session s
   JOIN core.contract c ON c.id =
s.contract_id
   JOIN core.subscriber sb ON
sb.id = c.subscriber_id
   JOIN ref.tariff t ON t.id =
s.tariff_id
   WHERE s.started_at >= p_from
AND s.started_at < p_to + 1
    AND c.provider_id = ANY
(v_scope)
    AND (p_contract_id IS NULL
OR s.contract_id =
p_contract_id)
    AND (v_ip IS NULL OR
c.ip_address <<= v_ip)
    AND (p_tariff_id IS NULL OR
s.tariff_id = p_tariff_id)
   ORDER BY s.started_at DESC
   LIMIT least(greatest(p_limit, 1),
200) OFFSET greatest(p_offset,
0);
END $$;
```

Листинг Б.13 – Выручка по периодам (api.report_revenue)

```
CREATE OR REPLACE
FUNCTION api.report_revenue(
  p_from date,
  p_to date,
  p_grain text DEFAULT 'month',
  p_provider_id integer
DEFAULT NULL,
  p_subscriber_type_id integer
DEFAULT NULL)
RETURNS TABLE (
  period date, period_title text,
new_contracts bigint,
volume_mb numeric,
  connection_fees numeric,
traffic_fees numeric, revenue
numeric)
LANGUAGE plpgsql STABLE
SECURITY DEFINER
SET search_path = pg_catalog,
pg_temp
SET TimeZone =
'Europe/Moscow'
SET plan_cache_mode =
force_custom_plan
AS $$
BEGIN
  PERFORM
sec.require_group(ARRAY['grp_
manager', 'grp_chief']);
  PERFORM
sec.check_report_period(p_from,
p_to, p_grain);
  RETURN QUERY
  SELECT k.period, CASE
WHEN k.is_total THEN 'Итого'
ELSE sec.period_title(p_grain,
k.period) END,
    k.new_contracts,
k.volume_mb, k.connection_fees,
k.revenue - k.connection_fees,
k.revenue
  FROM (SELECT
date_trunc(p_grain, r.at)::date AS
period,

grouping(date_trunc(p_grain,
r.at)::date) = 1                AS
is_total,
      count(*) FILTER (WHERE
r.kind = 'подключение')
AS new_contracts,

coalesce(sum(r.volume_mb), 0)
AS volume_mb,
      coalesce(sum(r.amount)
FILTER (WHERE r.kind =
'подключение'), 0)  AS
connection_fees,
      coalesce(sum(r.amount), 0)
AS revenue
    FROM api.v_revenue_items r
    WHERE r.at >= p_from AND
r.at < p_to + 1
     AND (p_provider_id IS
NULL OR r.provider_id =
p_provider_id)
     AND (p_subscriber_type_id
IS NULL OR
r.subscriber_type_id =
p_subscriber_type_id)
    GROUP BY ROLLUP
(date_trunc(p_grain, r.at)::date))
k
  ORDER BY k.is_total, k.period;
END $$;
```

Листинг Б.14 – Заключение договора (api.contract_create)

```
CREATE OR REPLACE
FUNCTION api.contract_create(
  p_subscriber_id integer,
…
RETURNS TABLE (contract_id
integer, contract_no text,
subscriber_id integer,
subscriber_created boolean)
LANGUAGE plpgsql
VOLATILE SECURITY
DEFINER
SET search_path = pg_catalog,
pg_temp
SET TimeZone =
'Europe/Moscow'
AS $$
    …
    SELECT s.id, s.address INTO
v_sub, v_sub_address
    FROM core.subscriber s
    WHERE s.id =
p_subscriber_id
     AND EXISTS (SELECT 1
FROM core.contract c WHERE
c.subscriber_id = s.id AND
c.provider_id = v_provider)
    FOR KEY SHARE;
    IF NOT FOUND THEN
      RAISE EXCEPTION
'Абонент не найден среди
абонентов провайдера' USING
ERRCODE = '22023';
    END IF;
  ELSE
    IF coalesce(btrim(p_name), '')
= '' OR coalesce(btrim(p_phone),
'') = '' THEN
      RAISE EXCEPTION
'Укажите наименование и
телефон абонента' USING
ERRCODE = '22023';
    END IF;
    SELECT s.id, s.address INTO
v_sub, v_sub_address
    FROM core.subscriber s
    WHERE lower(s.name) =
lower(btrim(p_name))
     AND translate(s.phone, '+
()-', '') =
translate(btrim(p_phone), '+
()-', '')
    FOR KEY SHARE;
    IF NOT FOUND THEN
      BEGIN
        INSERT INTO
core.subscriber (name, city_id,
subscriber_type_id, address,
phone)
        VALUES (btrim(p_name),
p_city_id, p_subscriber_type_id,
p_address, btrim(p_phone))
        RETURNING id, address
INTO v_sub, v_sub_address;
        v_created := true;
      EXCEPTION WHEN
unique_violation THEN
        SELECT s.id, s.address
INTO STRICT v_sub,
v_sub_address
        FROM core.subscriber s
        WHERE lower(s.name) =
lower(btrim(p_name))
         AND translate(s.phone, '+
()-', '') =
translate(btrim(p_phone), '+ ()-',
'');
      WHEN not_null_violation
THEN
        RAISE EXCEPTION 'Для
нового абонента укажите тип,
город и адрес' USING
ERRCODE = '22023';
      END;
  …
  END;
END $$;
```

Листинг Б.15 – Создание пользователя (api.user_create)

```
CREATE OR REPLACE
FUNCTION api.user_create(
  p_login text,
…
RETURNS TABLE (id integer,
temp_password text, valid_until
timestamptz)
LANGUAGE plpgsql
VOLATILE SECURITY
DEFINER
SET search_path = pg_catalog,
pg_temp
AS $$
  …
  EXECUTE format('CREATE
ROLE %I LOGIN
NOSUPERUSER
NOCREATEDB
NOCREATEROLE INHERIT
NOREPLICATION
NOBYPASSRLS',
        v_login);
  EXECUTE format('GRANT %I
TO %I', p_group, v_login);
  EXECUTE format('COMMENT
ON ROLE %I IS %L', v_login,
'Сотрудник: ' ||
btrim(p_full_name));
  INSERT INTO sec.app_user
(provider_id, login, full_name)
  VALUES (p_provider_id,
v_login, btrim(p_full_name))
  RETURNING sec.app_user.id
INTO v_id;
  IF NOT coalesce(p_can_login,
true) THEN
    EXECUTE format('ALTER
ROLE %I NOLOGIN', v_login);
  END IF;
  v_password :=
sec.issue_temp_password(v_logi
n);
  RETURN QUERY SELECT
v_id, v_password, r.rolvaliduntil
FROM pg_catalog.pg_roles r
WHERE r.rolname = v_login;
EXCEPTION WHEN
foreign_key_violation THEN
  RAISE EXCEPTION
'Провайдер не найден' USING
ERRCODE = '22023';
END $$;
```

Листинг Б.16 – Сброс пароля (api.user_reset_password)

```
CREATE OR REPLACE FUNCTION api.user_reset_password(p_login text)
RETURNS TABLE (temp_password text, valid_until timestamptz)
LANGUAGE plpgsql
VOLATILE SECURITY DEFINER
SET search_path = pg_catalog, pg_temp
AS $$
DECLARE
  v_group text := sec.current_group();
  v_target text := sec.group_of(p_login::name);
  v_password text;
BEGIN
  PERFORM sec.require_group(ARRAY['grp_admin', 'grp_dba']);
  IF p_login = session_user THEN
    RAISE EXCEPTION 'Свой пароль меняют через «Сменить пароль»' USING ERRCODE = '22023';
  END IF;
  IF v_target IS NULL OR NOT v_target = ANY (sec.manageable_groups(v_group))
    OR NOT EXISTS (SELECT 1 FROM sec.app_user u WHERE u.login = p_login) THEN
    RAISE EXCEPTION 'Нет прав на учётную запись «%»', p_login USING ERRCODE = '42501';
  END IF;
  v_password := sec.issue_temp_password(p_login);
  RETURN QUERY SELECT v_password, r.rolvaliduntil
  FROM pg_catalog.pg_roles r WHERE r.rolname = p_login;
END $$;
```

Листинг Б.17 – Генерация сеансов связи (seed/020_generate.sql)

```
WITH ids AS (SELECT array_agg(id ORDER BY id) AS ids FROM core.contract),
  pick AS (SELECT g, pg_temp.pick(ids.ids) AS contract_id FROM ids, generate_series(1, 100000) g),
  s AS (
    SELECT p.g, ct.id AS contract_id, ct.price_per_mb, st.name AS subscriber_type,

      ct.connected_on::timestamptz
        + random() * (least(coalesce(ct.closed_on + 1, current_date)::timestamptz, current_date::timestamptz)
            - ct.connected_on::timestamptz) AS started_at,
      round((1 + random() ^ 3 * 1999)::numeric, 3) AS volume_mb,
      random() AS manual_roll
    FROM pick p
    JOIN core.contract ct ON ct.id = p.contract_id
    JOIN core.subscriber sb ON sb.id = ct.subscriber_id
    JOIN ref.subscriber_type st ON st.id = sb.subscriber_type_id),
  t AS (
    SELECT s.*,
      CASE WHEN s.subscriber_type IN ('Школа', 'Колледж', 'Вуз') AND s.manual_roll < 0.40
           THEN (SELECT id FROM ref.tariff WHERE name = 'Образовательный')
        WHEN s.manual_roll > 0.97
           THEN (SELECT id FROM ref.tariff WHERE name = 'Акционный')
        ELSE (SELECT tr.id FROM ref.tariff tr
          WHERE tr.time_from IS NOT NULL
            AND tr.is_weekend = (extract(isodow FROM s.started_at) IN (6, 7))
            AND CASE WHEN tr.time_from < tr.time_to
                  THEN s.started_at::time >= tr.time_from AND s.started_at::time < tr.time_to
                  ELSE s.started_at::time >= tr.time_from OR s.started_at::time < tr.time_to
                END)
      END AS tariff_id
    FROM s)
INSERT INTO core.session (started_at, contract_id, tariff_id, volume_mb, discount_pct, cost)
SELECT t.started_at, t.contract_id, t.tariff_id, t.volume_mb, tr.discount_pct,
  round(t.volume_mb * t.price_per_mb * (1 - tr.discount_pct / 100), 2)
FROM t
JOIN ref.tariff tr ON tr.id = t.tariff_id
ORDER BY t.started_at;
```

Листинг Б.18 – Помощники тестов (tests/_helpers.sql)

```
SET client_min_messages = warning;
DROP TABLE IF EXISTS pg_temp.test_result;
SET client_min_messages = notice;
CREATE TEMP TABLE test_result (
  n      serial,
  name   text    NOT NULL,
  ok     boolean NOT NULL,
  detail text
);

CREATE OR REPLACE FUNCTION pg_temp.check(p_name text, p_ok boolean, p_detail text DEFAULT NULL)
RETURNS void LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO pg_temp.test_result (name, ok, detail) VALUES (p_name, coalesce(p_ok, false), p_detail);
  RAISE NOTICE '% %', CASE WHEN coalesce(p_ok, false) THEN 'PASS' ELSE 'FAIL' END,
    p_name || coalesce(' — ' || p_detail, '');
END $$;
```

# ПРИЛОЖЕНИЕ В.
# ЛИСТИНГ КЛИЕНТСКОГО ПРИЛОЖЕНИЯ

Клиентское приложение состоит из файлов каталога client/src (таблица В.1) и файла сборки CMakeLists.txt; формы, созданные в Qt Designer (файлы .ui), не приводятся. Ниже приведены ключевые объекты в сокращении (листинги В.1–В.16), общие шаблоны — в приложении А.

Таблица В.1 – Состав исходного кода клиента (каталог client/src)

| **Файл** | **Строк** | **Объекты** |
|---|---|---|
| app/AppConfig.h/.cpp | 64 | ConnectionParams, AppConfig |
| app/DevOptions.h/.cpp | 60 | DevOptions |
| app/RoleConfig.h/.cpp | 212 | NavLink, NavGroup, Section, RoleConfig |
| charts/ReportChart.h/.cpp | 235 | ReportChart |
| db/ApiCall.h/.cpp | 129 | ApiParam, ApiCall |
| db/Db.h/.cpp | 163 | Whoami, Db |
| export/ExcelExport.h/.cpp | 97 | ExcelExport |
| models/PagedTableModel.h/.cpp | 331 | ColumnSpec, TotalCell, PagedTableModel |
| ui/LoginDialog.h/.cpp | 123 | LoginDialog |
| ui/MainWindow.h/.cpp | 395 | MainWindow |
| ui/Theme.h | 27 | Theme |
| ui/dialogs/ContractDialog.h/.cpp | 164 | ContractDialog |
| ui/dialogs/PasswordDialogs.h/.cpp | 350 | ResetConfirmDialog, TempPasswordDialog, PasswordChangeDialog |
| ui/dialogs/PickerDialog.h/.cpp | 140 | PickerDialog |
| ui/dialogs/SubscriberDialog.h/.cpp | 87 | SubscriberDialog |
| ui/dialogs/UserDialog.h/.cpp | 188 | UserDialog |
| ui/pages/ContractsPage.h/.cpp | 179 | ContractsPage |
| ui/pages/DbAdminPages.h/.cpp | 413 | ChangeLogPage, DbTablesPage, DbConnectionsPage |
| ui/pages/HomePage.h/.cpp | 412 | HomePage |
| ui/pages/ListPages.h/.cpp | 161 | SharedSubscribersPage, IdleContractsPage |
| ui/pages/PlaceholderPage.h/.cpp | 46 | PlaceholderPage |
| ui/pages/ProvidersPage.h/.cpp | 246 | ProvidersPage, MyProviderPage |
| ui/pages/ReportPage.h/.cpp | 630 | ReportPage |
| ui/pages/SessionsPage.h/.cpp | 226 | SessionsPage |
| ui/pages/SubscribersPage.h/.cpp | 264 | SubscribersPage |
| ui/pages/TariffUsagePage.h/.cpp | 130 | TariffUsagePage |
| ui/pages/UsersPage.h/.cpp | 262 | UsersPage |
| ui/widgets/DataGrid.h/.cpp | 198 | DataGrid |
| ui/widgets/FormKit.h/.cpp | 238 | FormKit, CommandBar |
| ui/widgets/PagerBar.h/.cpp | 144 | PagerBar |
| main.cpp | 193 | функция main |

Листинг В.1 – Точка входа: вход, смена временного пароля, главное окно (main)

```
int main(int argc, char *argv[])
{
  QApplication app(argc, argv);

QApplication::setApplicationName(QStringLiteral("СуперБД"));

QApplication::setWindowIcon(QIcon(QStringLiteral(":/icons/app.svg")));

QLocale::setDefault(QLocale(QLocale::Russian, QLocale::Russia));
  applyStyle(app);
  …
  if (!QSqlDatabase::isDriverAvailable(QStringLiteral("QPSQL"))) {
    …
    return 1;
  …
  for (;;) {
    …
    if (!dev.screenshotLogin.isEmpty()) {
      LoginDialog dialog(config);
      …
      if (!dev.loginError.isEmpty())
      …
      scheduleScreenshot(&dialog, dev.screenshotLogin, 300);
      return app.exec();
    …
    if (!dev.devAs.isEmpty()) {
      if (!Db::open(config.connection, dev.devUser, QString(), &error)
        …

QMessageBox::critical(nullptr, QStringLiteral("СуперБД"), error);
        return 1;
      …
      LoginDialog dialog(config);
      if (dialog.exec() != QDialog::Accepted)
        return 0;
      …
      if (!Db::loadWhoami(&me, &error)) {

QMessageBox::critical(nullptr, QStringLiteral("СуперБД"), error);
        Db::close();
    …
    if (me.mustChangePassword) {
      if (me.groupName == QLatin1String("grp_dba")) {

PasswordChangeDialog::showDbaNotice(nullptr);
        Db::close();
        return 0;
      …
      PasswordChangeDialog change(me.login, true);
      …
      if (!dev.screenshot.isEmpty()) {
        …

scheduleScreenshot(&change, dev.screenshot, 600);
        …
        Db::close();
        return code;
      …
      if (change.exec() != QDialog::Accepted) {
        Db::close();
        return 0;
      …
      if (!Db::loadWhoami(&me, &reloadError)) {
…
}
```

Листинг В.2 – Подключение к серверу (Db::open)

```
bool Db::open(const ConnectionParams &params, const QString &user, const QString &password, QString *error)
{
  close();
  QSqlDatabase db = QSqlDatabase::addDatabase(QStringLiteral("QPSQL"), kConnection);
  db.setHostName(params.host);
  db.setPort(params.port);

db.setDatabaseName(params.database);
  db.setUserName(user);
  db.setPassword(password);

db.setConnectOptions(QStringLiteral("connect_timeout=5;application_name=superbd-client"));
  if (!db.open()) {
    if (error)
      *error = userMessage(db.lastError());
    close();
    return false;
  }
  return true;
}
```

Листинг В.3 – Сведения о пользователе (Db::loadWhoami)

```
bool Db::loadWhoami(Whoami *out, QString *error)
{
  QSqlQuery q(database());
  if (!ApiCall::select(QStringLiteral("whoami"), {}, &q, error))
    return false;
  if (!q.next()) {
    if (error)
      *error = QStringLiteral("Сервер не вернул сведения о пользователе");
    return false;
  }
  const QSqlRecord r = q.record();
  out->login = q.value(r.indexOf(QStringLiteral("login"))).toString();
  out->fullName = q.value(r.indexOf(QStringLiteral("full_name"))).toString();
  out->shortName = q.value(r.indexOf(QStringLiteral("short_name"))).toString();
  out->groupName = q.value(r.indexOf(QStringLiteral("group_name"))).toString();
  out->roleTitle = q.value(r.indexOf(QStringLiteral("role_title"))).toString();
  out->roleShort = q.value(r.indexOf(QStringLiteral("role_short"))).toString();
  out->providerName = q.value(r.indexOf(QStringLiteral("provider_name"))).toString();
  out->organization = q.value(r.indexOf(QStringLiteral("organization"))).toString();
  out->adminContact = q.value(r.indexOf(QStringLiteral("admin_contact"))).toString();
  out->databaseName = q.value(r.indexOf(QStringLiteral("database_name"))).toString();
  out->serverVersion = q.value(r.indexOf(QStringLiteral("server_version"))).toString();
  out->sessionStarted = q.value(r.indexOf(QStringLiteral("server_time"))).toDateTime().toLocalTime();
  out->mustChangePassword = q.value(r.indexOf(QStringLiteral("must_change_password"))).toBool();
  const QSqlDatabase db = database();
  out->connection.host = db.hostName();
  out->connection.port = db.port();
  out->connection.database = db.databaseName();
  return true;
}
```

Листинг В.4 – Сообщение об ошибке сервера (Db::userMessage)

```
QString Db::userMessage(const QSqlError &error)
{
  QString text = error.databaseText().trimmed();
  if (text.isEmpty())
    text = error.driverText().trimmed();
  text = text.section(QLatin1Char('\n'), 0, 0);
  static const QStringList levels = {QStringLiteral("ОШИБКА:"), QStringLiteral("ВАЖНО:"), QStringLiteral("ПАНИКА:"), QStringLiteral("ERROR:"), QStringLiteral("FATAL:"), QStringLiteral("PANIC:")};
  for (const QString &level : levels) {
    if (text.startsWith(level)) {
      text = text.mid(level.size()).trimmed();
      break;
    }
  }
  if (text.endsWith(QLatin1Char(')')) && text.size() >= 7 && text.at(text.size() - 7) == QLatin1Char('(')) {
    const QStringView code = QStringView(text).sliced(text.size() - 6, 5);
    bool isCode = true;
    for (const QChar ch : code)
      isCode = isCode && ((ch >= QLatin1Char('0') && ch <= QLatin1Char('9')) || (ch >= QLatin1Char('A') && ch <= QLatin1Char('Z')));
    if (isCode)
      text = text.left(text.size() - 7).trimmed();
  }
  return text.isEmpty() ? QStringLiteral("Неизвестная ошибка базы данных") : text;
}
```

Листинг В.5 – Разделы главного окна по группе (RoleConfig::sectionsFor)

```
QList<Section> RoleConfig::sectionsFor(const QString &group)
{
  const QString subscribersTitle = QStringLiteral("Абоненты и\ nдоговоры");
  if (group == QLatin1String("grp_operator"))
    return {home(),

{QStringLiteral("subscribers"), subscribersTitle, QStringLiteral("card"), QStringLiteral("subscribers"), subscribersNav(true)},
        {QStringLiteral("sessions"), QStringLiteral("Сеансы связи"), QStringLiteral("activity"), QStringLiteral("sessions"), sessionsNav(true)},

{QStringLiteral("my_provider"), QStringLiteral("Мой провайдер"), QStringLiteral("building"), QStringLiteral("my_provider"), myProviderNav()}};
  if (group == QLatin1String("grp_manager"))
    return {home(),

{QStringLiteral("subscribers"), subscribersTitle, QStringLiteral("card"), QStringLiteral("subscribers"), subscribersNav(false)},
        {QStringLiteral("sessions"), QStringLiteral("Сеансы связи"), QStringLiteral("activity"), QStringLiteral("sessions"), sessionsNav(false)},
        {QStringLiteral("stats"), QStringLiteral("Статистика"), QStringLiteral("chart"), QStringLiteral("report_revenue"), statsNav()}};
  if (group == QLatin1String("grp_admin"))
    return {home(),
        {QStringLiteral("admin"), QStringLiteral("Администри-\ nрование"), QStringLiteral("users"), QStringLiteral("users"), adminNav()},
        {QStringLiteral("refs"), QStringLiteral("Справочники"), QStringLiteral("book"), QStringLiteral("tariffs"), adminNav()},

{QStringLiteral("providers"), QStringLiteral("Провайдеры"), QStringLiteral("building"), QStringLiteral("providers"), adminNav()}};
  if (group == QLatin1String("grp_chief"))
    return {home(),

{QStringLiteral("providers"), QStringLiteral("Провайдеры"), QStringLiteral("building"), QStringLiteral("providers"), chiefProvidersNav()},

{QStringLiteral("subscribers"), subscribersTitle, QStringLiteral("card"), QStringLiteral("subscribers"), subscribersNav(false)},
        {QStringLiteral("sessions"), QStringLiteral("Сеансы связи"), QStringLiteral("activity"), QStringLiteral("sessions"), sessionsNav(false)},
        {QStringLiteral("stats"), QStringLiteral("Статистика"), QStringLiteral("chart"), QStringLiteral("report_revenue"), statsNav()}};
  if (group == QLatin1String("grp_dba"))
    return {home(),
        {QStringLiteral("admin"), QStringLiteral("Администри-\ nрование"), QStringLiteral("users"), QStringLiteral("users"), dbaNav()},
        {QStringLiteral("db"), QStringLiteral("База данных"), QStringLiteral("table"), QStringLiteral("db_tables"), dbaNav()}};
  return {home()};
}
```

Листинг В.6 – Создание страницы формы (MainWindow::createPage)

```
QWidget *MainWindow::createPage(const QString &form)
{
  if (form == QLatin1String("home")) {
    auto *home = new HomePage(m_me, ui->pages);
    connect(home, &HomePage::openFormRequested, this, &MainWindow::openForm);
    connect(home, &HomePage::openSectionRequested, this, &MainWindow::openSection);
    return home;
  …
  if (form == QLatin1String("subscribers")) {
    …
    connect(page, &SubscribersPage::openFormRequested, this, &MainWindow::openFormWithArgs);
    return page;
  …
  if (form == QLatin1String("contracts"))
    return new ContractsPage(m_me, readOnly, ui->pages);
  if (form == QLatin1String("sessions"))
    return new SessionsPage(m_me, readOnly, ui->pages);
  if (form == QLatin1String("providers")) {
    …
    connect(page, &ProvidersPage::openFormRequested, this, &MainWindow::openFormWithArgs);
    return page;
  …
  if (form == QLatin1String("my_provider"))
    return new MyProviderPage(m_me, ui->pages);
  if (form == QLatin1String("report_traffic"))
    return new TariffUsagePage(m_me, ui->pages);
  if (form == QLatin1String("report_revenue"))
    return new ReportPage(m_me, ReportPage::Revenue, ui->pages);
  if (form == QLatin1String("report_new_contracts"))
    return new ReportPage(m_me, ReportPage::NewContracts, ui->pages);
  if (form == QLatin1String("report_top_subscribers"))
    return new ReportPage(m_me, ReportPage::TopSubscribers, ui->pages);
  if (form == QLatin1String("users"))
    return new UsersPage(m_me, ui->pages);
  if (form == QLatin1String("shared_subscribers"))
    return new SharedSubscribersPage(ui->pages);
  if (form == QLatin1String("idle_contracts"))
    return new IdleContractsPage(m_me, ui->pages);
  if (form == QLatin1String("change_log"))
    return new ChangeLogPage(ui->pages);
  if (form == QLatin1String("db_tables"))
    return new DbTablesPage(m_me, ui->pages);
  if (form == QLatin1String("db_connections"))
    return new DbConnectionsPage(ui->pages);
  …
}
```

Листинг В.7 – Вход в программу (LoginDialog::tryLogin)

```
void LoginDialog::tryLogin()
{
  const QString user = ui->userEdit->text().trimmed();
  if (user.isEmpty()) {
    QMessageBox::warning(this, windowTitle(), QStringLiteral("Укажите пользователя."));
    ui->userEdit->setFocus();
    return;
  }
  ConnectionParams params;
  params.host = ui->hostEdit->text().trimmed();
  params.port = ui->portEdit->text().toInt();
  params.database = ui->databaseEdit->text().trimmed();

  QString error;
  Whoami me;
  if (!Db::open(params, user, ui->passwordEdit->text(), &error) || !Db::loadWhoami(&me, &error)) {
    Db::close();
    showError(error);
    ui->passwordEdit->selectAll();
    ui->passwordEdit->setFocus();
    return;
  }
  ui->passwordEdit->clear();
  hideError();

  m_config.connection = params;
  m_config.lastLogin = user;
  m_config.remember = ui->rememberCheck->isChecked();
  m_config.save();
  accept();
}
```

Листинг В.8 – Счётчики начальной страницы (HomePage::refresh)

```
void HomePage::refresh()
{
  QVBoxLayout *l = ui->todoLayout;
  clearLayout(l);
  QSqlQuery q;
  QString error;
  if (!ApiCall::select(QStringLiteral("home_todo"), {}, &q, &error)) {
    l->addWidget(label(error, "hint", this, true));
    return;
  }
  const QLocale ru(QLocale::Russian, QLocale::Russia);
  QDateTime asOf;
  while (q.next()) {
    const QSqlRecord r = q.record();
    auto *row = new QHBoxLayout;
    row->setSpacing(10);
    QPushButton *b = linkButton(q.value(r.indexOf(QStringLiteral("caption"))).toString(), false, this);
    const QString form = q.value(r.indexOf(QStringLiteral("form"))).toString();
    connect(b, &QPushButton::clicked, this, [this, form] { emit openFormRequested(form); });
    row->addWidget(b);
    row->addStretch(1);
    auto *value = label(ru.toString(q.value(r.indexOf(QStringLiteral("value"))).toLongLong()), nullptr, this);
    value->setProperty("strong", true);
    row->addWidget(value);
    l->addLayout(row);
    asOf = q.value(r.indexOf(QStringLiteral("as_of"))).toDateTime().toLocalTime();
  }
  l->addWidget(label(QStringLiteral("Данные на %1 · обновляются при открытии страницы")
          .arg(asOf.toString(QStringLiteral("dd.MM.yyyy HH:mm"))),
        "hint", this));
}
```

Листинг В.9 – Сброс пароля пользователя (UsersPage::resetPassword)

```
void UsersPage::resetPassword()
{
  const QString login = m_users->currentValue(QStringLiteral("login")).toString();
  if (login.isEmpty())
    return;
  const QString fullName = m_users->currentValue(QStringLiteral("full_name")).toString();
  ResetConfirmDialog confirm(login, fullName, m_users->currentValue(QStringLiteral("group_title")).toString(),
            m_users->currentValue(QStringLiteral("provider_name")).toString(), this);
  if (confirm.exec() != QDialog::Accepted)
    return;
  QSqlQuery q;
  QString error;
  if (!ApiCall::select(QStringLiteral("user_reset_password"), {{QStringLiteral("p_login"), login}}, &q, &error)
    || !q.next()) {
    FormKit::error(this, error.isEmpty() ? QStringLiteral("Сервер не вернул временный пароль") : error);
    return;
  }
  QString password = q.value(0).toString();
  const QDateTime validUntil = q.value(1).toDateTime();
  q = QSqlQuery();
  m_users->reload();

TempPasswordDialog::showOnce(this, login, fullName, password, validUntil);
}
```

Листинг В.10 – Запись нового пользователя (UserDialog::save)

```
bool UserDialog::save()
{
  if (m_created)
    return true;
  QSqlQuery q;
  QString error;
  const QList<ApiParam> params{{QStringLiteral("p_login"), m_login->text().trimmed()},

{QStringLiteral("p_full_name"), m_fullName->text().trimmed()},

{QStringLiteral("p_group"), m_group->currentData()},

{QStringLiteral("p_provider_id"), m_providerId},

{QStringLiteral("p_can_login"), m_canLogin->isChecked()}};
  if (!ApiCall::select(QStringLiteral("user_create"), params, &q, &error) || !q.next()) {
    FormKit::error(this, error.isEmpty() ? QStringLiteral("Сервер не вернул временный пароль") : error);
    return false;
  }
  QString temp = q.value(QStringLiteral("temp_password")).toString();
  const QDateTime validUntil = q.value(QStringLiteral("valid_until")).toDateTime();
  q = QSqlQuery();
  m_created = true;

TempPasswordDialog::showOnce(this, m_login->text().trimmed(), m_fullName->text().trimmed(), temp, validUntil);

setWindowTitle(QStringLiteral("Пользователь — СуперБД"));
  for (QWidget *w : {static_cast<QWidget *>(m_login), static_cast<QWidget *>(m_fullName),
        static_cast<QWidget *>(m_group), static_cast<QWidget *>(m_canLogin)})
    w->setEnabled(false);
  m_provider->parentWidget()->setEnabled(false);
  m_save->setEnabled(false);
  m_saveClose->setText(QStringLiteral("Закрыть"));
  return true;
}
```

Листинг В.11 – Смена пароля (PasswordChangeDialog::save)

```
void PasswordChangeDialog::save()
{
  if (m_new->text() != m_repeat->text()) {
    FormKit::error(this, QStringLiteral("Пароли не совпадают."));
    m_repeat->setFocus();
    return;
  }
  QString error;
  if (!ApiCall::exec(QStringLiteral("change_own_password"),
      {{QStringLiteral("p_new"), m_new->text()}}, &error)) {
    FormKit::error(this, error);
    m_new->setFocus();
    m_new->selectAll();
    return;
  }
  m_new->clear();
  m_repeat->clear();
  accept();
}
```

Листинг В.12 – Доли провайдеров в выручке (ReportPage::buildShares)

```
void ReportPage::buildShares()
{
  …
  setHeading(QStringLiteral("Выручка провайдеров объединения %1").arg(byGrain()),
             QStringLiteral("все провайдеры; %1").arg(typeFilter()));
  …
  if (!ApiCall::select(QStringLiteral("report_provider_shares"), params, &q, &error)) {
    FormKit::error(this, error);
    return;
  }
  …
  setHeadings({QStringLiteral("Период"),
               QStringLiteral("Выручка, тыс. руб."),
               QStringLiteral("Доля, %")});
  …
  while (q.next()) {
    …
    if (period.isNull()) {
      …
      if (!periods.contains(d))
        …
      if (!series.contains(ord))
        …
  std::sort(periods.begin(), periods.end());
  …
  for (const QDate &d : periods)
    categories << periodLabel(d);
  …
  for (auto it = series.begin(); it != series.end(); ++it) {
    …
    for (const QDate &d : periods)
  …
  if (!categories.isEmpty())
…
}
```

Листинг В.13 – Диаграмма отчёта (ReportChart::setData)

```
void ReportChart::setData(Kind kind, const QString &title, const QStringList &categories, const QList<Series> &series)
{
  auto *chart = new QChart;
  chart->setTitle(title);
  …
  styleAxis(axisValues);
  …
  styleAxis(axisCategories);
  …
  if (kind == Lines) {
    …
    for (const Series &s : series) {
      …
      QPen pen(s.color);
      …
      for (int i = 0; i < s.values.size(); ++i) {
    …
    if (kind == HorizontalBars)
      …
    QList<double> totals(categories.size(), 0.0);
    for (const Series &s : series) {
      …
      for (int i = 0; i < categories.size(); ++i) {
    …
    if (kind == HorizontalBars) {
  …
  fillLegend(series);
}
```

Листинг В.14 – Выгрузка отчёта в Excel (ExcelExport::save)

```
bool save(QWidget *parent,
const QString &suggestedName,
const QStringList &heading,
const QAbstractItemModel
*model)
{
  const QString dir =
QStandardPaths::writableLocatio
n(QStandardPaths::DocumentsLo
cation);
  QString path =
QFileDialog::getSaveFileName(p
arent,
QStringLiteral("Сохранить в
Excel"),
                    …

QStringLiteral("Книга Excel
(*.xlsx)"));
  if (path.isEmpty())
    return false;
  if (!
path.endsWith(QStringLiteral(".x
lsx"), Qt::CaseInsensitive))
  …
  for (int i = 0; i < heading.size();
++i)
  …
  for (int c = 0; c < columns; ++c)
{
  …
  for (int r = 0; r < model-
>rowCount(); ++r, ++row) {
    …
    for (int c = 0; c < columns; +
+c) {
      …
      if (!v.isValid())
      …
      if (v.typeId() ==
QMetaType::Double)
  …
  if (!xlsx.saveAs(path)) {
    FormKit::error(parent,
QStringLiteral("Не удалось
сохранить файл
%1").arg(QDir::toNativeSeparato
rs(path)));
    return false;
  …
  FormKit::status(parent,
QStringLiteral("Отчёт сохранён:
%1").arg(QDir::toNativeSeparato
rs(path)));
  return true;
}
```

Листинг В.15 – Журнал изменений: поля записи (ChangeLogPage::showFields)

```
void
ChangeLogPage::showFields()
{
  m_fieldsModel->clear();
  m_fieldsModel-
>setHorizontalHeaderLabels({QS
tringLiteral("Поле"),
QStringLiteral("Было"),
QStringLiteral("Стало")});
  …
  if (!id.isValid() || id.isNull()) {
                …
                :
QStringLiteral("Выберите
запись журнала"));
    return;
  …
  if (!
ApiCall::select(QStringLiteral("q
_change_log_fields"),
{{QStringLiteral("p_id"), id}},
&q, &error)) {
    FormKit::error(this, error);
    return;
  …
  while (q.next()) {
    …
    for (const char *field : {"field",
"old_value", "new_value"}) {
      …
      items << new
QStandardItem(v.isNull() ?
QString() : v.toString());
    …
    if
(q.value(r.indexOf(QStringLiteral
("changed"))).toBool())
      for (QStandardItem *item :
items)
…
}
```

Листинг В.16 – Файл сборки (CMakeLists.txt)

```
cmake_minimum_required(VERSION 3.21)
project(superbd_client VERSION 0.1 LANGUAGES CXX)

# Тонкий клиент СуперБД: Qt 6 Widgets + QPSQL. Интерфейс — строго по макету docs/ui/ (ui-spec.md).
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
…
find_package(Qt6 6.5 REQUIRED COMPONENTS Core Gui Widgets Sql Svg Charts PrintSupport)
…
find_package(QXlsxQt6 REQUIRED)
qt_standard_project_setup()
…
set(CMAKE_AUTOUIC_SEARCH_PATHS ${CMAKE_CURRENT_SOURCE_DIR}/src/ui ${CMAKE_CURRENT_SOURCE_DIR}/src/ui/pages)
…
option(SUPERBD_DEV_TOOLS "Ключи --dev-as и --screenshot для разработки" OFF)
…
target_include_directories(superbd PRIVATE src)
…
target_include_directories(superbd PRIVATE src/export/compat)
…
if(SUPERBD_DEV_TOOLS)

target_compile_definitions(superbd PRIVATE SUPERBD_DEV_TOOLS)
endif()
```

# ПРИЛОЖЕНИЕ Г.
# РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

## Г.1 Назначение программы и требования

Программа «СуперБД» предназначена для учёта услуг связи интернет-провайдеров объединения: просмотра абонентов, договоров и сеансов связи, анализа выручки и трафика. Для работы нужен компьютер с Windows 10 или 11, папка программы (исполняемый файл superbd.exe с библиотеками Qt и PostgreSQL) и учётная запись, выданная администратором приложения. Программа подключается к серверу базы данных объединения по сети.

## Г.2 Вход в программу

При запуске открывается окно «Доступ к информационной базе» (рисунок Г.1). Введите имя пользователя (логин) и пароль, выданные администратором. Кнопка «Параметры подключения» раскрывает поля «Сервер», «Порт» и «База данных» — их сообщает администратор, обычно менять их не нужно. Флажок «Запомнить сервер, базу и пользователя» сохраняет эти значения на компьютере; пароль никогда не сохраняется. Нажмите «ОК».

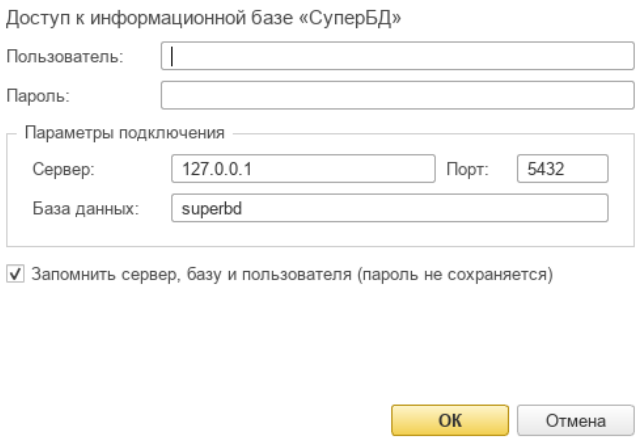



Рисунок Г.1 – Окно входа

Если вход не удался, под полями выводится причина: неверный пароль, запрещённый вход или истёкший срок временного пароля. После сброса пароля администратором первый вход выполняется с временным паролем, и программа

сразу открывает форму «Смена пароля» (рисунок Г.2): введите новый пароль дважды — не короче 8 символов и не совпадающий с логином — и нажмите «Сохранить и продолжить». Свой пароль можно сменить и позже по ссылке «Сменить пароль» начальной страницы.

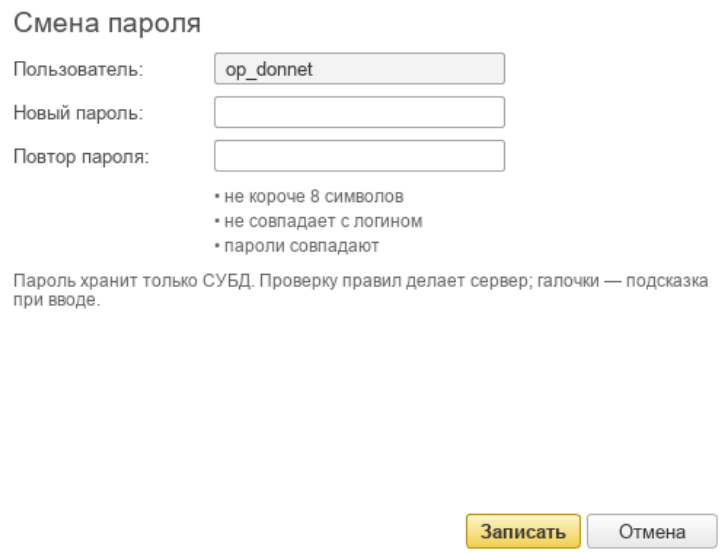



Рисунок Г.2 – Форма «Смена пароля»

### Г.3 Главное окно

Главное окно состоит из панели разделов слева, панели навигации раздела, вкладок открытых форм и строки состояния (рисунок Г.3). Состав разделов зависит от роли: оператор работает с разделами «Абоненты и договоры», «Сеансы связи», «Мой провайдер»; руководитель — дополнительно со «Статистикой»; главный аналитик — с разделами «Провайдеры», «Абоненты и договоры», «Сеансы связи», «Статистика». Руководитель и главный аналитик только просматривают данные — в заголовке форм выводится пометка «(только просмотр)».

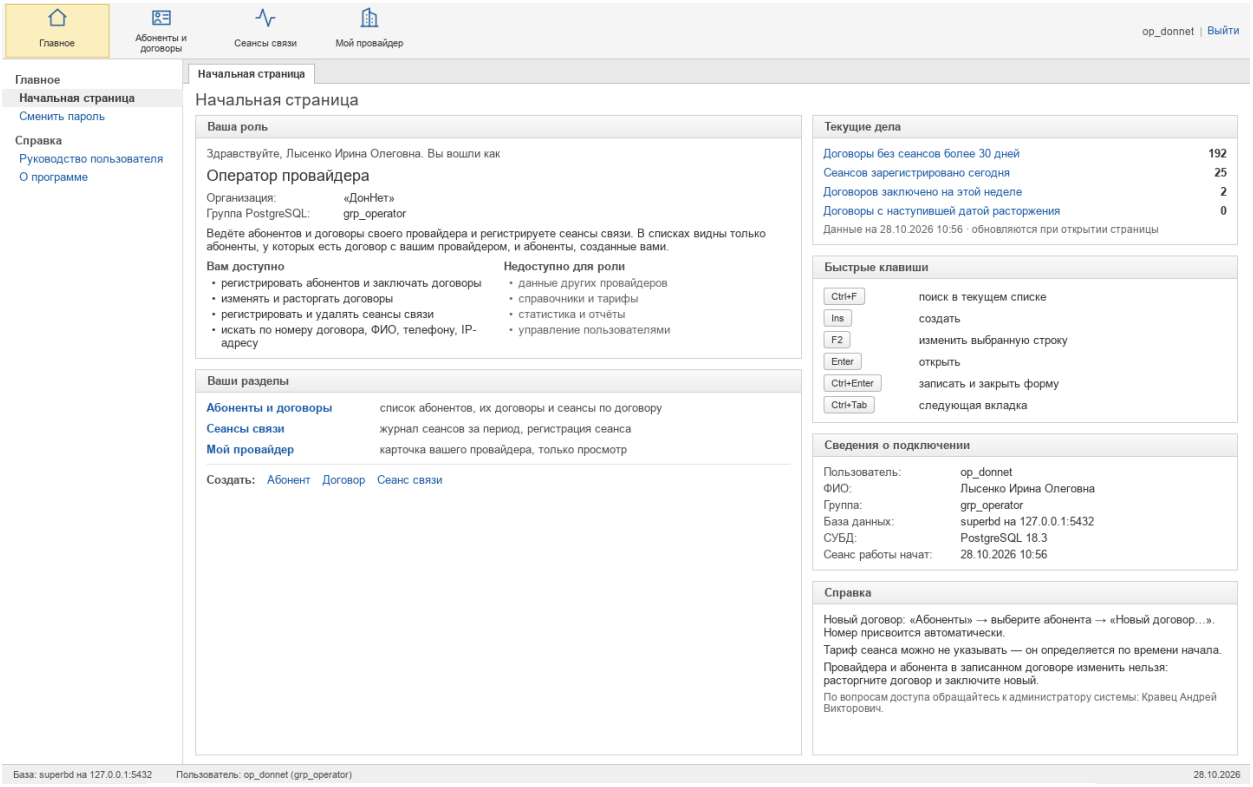



Рисунок Г.3 – Главное окно, начальная страница оператора

Начальная страница открыта всегда. Блок «Текущие дела» содержит счётчики (например, «Договоры без сеансов более 30 дней», «Сеансов зарегистрировано сегодня»); щелчок по счётчику открывает соответствующий список или отчёт. Открытые формы располагаются на вкладках, вкладку закрывает кнопка на её заголовке.

Во всех списках действуют общие правила:

– список выводится страницами; размер страницы (20, 100 или 200 записей) выбирается в поле «Показывать по», переход между страницами — кнопками и полем «Страница N из M» под таблицей;

– поиск выполняется по полю «Поиск» командной панели (Ctrl+F): по началу наименования, номера договора или цифрам телефона в любом написании, Enter — найти, «Отменить поиск» — показать всё;

– двойной щелчок по строке или Enter открывает карточку записи;

– итоговая строка «Итого…» считается сервером по всем отобранным записям, а не только по видимой странице.

### Г.4 Работа оператора

Раздел «Абоненты и договоры», форма «Абоненты» (рисунок Г.4). Верхняя таблица — абоненты, у которых есть договор с вашим провайдером. Отбор задаётся полями «Тип абонента» и флажком «Только с действующими договорами». При выборе абонента в средней таблице выводятся его договоры с вашим провайдером, при выборе договора в нижней — сеансы договора с итогами. Двойной щелчок по абоненту открывает карточку абонента, по договору — карточку договора с вкладкой «Сеансы».

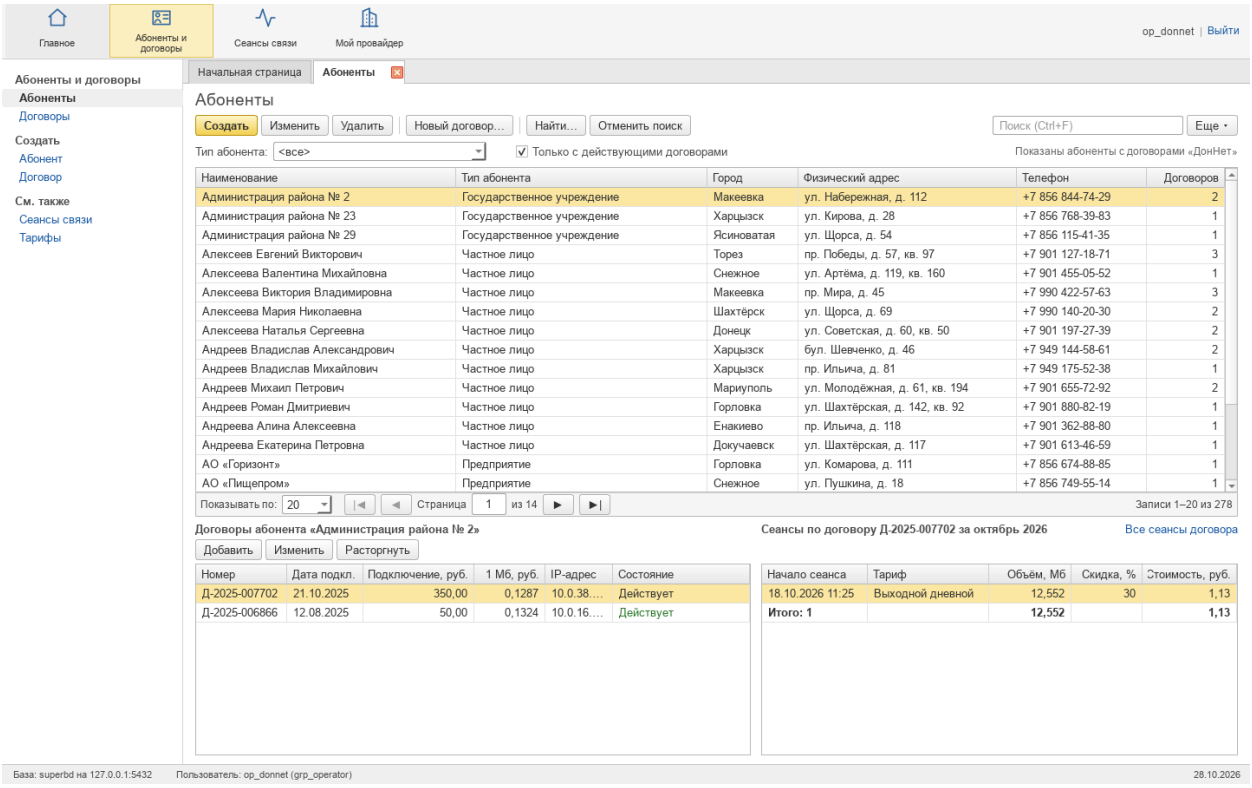



Рисунок Г.4 – Форма «Абоненты»

Форма «Договоры» выводит договоры, заключённые в периоде: задайте даты «с» и «по» (или выберите период в меню «…»), при необходимости выберите абонента кнопкой «…» и отметьте «Только действующие», затем нажмите «Применить».

Раздел «Сеансы связи», форма «Журнал сеансов» (рисунок Г.5): задайте период, при необходимости договор, IP-адрес или подсеть (например, 10.200.0.0/24) и тариф, нажмите «Применить». Строка «Итого за период»

показывает число сеансов, объём и стоимость по всему отбору. «Сбросить» возвращает отбор по умолчанию.

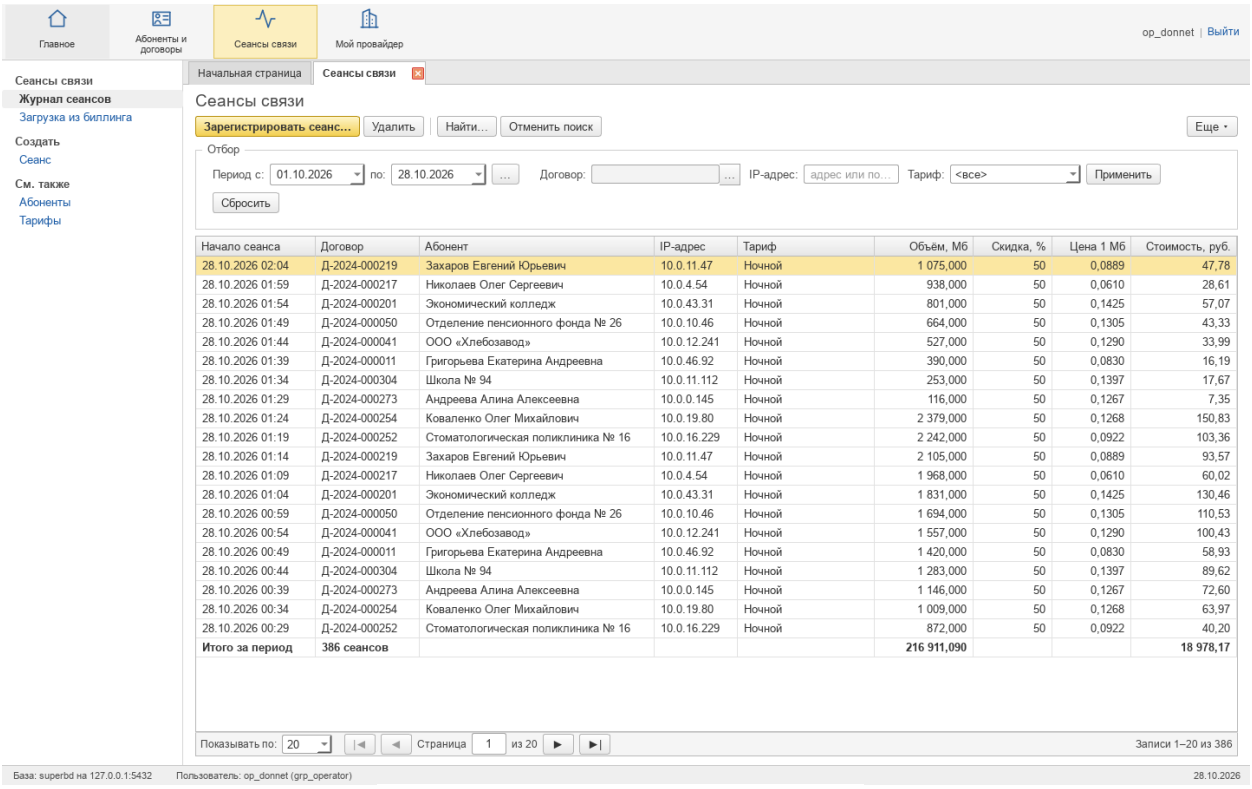



Рисунок Г.5 – Журнал сеансов

Раздел «Мой провайдер» показывает карточку вашего провайдера и список сотрудников. Список «Договоры без сеансов» открывается счётчиком начальной страницы; поле «Без сеансов более … дней» меняет границу простоя.

Создание абонента и договора, регистрация сеанса, изменение и расторжение договора в текущей версии программы недоступны: по нажатию соответствующих кнопок в строке состояния выводится сообщение о том, что форма будет добавлена на следующем этапе.

### Г.5 Работа руководителя провайдера и главного аналитика

Руководитель видит данные своего провайдера, главный аналитик — данные всех провайдеров; формы «Абоненты», «Договоры» и «Журнал сеансов» работают так же, как у оператора, без команд изменения. Главный аналитик дополнительно работает с разделом «Провайдеры»: список провайдеров с

числом действующих договоров и договоров, заключённых в периоде, и сотрудники выбранного провайдера.

Раздел «Статистика» содержит отчёты «Выручка по периодам», «Трафик по тарифам», «Новые договоры по типам абонентов» и «Крупнейшие абоненты» (рисунок Г.6). Порядок работы с отчётом:

1) Задайте период (поля дат или меню «…»: этот месяц, прошлый месяц, этот квартал, прошлый квартал, с начала года, прошлый год).

2) Выберите периодичность — месяц, квартал или год; главный аналитик может выбрать провайдера, для некоторых отчётов задаётся тип абонента или число абонентов.

3) Нажмите «Сформировать»: выводится табличный документ с итоговой строкой и диаграмма с легендой.

4) Для выгрузки нажмите «Сохранить в Excel…» и укажите имя файла; для печати — «Печать».

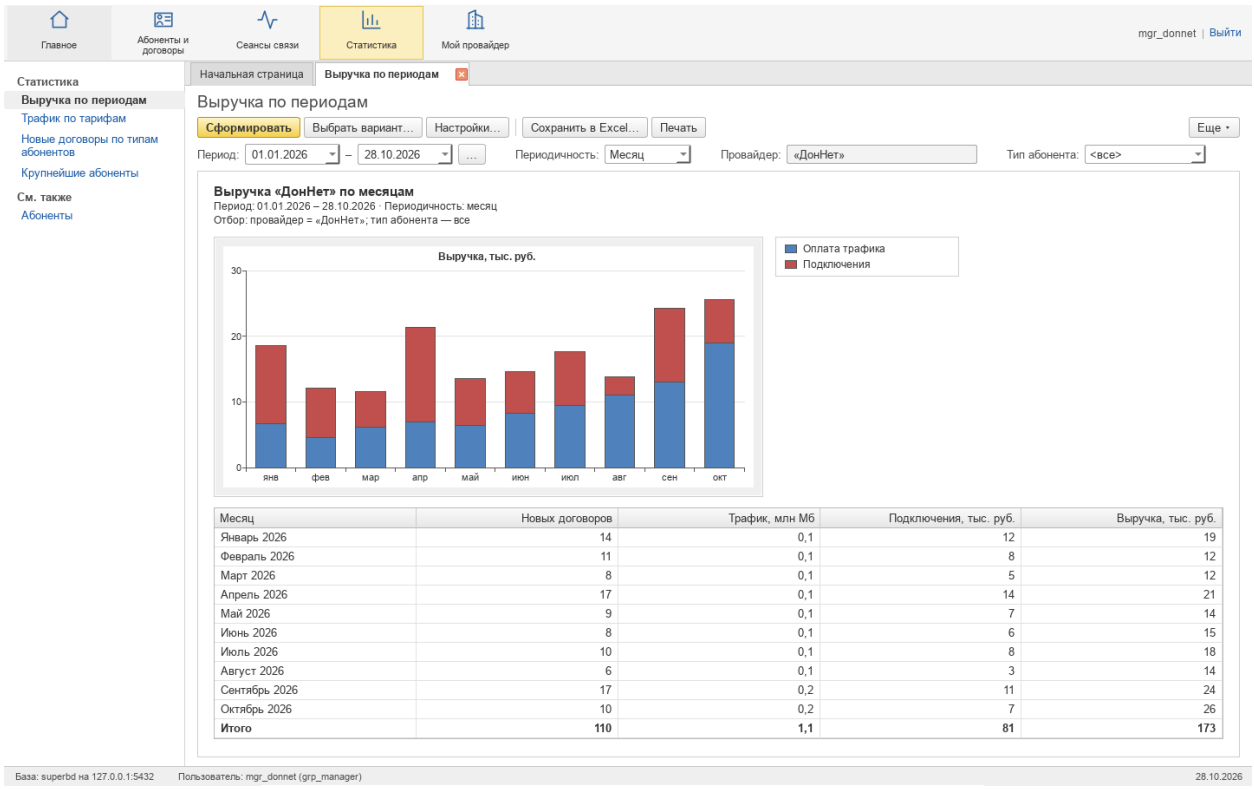



Рисунок Г.6 – Отчёт «Выручка по периодам»

У главного аналитика отчёт «Выручка по периодам» без отбора по провайдеру показывает доли крупнейших провайдеров и группы «Прочие».

### Г.6 Сообщения программы

Если действие недоступно вашей роли, программа выводит сообщение сервера «Недостаточно прав для этого действия». Сообщение «Вам выдан временный пароль: смените его, чтобы продолжить работу» означает, что нужно сменить пароль в форме «Смена пароля». При потере связи с сервером закройте программу и войдите повторно; если ошибка повторяется, обратитесь к администратору — его ФИО указаны в блоке «Справка» начальной страницы.

# ПРИЛОЖЕНИЕ Д.
# РУКОВОДСТВО СУПЕРПОЛЬЗОВАТЕЛЯ

## Д.1 Требования к серверу

Сервер базы данных — PostgreSQL 18 (допустима версия 16 и выше) на Windows или Linux, кодировка базы UTF-8. Рекомендуемые параметры сервера приведены в разделе 2 (таблица 2.1); для защиты паролей в журнале сервера установите log_parameter_max_length = 0. На компьютере администратора нужны программы psql и pgAdmin 4. Пароль суперпользователя postgres хранится в файле %APPDATA%\postgresql\pgpass.conf и в сценарии не записывается.

## Д.2 Развёртывание базы данных

1) Создайте базу и роль-владельца (один раз, суперпользователем из базы postgres): psql -U postgres -d postgres -v ON_ERROR_STOP=1 -f db/create_db.sql

2) Создайте объекты и данные (повторный запуск пересоздаёт схемы проекта вместе с данными; группы и учётные записи пользователей сохраняются): psql -U postgres -d superbd -v ON_ERROR_STOP=1 -f db/build.sql

3) Проверьте базу автоматическими тестами: psql -U postgres -d superbd -v ON_ERROR_STOP=1 -f db/tests/run_all.sql В конце прогона выводится «Тесты: пройдено 117 из 117»; при ошибке — номера непройденных тестов.

На Windows шаги 2 и 3 выполняет сценарий db\build_and_test.cmd: полный вывод записывается в db\build_and_test.log, в окне выводятся результаты тестов и итог (рисунок Д.1).

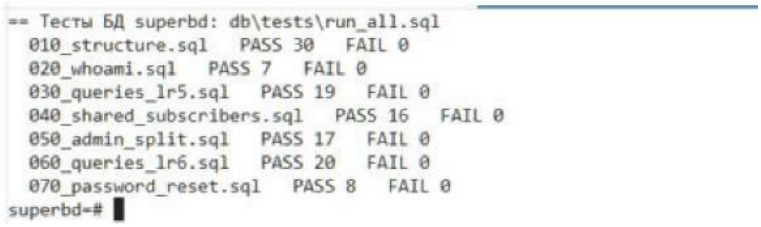



Рисунок Д.1 – Пересборка базы и прогон тестов

Демонстрационные учётные записи (op_donnet, mgr_donnet, chief_main, adm_main, dba_main) создаются сценарием db/seed/030_demo_users.sql без паролей; пароль задаётся командой \password <логин> в psql.

### Д.3 Учётные записи администраторов

Первого администратора БД создаёт суперпользователь по образцу сценария db/seed/030_demo_users.sql: роль с правом входа, членство в группе grp_dba и строка таблицы «Сотрудник» с ФИО; пароль задаётся командой \ password <логин>. Администратор БД входит в программу своим логином и в разделе «Администрирование» → «Пользователи» создаёт администраторов приложения командой «Создать» (выдаётся временный пароль на сутки), запрещает и разрешает им вход, сбрасывает пароль. Свой пароль администратор БД меняет командой \password в psql: программа такую смену не выполняет, а psql передаёт на сервер только хеш пароля.

Для изменения структуры базы администратор БД переключается на владельца объектов командой SET ROLE app_owner; без неё права владельца не действуют.

### Д.4 Работа в программе

Раздел «База данных» содержит формы «Таблицы» и «Подключения»:

– «Таблицы базы данных» (рисунок Д.2) — для каждой таблицы проекта назначение, оценка числа строк, размер с индексами, последняя очистка и сбор статистики, число последовательных и индексных чтений. Таблицы, по которым статистика не собиралась больше 7 дней, учитываются счётчиком начальной страницы; статистику собирает команда ANALYZE, очистку — VACUUM (право выдаётся встроенной ролью pg_maintain);

– «Подключения к базе» (рисунок Д.3) — текущие сеансы: пользователь, программа, адрес, начало сеанса, состояние и текущий запрос.

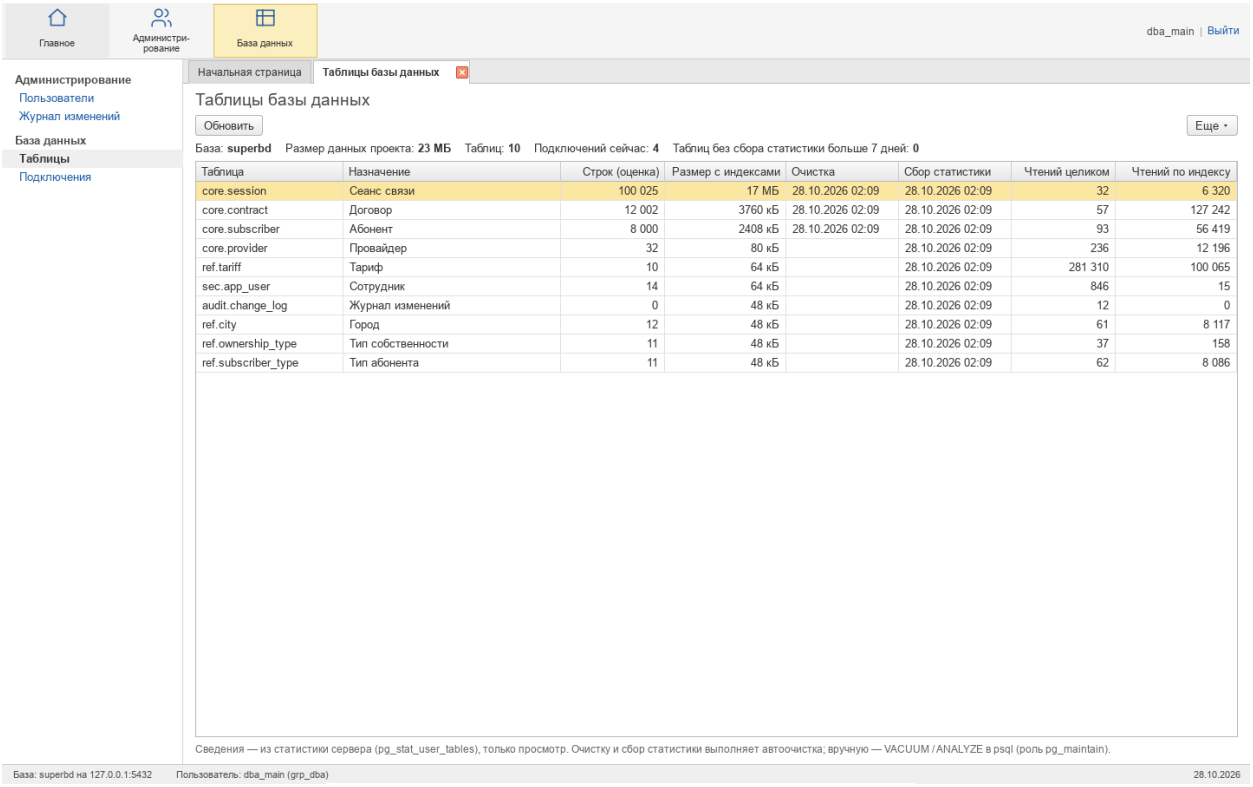



Рисунок Д.2 – Таблицы базы данных

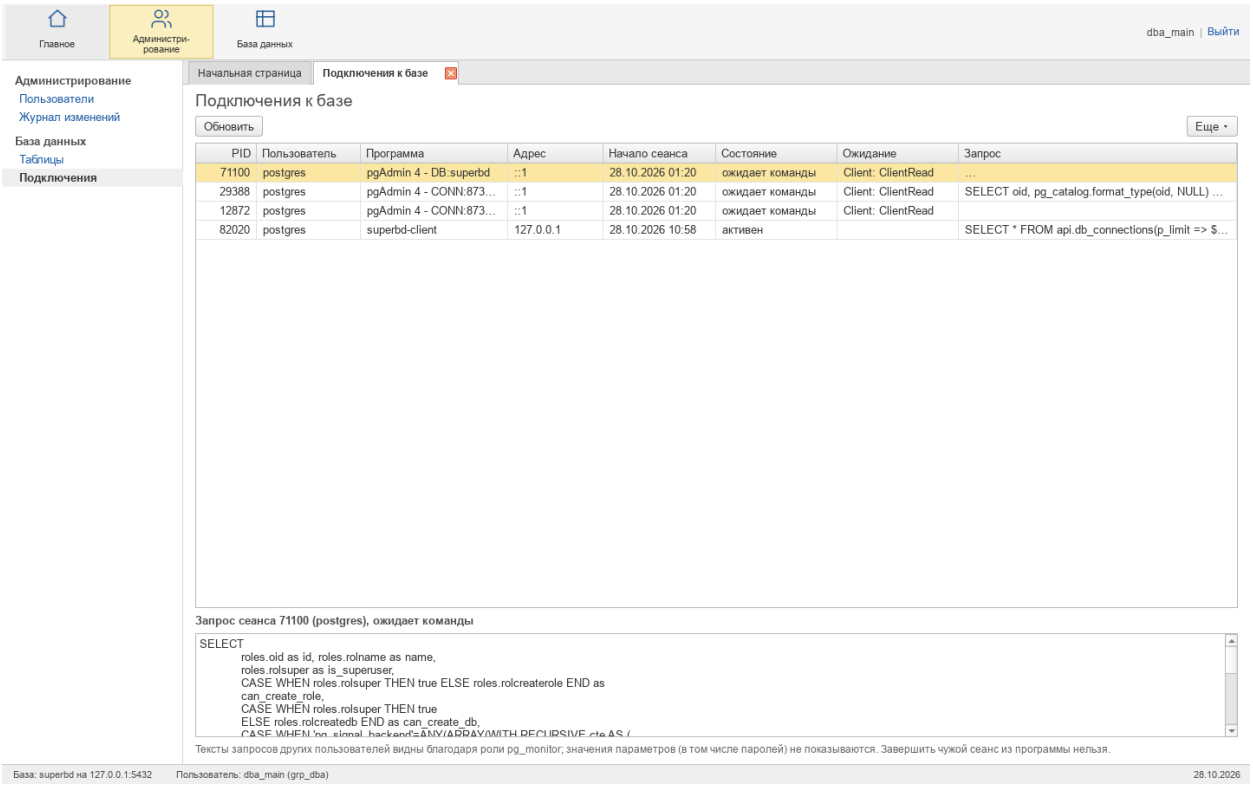



Рисунок Д.3 – Подключения к базе

Форма «Журнал изменений» раздела «Администрирование» выводит записи журнала за период с отбором по таблице, операции и пользователю; команда просмотра показывает изменённые поля записи «было / стало».

[ЗАГЛУШКА: журнал изменений пока не заполняется: записи должны добавлять триггеры таблиц (подраздел 4.3)]

### Д.5 Резервное копирование и восстановление

Администратор БД входит во встроенную роль pg_read_all_data и может выполнять резервное копирование без прав суперпользователя:

pg_dump -U <логин администратора БД> -h <сервер> -d superbd -Fc -f superbd_<дата>.backup

Резервная копия в формате custom (-Fc) содержит структуру и данные всех схем проекта. Копирование от имени администратора БД проверено 28.10.2026: копия рабочей базы — около 17 МБ. Восстановление выполняет суперпользователь в пустую базу, созданную сценарием create_db.sql:

pg_restore -U postgres -d superbd --clean --if-exists superbd_<дата>.backup

Группы и учётные записи пользователей — объекты кластера и в копию базы не входят; их список сохраняется командой pg_dumpall --roles-only суперпользователем.

# ПРИЛОЖЕНИЕ Е.
# РУКОВОДСТВО АДМИНИСТРАТОРА

## Е.1 Назначение роли

Администратор приложения ведёт учётные записи операторов, руководителей провайдеров и главных аналитиков, справочники, провайдеров и сведения общих абонентов. Абонентов, договоров и сеансов администратор не видит. Учётную запись администратора приложения создаёт администратор БД (приложение Д). Начальная страница администратора содержит счётчики «Пароли истекают до …», «Пользователи с запрещённым входом», «Провайдеры без сотрудников», «Пользователей системы» (рисунок Е.1).

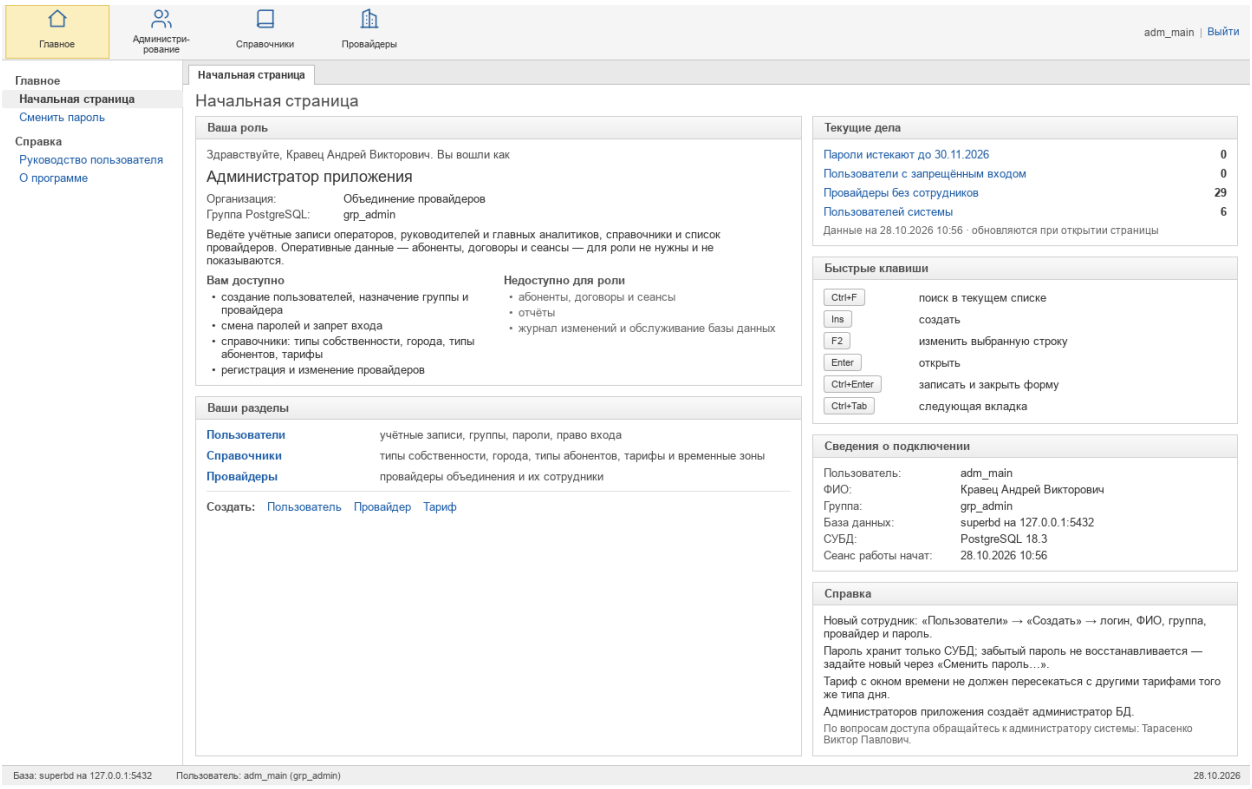



Рисунок Е.1 – Начальная страница администратора приложения

## Е.2 Пользователи

Раздел «Администрирование» → «Пользователи» (рисунок Е.2): слева — группы с числом пользователей, справа — пользователи выбранной группы

(логин, ФИО, группа, провайдер, «Вход разрешён», «Пароль действует до»; у временного пароля — пометка «(временный)»).

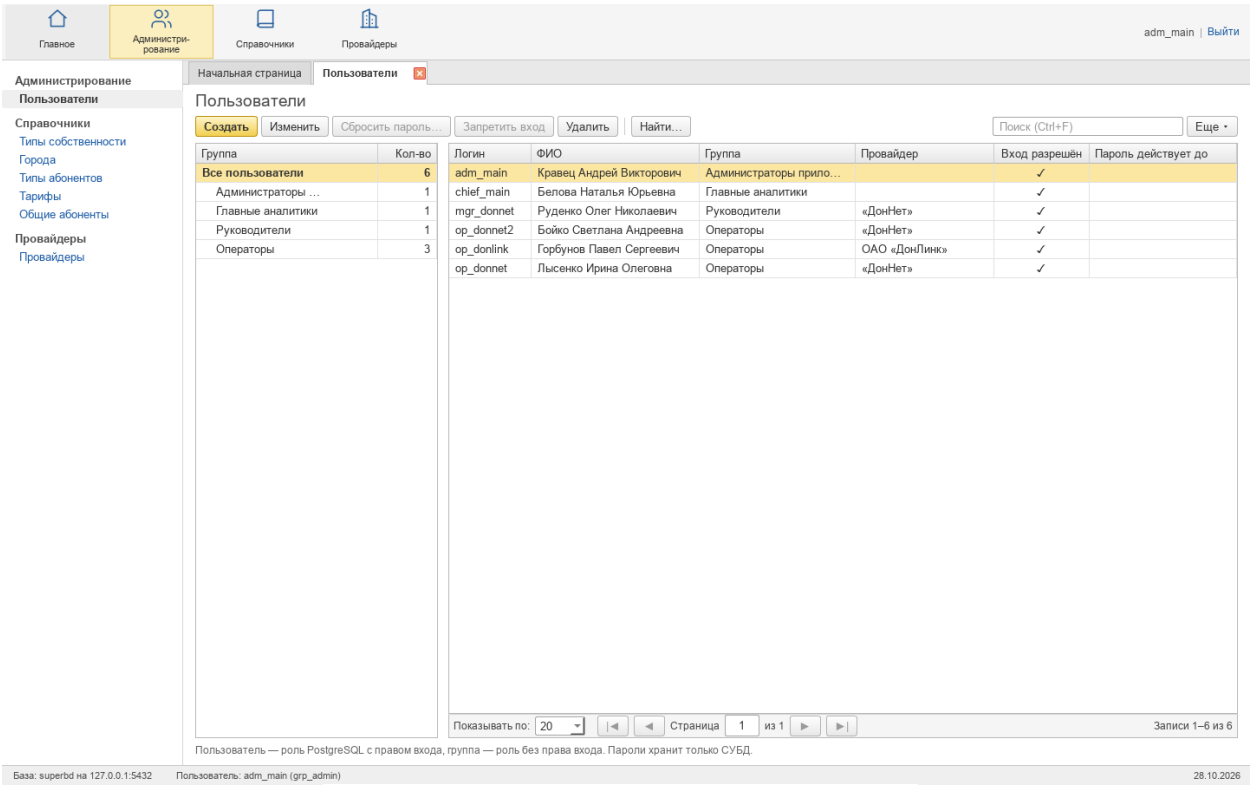



Рисунок Е.2 – Пользователи системы

Создание пользователя:

1) Нажмите «Создать» — откроется форма «Пользователь (создание)».

2) Введите логин (латинские строчные буквы, цифры и «_», от 3 до 31 символа, начинается с буквы) и ФИО.

3) Выберите группу: оператор, руководитель провайдера или главный аналитик. Для оператора и руководителя выберите провайдера кнопкой «…»; главному аналитику провайдер не указывается.

4) При необходимости снимите флажок «Вход разрешён» и нажмите «Записать и закрыть».

5) Программа покажет временный пароль, действующий сутки: кнопка «Копировать» помещает его в буфер обмена. Пароль показывается один раз — передайте его сотруднику; при первом входе сотрудник задаст свой пароль.

Команда «Запретить вход» блокирует учётную запись выбранного пользователя (данные сохраняются), «Разрешить вход» — снимает блокировку. Собственную учётную запись и учётные записи администраторов заблокировать нельзя.

Сброс пароля: выберите пользователя и нажмите «Сбросить пароль…», подтвердите сброс (рисунок Е.3). Сервер выдаёт новый временный пароль на сутки; до его смены сотрудник может только сменить пароль. Себе сбросить пароль нельзя — свой пароль меняется по ссылке «Сменить пароль» начальной страницы.

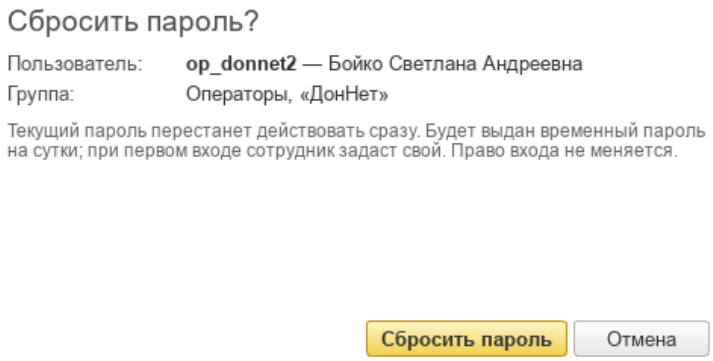



Рисунок Е.3 – Подтверждение сброса пароля

[ЗАГЛУШКА: команды «Изменить» и «Удалить» пользователя в текущей версии не реализованы (нет функций api), выводится сообщение в строке состояния]

### Е.3 Провайдеры

Раздел «Провайдеры» выводит всех провайдеров объединения с типом собственности, городом, адресом, телефоном, годом начала работы и числом договоров; в нижней таблице — сотрудники выбранного провайдера с группой и признаком входа. Флажок «Без сотрудников» (его включает и счётчик начальной страницы) оставляет провайдеров, которым ещё не назначены операторы и руководители (рисунок Е.4); назначьте им сотрудников в разделе «Пользователи».

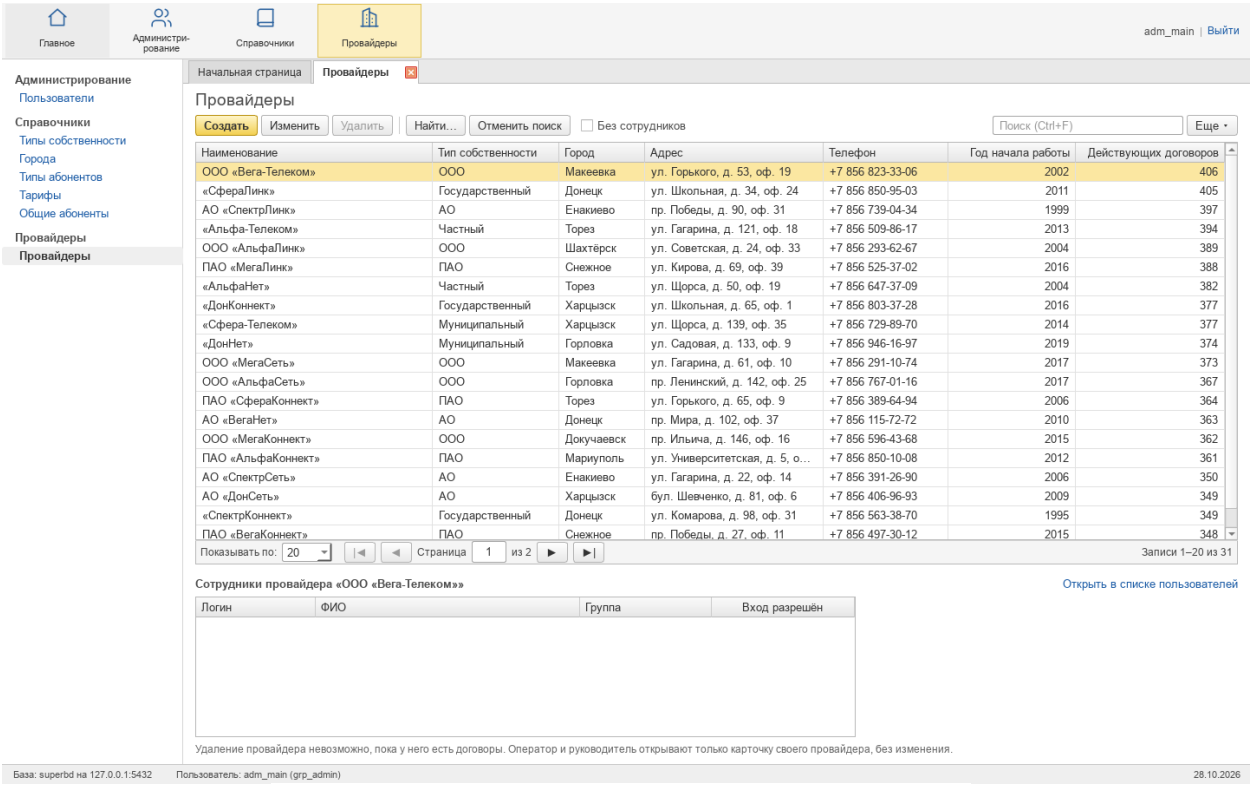



Рисунок Е.4 – Провайдеры без сотрудников

[ЗАГЛУШКА: создание, изменение и удаление провайдера в клиенте не реализованы]

### Е.4 Справочники и общие абоненты

Список «Общие абоненты» раздела «Справочники» выводит абонентов, у которых есть договоры не меньше чем с заданным числом провайдеров (поле «Провайдеров не меньше»). Сведения таких абонентов (наименование, тип, город, адрес, телефон) меняет только администратор приложения: оператор может менять лишь абонента, которого не обслуживают другие провайдеры.

[ЗАГЛУШКА: формы справочников (типы собственности, города, типы абонентов, тарифы) и изменение сведений общего абонента в клиенте не реализованы: открывается временная страница-заглушка. Описание добавить после реализации]